AtomVM Documentation

Release 0.6.0-alpha.1+git.aa2ee88b

© 2017-2023, AtomVM

October 01, 2023

Table of Contents

1 Welcome to AtomVM! 3
1.1 What is AtomVM? . . . 3
1.2 Why Erlang/Elixir? . . . 3
1.3 Design Philosophy . . . 4
1.4 Licensing . . . 4
1.5 Source Code. . . . 5
1.6 Contributing . . . 5
1.7 Where to go from here. . . . 5

2 Release Notes 7
2.1 Required Software . . . 7

3 Getting Started Guide 9
3.1 Getting Started on the ESP32 platform . . . 9
3.2 Getting Started on the STM32 platform . . . 13
3.3 Getting Started on the Raspberry Pi Pico platform . . . 14
3.4 Getting Started on the Generic UNIX platform . . . 15
3.5 Where to go from here . . . 15
3.6 Getting Started with AtomVM WebAssembly port. . . . 16
3.7 Where to go from here . . . 16

4 Programmers Guide 17
4.1 AtomVM Features . . . 17
4.2 AtomVM Development . . . 18
4.3 Applications. . . . 20
4.4 Core APIs . . . 22
4.5 ESP32-specific APIs. . . . 35
4.6 Peripherals. . . . 38
4.7 Protocols . . . 44

5 Example Programs 51
5.1 Erlang Examples . . . 51
5.2 ESP32 Examples. . . . 54
5.3 Flashing AtomVM Examples for ESP32 . . . 54

6 Network Programming Guide 61
6.1 Station (STA) mode . . . 61
6.2 AP mode . . . 63
6.3 STA+AP mode. . . . 65
6.4 SNTP Support . . . 65
6.5 NVS Credentials . . . 66

6.6 Stopping the Network . . . 66

7 Build Instructions **67**
7.1 Downloading AtomVM . . . 67
7.2 Source code organization . . . 67
7.3 Building for Generic UNIX . . . 68
7.4 Building for ESP32 . . . 70
7.5 Building for STM32 . . . 77
7.6 Building for Raspberry Pi Pico . . . 78
7.7 Building for NodeJS/Web . . . 79

8 AtomVM Internals **81**
8.1 What is an Abstract Machine? . . . 81
8.2 AtomVM Data Structures . . . 82
8.3 Runtime Execution Loop . . . 83
8.4 Module Loading . . . 83
8.5 Function Calls and Return Values . . . 83
8.6 Exception Handling . . . 83
8.7 The Scheduler . . . 83
8.8 Mailboxes and signals . . . 84
8.9 Stacktraces . . . 84
8.10 AtomVM WebAssembly port . . . 85

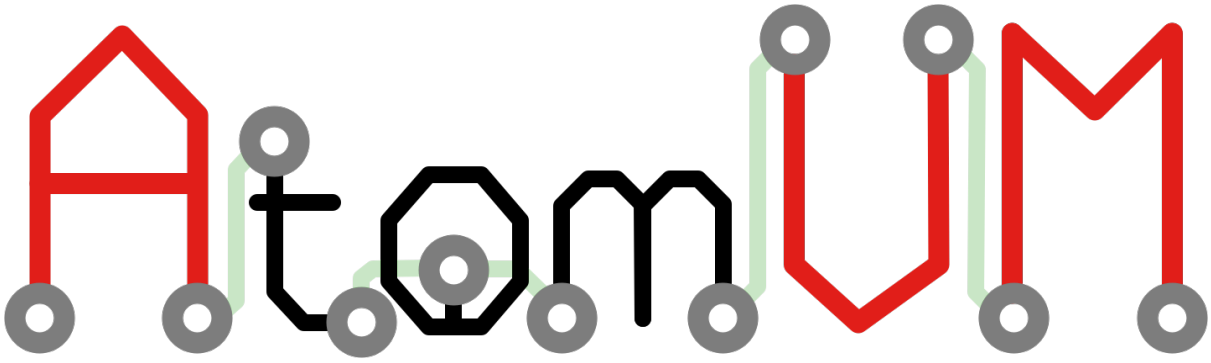
9 Memory Management **87**
9.1 The Context structure . . . 87
9.2 Simple Terms . . . 90
9.3 Boxed terms . . . 92
9.4 Lists . . . 97
9.5 Special Stack Types . . . 98
9.6 Garbage Collection . . . 99

10 Packbeam Format **105**
10.1 Overview . . . 105
10.2 Packbeam Header . . . 105
10.3 File encodings . . . 106

11 API Reference Documentation **109**
11.1 Erlang Libraries . . . 109
11.2 AtomVM 'C' Internal Libraries . . . 200

12 Contributing **201**
12.1 Git Recommended Practices . . . 201
12.2 Coding Style . . . 201

13 Changelog **205**
13.1 [0.6.0-alpha.1] - Unreleased . . . 205
13.2 [0.6.0-alpha.0] - 2023-08-13 . . . 205
13.3 [0.5.1] - Unreleased . . . 208
13.4 [0.5.0] - 2022-03-22 . . . 208



Welcome to AtomVM, the Erlang virtual machine for IoT devices!

AtomVM is a lightweight implementation of the the Bogdan Erlang Abstract Machine (_aka_, the BEAM), a virtual machine that can execute byte-code instructions compiled from Erlang or Elixir source code. AtomVM supports a limited but functional subset of the BEAM opcodes, and also includes a small subset of the Erlang/OTP standard libraries, all optimized to run on tiny micro-controllers. With AtomVM, you can write your IoT applications in a functional programming language, using a modern actor-based concurrency model, making them vastly easier to write and understand!

AtomVM includes many advanced features, including process spawning, monitoring, message passing, pre-emptive scheduling, and efficient garbage collection. It can also interface directly with peripherals and protocols supported on micro-controllers, such as GPIO, I2C, SPI, and UART. It also supports WiFi networking on devices that support it, such as the Espressif ESP32. All of this on a device that can cost as little as $2!

Warning AtomVM is currently in Alpha status. Software may contain bugs and should not be used for mission-critical applications. Application Programming Interfaces may change without warning.

Chapter 1

Welcome to AtomVM!

Welcome to AtomVM, the Erlang virtual machine for IoT devices!

1.1 What is AtomVM?

AtomVM is a ground-up implementation of the Bogdan Erlang Abstract Machine (a.k.a the BEAM) and is designed specifically to run on small systems, such as the Espressif ESP32 and ST Microelectronics STM32 micro-controllers. It allows developers to implement IoT applications in the Erlang or Elixir programming languages and to deploy those applications onto tiny devices. (Users may also target their applications for fully-fledged operating systems, such as Linux, FreeBSD, and MacOS, though in most cases deployment to traditional computers is done for development and testing purposes, only.)

AtomVM features include:

- An Erlang runtime, capable of executing bytecode instructions in compiled BEAM files;
- Support for all the major Erlang and Elixir types, including integers, strings, lists, maps, binaries, Enums, and more;
- A memory-managed environment, with efficient garbage collection and shared data, where permissible;
- Support for truly functional programming languages, making your programs easier to understand and debug;
- A concurrency-oriented platform, allowing users to spawn, monitor, and communicate with lightweight processes, making it easy for your IoT devices to perform tasks simultaneously;
- Support for symmetric multi-processing (SMP); leverage all available cores on platforms that support it (e.g., ESP32) without any code changes;
- A rich set of networking APIs, for writing robust IoT applications that communicate over IP networks;
- A rich set of APIs for interfacing with standard device protocols, such as GPIO, I2C, SPI, and UART;
- And more!

1.2 Why Erlang/Elixir?

The environments on which AtomVM applications are deployed are significantly more constrained than typical programming environments. For example, the typical ESP32 ships with 520K of RAM and 4MB of flash storage, roughly the specs of a mid 1980s desktop computer. Moreover, most micro-controller environments do not support native POSIX APIs for interfacing with an operating

system, and in many cases, common operating system abstractions, such as processes, threads, or files, are simply unavailable.

However, because the BEAM is provides a pre-emptive multitasking environment for your applications, many of the common operating system abstractions, particularly involving threading and concurrency, are simply not needed. As concurrently-oriented languages, Erlang and Elixir support lightweight "processes", with message passing as the mechanism for inter-(erlang)process communication, pre-emptive multi-tasking, and per-process heap allocation and garbage collection.

In many ways, the programming model for Erlang and Elixir is closer to that of an operating system and multiple concurrent processes running on it, where operating system processes are single execution units, communicate through message passing (signals), and don't share any state with one another. Contrast that with most popular programming languages today (C, C++, Java, Python, etc), which use threading abstractions to achieve concurrency within a single memory space, and which subsequently require close attention to cases in which multiple CPUs operate on a shared region of memory, requiring threads, locks, semaphores, and so forth.

As an implementation of the BEAM, AtomVM provides a modern, memory managed, and concurrency-oriented environment for developing applications on small devices. This makes writing concurrent code for micro-controllers (e.g., and application that reads sensor data, services HTTP requests, and updates the system clock, all at the same time) incredibly simple and natural – far easier writing programs that use concurrency than C, C++, or even, for example, Micropython.

In addition, because it is targeted for micro-controller environments, AtomVM provides interfaces for integrating with features commonly seen on micro-controllers, such as GPIO pins, analog-to-digital conversion, and common industry peripheral interfaces, such as I2C, SPI, and UART, making AtomVM a rich platform for developing IoT applications.

Finally, one of the exciting aspects about modern micro-controllers, such as the ESP32, is their integration with modern networking technologies, such as WiFi and Bluetooth. AtomVM leverages Erlang and Elixir's natural affinity with telecommunications technologies to open up further possibilities for developing networked and wireless IoT devices.

We think you will agree that AtomVM provides a compelling environment not only for Erlang and Elixir development, but also as a home for interesting and fun IoT projects.

1.3 Design Philosophy

AtomVM is designed from the start to run on small, cheap embedded devices, where system resources (memory, cpu, storage) are tightly constrained. The smallest environment in which AtomVM runs has around 512k of addressable RAM, some of which is used by the underlying runtime (FreeRTOS), and some of which is used by the AtomVM virtual machine, itself, leaving even less RAM for your own applications. Where there is a tradeoff between memory consumption and performance, minimizing memory consumption (and heap fragmentation) always wins.

From the developer's point of view, AtomVM is designed to make use of the existing tool chain from the Erlang and Elixir ecosystems. This includes the Erlang and Elixir compilers, which will compile Erlang and Elixir source code to BEAM bytecode. Where possible, AtomVM makes use of existing tool chains to reduce the amount of unnecessary features in AtomVM, thus reducing complexity, as well as the amount of system resources in use by the runtime. AtomVM is designed to be as small and lean as possible, providing as many resources to user applications, as possible.

1.4 Licensing

AtomVM is licensed under the terms of the Apache2 and LGPLv2 licenses.

1.5 Source Code

The AtomVM Github Repository contains the AtomVM source code, including the AtomVM virtual machine and core libraries. The AtomVM Build Instructions contains instructions for building AtomVM for Generic UNIX, ESP32, and STM32 platforms.

1.6 Contributing

The AtomVM community welcomes contributions to the AtomVM code base and upstream and downstream projects. Please see the contributing guidelines for information about how to contribute.

AtomVM developers can be reached on the #AtomVM discord server or on Telegram at AtomVM - Erlang and Elixir on Microcontrollers.

1.7 Where to go from here

The following guides provide more detailed information about getting started with the AtomVM virtual machine, how to develop and deploy applications, and implementation information, for anyone interested in getting more involved:

- Getting Started Guide
- Programmers Guide
- Example Programs
- Build Instructions

Chapter 2

Release Notes

Welcome to AtomVM `0.6.0-alpha.1+git.aa2ee88b`

These release notes provide version information about the current release of AtomVM.

2.1 Required Software

The following software is required to develop Erlang or Elixir applications on AtomVM:

- An Erlang/OTP compiler (`erlc`)
- The Elixir runtime, if developing Elixir applications.
- (recommended) For Erlang programs, `rebar3`
- (recommended) For Elixir programs, `mix`, which ships with the Elixir runtime.

AtomVM will run BEAM files that have been compiled using the following Erlang and Elixir versions:

Erlang Version	Elixir Version
?	?
?	?
?	?
?	?
?	?
?	?

> Note. Versions of Elixir that are compatible with a particular OTP version may work. This table reflects the versions that are tested.

Not all BEAM instructions are supported for every Erlang and Elixir compiler. For details about which instructions are supported, see the `src/libAtomVM/opcodes.h` header file in the AtomVM github repository corresponding to the current release.

For detailed information about features and bug fixes in the current release, see the AtomVM Change Log.

2.1.1 ESP32 Support

To run applications built for AtomVM on the ESP32 platform you will need:

- The `esptool` program, for flashing the AtomVM image and AtomVM programs to ESP32 MCUs.
- A serial console program, such as `minicom` or `screen`, so that you can view console output from your AtomVM application.

AtomVM currently supports the following versions of ESP-IDF:

Espressif supported versions	AtomVM support
ESP-IDF v4.4	?
ESP-IDF v5.0	?
ESP-IDF v5.1	?

Building the AtomVM virtual machine for ESP32 is optional. In most cases, you can simply download a release image from the AtomVM release repository. If you wish to work on development of the VM or use one on the additional drivers that are available in the (AtomVM repositories) you will need a compatible version of (Espressif's) ESP-IDF. Espressif provides excellent installation documentation.

2.1.2 STM32 Support

The following software is required to build AtomVM for the STM32 platform:

Package
11.3 ARM toolchain
libopencm3 version 0.8.0

Note. AtomVM tests this build on the latest Ubuntu github runner.

2.1.3 Raspberry Pi Pico Support

The following software is required to build AtomVM for the Raspberry Pi Pico platform:

Package
`gcc-arm-none-eabi`
`libnewlib-arm-none-eabi`
`libstdc++-arm-none-eabi-newlib`

Note. AtomVM tests this build on the latest Ubuntu github runner.

Chapter 3

Getting Started Guide

The getting started is broken up into the following sections:

- Section 3.1
- Section 3.2
- Section 3.3
- Section 3.4

3.1 Getting Started on the ESP32 platform

The AtomVM virtual machine is supported on the Espressif ESP32 platform, allowing users to write Erlang and Elixir programs and deploy them to the ESP32 micro-controller.

These instructions cover how to get the AtomVM virtual machine flashed to your ESP32 device, as well as how to flash your Erlang and Elixir programs that will be executed by the virtual machine running on the device.

For most applications, you should only need to install the VM once (or at least once per desired AtomVM release). Once the VM is uploaded, you can then begin development of Erlang or Elixir applications, which can then be flashed as part of your routine development cycle.

3.1.1 Requirements

Deployment of AtomVM applications requires the following components:

- A computer running MacOS or Linux (Windows support is not currently supported);
- An ESP32 module with a USB/UART connector (typically part of an ESP32 development board);
- A USB cable capable of connecting the ESP32 module or board to your development machine (laptop or PC);
- The `esptool` program, for flashing the AtomVM image and AtomVM programs;
- An Erlang/OTP;
- A serial console program, such as `minicom` or `screen`, so that you can view console output from your AtomVM application.
- (recommended) For Erlang programs, `rebar3`;
- (recommended) For Elixir programs, `mix`, which ships with the Elixir runtime;

For information about specific versions of required software, see the Release Notes.

3.1.2 Deployment Overview

The ES32 AtomVM virtual machine is an IDF application that runs on the ESP32 platform. As an IDF application, it provides the object code to boot the ESP device and execute the AtomVM virtual machine code, which in turn is responsible for execution of an Erlang/Elixir application.

The AtomVM virtual machine is implemented in C, and the AtomVM binary image contains the binary object code compiled from C source files, as well as the ESP boot loader and partition map, which tells the ESP32 how the flash module is laid out.

AtomVM developers will typically write their applications in Erlang or Elixir. These source files are compiled into BEAM bytecode, which is then assembled into AtomVM "packbeam" (`.avm`) files. This packbeam file is flashed onto the ESP32 device, starting at the data partition address `0x210000`. When AtomVM starts, it will look in this partition for the first occurrence of a BEAM module that exports a `start/0` function. Once that module is located, execution of the BEAM bytecode will commence at that point.

The following diagram provides a simplified overview of the layout of the AtomVM virtual machine and Erlang/Elixir applications on the ESP32 flash module.

```
|               |
+---------------+  ----------- 0x1000
| boot loader   |           ^
+---------------+           |
| partition map |           | AtomVM
+---------------+           | binary
|               |           | image
|   AtomVM      |           |
|   Virtual     |           |
|   Machine     |           |
|               |           v
+---------------+  ----------- 0x210000
|               |           ^
|               |           |
|     data      |           | Erlang/Elixir
|   partition   |           | Application
|               |           |
|               |           v
+---------------+  ----------- end
```

Deploying an AtomVM application to an ESP32 device typically involved two steps:

1. Connecting the ESP32 device;
2. Deploying the AtomVM virtual machine;
3. Deploying an AtomVM application (typically an iterative process)

These steps are described in more detail below.

3.1.3 Connecting the ESP32 device

Connect the ESP32 to your development machine (e.g., laptop or PC) via a USB cable.

```
+---------------+
| laptop or PC  |
|               |           +-------+
|               | USB       |       |
|               x-----------x       |
|               |           |       |
|               |           +-------+
+---------------+           ESP32
```

Note. There are a wide variety of ESP32 modules, ranging from home-made breadboard solu-

tionsto all-in-one development boards. For simplicity, we assume a development board that can both be powered by a USB cable and which can be simultaneously flashed using the same cable, e.g., the Espressif ESP32 DevKit.

Consult your local development board documentation for instructions about how to connect your device to your development machine.

3.1.4 Deploying the AtomVM virtual machine

The following methods can be used to deploy the AtomVM virtual machine to an ESP32 device:

1. Flashing a binary image;
2. Building from source.

Flashing a binary images

Flashing the ESP32 using a pre-built binary image is by far the easiest path to getting started with development on the ESP32. Binary images contain the virtual machine image and all of the necessary components to run your application.

We recommend first erasing any existing applications on the ESP32 device. E.g.,

```
shell$ esptool.py --chip esp32 --port /dev/ttyUSB0 --baud 115200 erase_flash
...
```

> Note. Specify the device port and baud settings and AtomVM image name to suit your particular environment.

Next, download a stable or latest development ESP32 release image.

> Note. Development images may be unstable and may result in unpredictable behavior.

Finally, use the `esptool` to flash the image to the start address `0x1000` on the ESP32. E.g.,

```
shell$ esptool.py --chip esp32 --port /dev/ttyUSB0 --baud 115200 \
    --before default_reset --after hard_reset \
    write_flash -u --flash_mode dio --flash_freq 40m --flash_size detect \
    0x1000 atomvm-esp32-v0.1.0.bin
...
```

Once completed, your ESP32 device is ready to run Erlang or Elixir programs targeted for AtomVM.

Building from source

You may optionally build AtomVM from source and deploy the AtomVM virtual machine to your ESP32 device manually. Building AtomVM from source is slightly more involved, as it requires the installation of the Espressif IDF SDK and tool chain and is typically recommended only for users who are doing development on the AtomVM virtual machine, or for developers implementing custom Nifs or ports.

Instructions for building AtomVM from source are covered in the AtomVM Build Instructions

3.1.5 Deploying an AtomVM application

An AtomVM application is a collection of BEAM files, which have been compiled using the Erlang or Elixir compiler. These BEAM files are assembled into an AtomVM "packbeam" (`.avm`) file, which in turn is flashed to the `main` data partition on the ESP32 flash module, starting at address `0x210000`.

When the AtomVM virtual machine starts, it will search for the first module that contains an exported `start/0` function in this partition, and it will begin execution of the BEAM bytecode at that function.

AtomVM applications can be written in Erlang or Elixir, or a combination of both. The AtomVM community has provided tooling for both platforms, making deployment of AtomVM applications as seamless as possible.

This section describes both Erlang and Elixir tooling for deploying AtomVM applications to ESP32

devices.

Erlang Tooling

Deployment of AtomVM applications written in the Erlang programming language is supported via the `atomvm_rebar3_plugin` plugin, a community-supported plugin to the `rebar3` Erlang build tool.

You can generate a simple application from scratch using the `atomvm_rebar3_plugin` template, as follows:

Edit or create the `$HOME/.config/rebar3/rebar.config` file to include the `atomvm_rebar3_plugin` plugin:

```
%% $HOME/.config/rebar3/rebar.config
{plugins, [
    atomvm_rebar3_plugin,
    ...
]}.
```

In any directory in which you have write permission, issue

```
shell$ rebar3 new atomvm_app <app-name>
```

where `<app-name>` is the name of the application you would like to create (e.g., `myapp`). This command will generate a rebar project under the directory `<app-name>`.

The generated application will contain the proper `rebar.config` configuration and will contain the `<app-name>.erl` module, which exports the `start/0` function with a stubbed implementation.

Specifically, note the following stanza in the generated `rebar.config` file:

```
%% rebar.config
{plugins, [
    atomvm_rebar3_plugin,
    ...
]}.
```

And note the `myapp` application exports a `start/0` function, e.g.,

```
%% erlang
-module(myapp).
-export([start/0]).

start() ->
    ok.
```

With this plugin installed, you have access to the `esp32_flash` target, which will build an AtomVM packbeam

```
shell$ rebar3 esp32_flash --port /dev/ttyUSB0
===> Fetching atomvm_rebar3_plugin v0.6.0
===> Fetching rebar3_hex v6.11.3
===> Fetching hex_core v0.7.1
===> Fetching verl v1.0.2
===> Analyzing applications...
===> Compiling verl
===> Compiling hex_core
===> Compiling rebar3_hex
===> Fetching atomvm_packbeam v0.6.0
===> Analyzing applications...
===> Compiling atomvm_rebar3_plugin
===> Compiling packbeam
===> Verifying dependencies...
===> Analyzing applications...
===> Compiling myapp
```

```
===> AVM file written to : myapp.avm
===> esptool.py --chip esp32 --port /dev/ttyUSB0 --baud 115200 --before
default_reset --after hard_reset write_flash -u --flash_mode dio --flash_freq 40m
--flash_size detect 0x210000 /home/frege/myapp/_build/default/lib/myapp.avm
```

Note. Consult the the `atomvm_rebar3_plugin` plugin documentation, for more detailed information about how to use this tool.

Once the application has been flashed, you may connect to the ESP32 over the serial port using `minicom`, `screen`, or equivalent.

Elixir Tooling

TODO mix + https://github.com/atomvm/ExAtomVM + hex

3.2 Getting Started on the STM32 platform

AtomVM can run on a wide variety of STM32 chipsets available from STMicroelectronics. The support is not nearly as mature as for the ESP32 platform, but work is ongoing, and pull requests are always welcome. At this time AtomVM will work on any board with a minimum of around 128k ram and 512k (1M recommended) flash. Simple applications and tests have been successfully run on a stm32f411ceu6 (A.K.A. Black Pill V2). These minimum requirements may need to be raised as platform support matures.

3.2.1 Prerequisites

- st-flash, to flash both AtomVM and your packed AVM applications. Make sure to follow its installation procedure before proceeding further.
- `packbeam` the AtomVM for packing and stripping `*.beam` files into the AtomVM `*.avm` format.
- A serial console program, such as `minicom` or `screen`, so that you can view console output from your AtomVM application.

3.2.2 Build an AtomVM binary

You will first need to build a binary configured for your processor and board layout. Consult the Build Instruction for STM32

3.2.3 Flashing

To flash AtomVM, use

```
$ st-flash --reset write AtomVM-stm32f407vgt6.bin 0x8000000
```

To flash your packed AVM, use

```
$ st-flash --reset write /path/to/your/packed.avm 0x8080000
```

You must include the atomvmlib.avm with your application when using packbeam, and it should be pruned:

```
$ packbeam create -p -i application.avm application.beam
/path/to/AtomVM/build/libs/atomvmlib.avm
```

Note: The option `-i` will instruct packbeam to include file names and line numbers in stack traces. This makes debugging applications far easier, but also increases size, so it may be omitted if desired. The `-p` option should be used, it instructs packbeam to prune the unused functions from the packed `.avm` file, and is strongly recommended.

AtomVM expects to find the AVM at the address 0x808000. On a STM32 Discovery board this means

that the 1MB of flash will be split in 512KB available for the program and 512KB available for the packed AVM. If for any reason you want to modify this, you can change `AVM_ADDRESS` and `AVM_FLASH_MAX_SIZE` defines in `main.c`.

3.2.4 Printing

By default, stdout and stderr are printed on USART2. On the STM32F4Discovery board, you can see them using a TTL-USB with the TX pin connected to board's pin PA2 (USART2 RX). Baudrate is 115200 and serial transmission is 8N1 with no flow control.

3.2.5 Distributed Binaries

Due to the very large number of supported chipsets, the wide variety of board configurations, and the code changes required to support them, it is unlikely pre-built binaries will be available for the stm32 platform in the near future. Consult the Build Instruction to create a binary compatible with your board.

3.3 Getting Started on the Raspberry Pi Pico platform

3.3.1 Prerequisites

None of these tools are strictly required, but all are recommended for easier development:

- `rebar3`
- `atomvm_rebar3_plugin`
- `packbeam` the AtomVM for packing and stripping `*.beam` files into the AtomVM `*.avm` format. (included as part of the `atomvm_rebar3_plugin`)
- A serial console program, such as `minicom` or `screen`, so that you can view console output from your AtomVM application.

3.3.2 Building AtomVM for Raspberry Pico

If you want to use a custom built VM for testing consult the Build Instructions for Raspberry Pi Pico

3.3.3 Installing AtomVM and programs on Raspberry Pico

The approach consists in installing various uf2 files which include the address they should be loaded to.

You typically need three uf2 files:

- `AtomVM.uf2` for the VM
- `atomvmlib.uf2` for the standard libraries
- your application's uf2.

We provide an escript-based (what else?) tool to build uf2 files called `uf2tool` that you can use to bundle your `avm` into uf2.

If you need to upgrade AtomVM or the standard libraries, simply copy them again.

3.3.4 Installing AtomVM on Raspberry Pico

VM binary is file `AtomVM.uf2` - `src/platforms/rp2040/build/src/AtomVM.uf2` if build from source. Simply copy it to the pico. The VM will crash because there is no application.

3.3.5 Installing atomvm library to Raspberry Pico

AtomVM library must be installed as well. For Build instructions consult the Raspberry Pi Pico libAtomVM build steps

Installing it

The library to install is `atomvmlib.uf2`, `build/libs/atomvmlib.uf2` if build from source. Copy the library to the pico.

3.3.6 Running Hello Pico

This example will print a Hello Pico message repeatedly.

It is built into `build/examples/erlang/rp2040/hello_pico.uf2`.

You can install it and then connect to the serial port with minicom.

3.3.7 Running your own BEAM code on Raspberry Pico

You need to create an avm file using the `packbeam` tool the `atomvm_rebar3_plugin`.

```
packbeam create -p -i packed.avm module.beam
```

or

```
rebar3 packbeam -p -i packed.avm module.beam
```

Then the BEAM file must be converted to UF2. The VM currently expects the application to be loaded at address 0x100A0000.

```
./uf2tool create -o packed.uf2 -s 0x100A0000 packed.avm
```

Copy this UF2 to the Pico after you copied the VM (`AtomVM.uf2`) and the standard libraries (`atomvmlib.uf2`).

3.4 Getting Started on the Generic UNIX platform

AtomVM may be run on UNIX-like platforms using the `atomvm` command.

You may specify one or more AVM files on the command line when running the `atom` command. BEAM modules defined in earlier AVM modules on the command line take higher precedence that BEAM modules included in AVM files later in the argument list.

```
shell$ atomvm /path/to/myapp.avm
```

Currently, the `atomvm` command and libraries must be built and installed from source.

> See the AtomVM Build Instructions for instructions about how to build AtomVM on the Generic UNIX platform.

3.5 Where to go from here

The following resources may be useful for understanding how to develop Erlang or Elixir applications for the AtomVM platform:

- Example Programs
- Programmers Guide

3.6 Getting Started with AtomVM WebAssembly port.

AtomVM may be run on platforms with NodeJS from the AtomVM.js file (with its companion AtomVM.wasm file).

Currently, these files must be built and installed from source.

> See the AtomVM Build Instructions for instructions about how to build AtomVM WebAssembly port.

AtomVM may also be run in modern browsers (Safari, Chrome and Chrome-based, Firefox) from the AtomVM.js, AtomVM.worker.js and AtomVM.wasm files.

Currently, these files must be built and installed from source. They also need to be installed on a web server that send `Cross-Origin-Opener-Policy` and `Cross-Origin-Embedder-Policy` headers as explained in Mozilla's documentation.

AtomVM comes with a toy web server you can use and run with AtomVM built for Generic Unix with:

```
./src/AtomVM examples/emscripten/wasm_webserver.avm
```

This web server serves HTML files from `examples/emscripten/` which you can copy to a webserver along with binaries and built files.

3.7 Where to go from here

The following resources may be useful for understanding how to develop Erlang or Elixir applications for the AtomVM platform:

- Example Programs
- Programmers Guide

Chapter 4

Programmers Guide

This guide is intended for programmers who develop applications targeted for AtomVM.

As an implementation of the Erlang virtual machine, AtomVM is designed to execute unmodified byte-code instructions compiled into BEAM files, either by the Erlang or Elixir compilers. This allow developers to write programs in their BEAM programming language of choice, and to use the common Erlang community tool-chains specific to their language platform, and to then deploy those applications onto the various devices that AtomVM supports.

This document describes the development workflow when writing AtomVM applications, as well as a high-level overview of the various APIs that are supported by AtomVM. With an understanding of this guide, you should be able to design, implement, and deploy applications onto a device running the AtomVM virtual machine.

4.1 AtomVM Features

Currently, AtomVM implements a strict subset of the BEAM instruction set.

A high level overview of the supported language features include:

- All the major Erlang types, including
  - integers (with size limits)
  - floats
  - tuples
  - lists
  - binaries
  - maps
- support for many Erlang BIFs and guard expressions to support the above types
- pattern matching (case statements, function clause heads, etc)
- `try ... catch ... finally` constructs
- anonymous functions
- process `spawn` and `spawn_link`
- send (`!`) and `receive` messages
- bit syntax (with some restrictions)
- reference counted binaries
- stacktraces
- symmetric multi-processing (SMP)

In addition, several features are supported specifically for integration with micro-controllers, including:

- Wifi networking (`network`)
- UDP and TCP/IP support (`inet`, `gen_tcp` and `gen_udp`)
- Peripheral and system support on micro-controllers, including
  - GPIO, including pins reads, writes, and interrupts
  - I2C interface
  - SPI interface
  - UART interface
  - LEDC (PWM)
  - non-volatile storage (NVS)
  - RTC storage
  - deep sleep

4.1.1 Limitations

While the list of supported features is long and growing, the currently unsupported Erlang/OTP and BEAM features include (but are not limited to):

- Bignums. Integer values are restricted to 64-bit values.
- Bit Syntax. While packing and unpacking of arbitrary (but less than 64-) bit values is support, packing and unpacking of integer values at the start or end of a binary, or bordering binary packing or extraction must align on 8-bit boundaries. Arbitrary bit length binaries are not currently supported.
- The `epmd` and the `disterl` protocols are not supported.
- There is no support for code hot swapping.
- There is no support for a Read-Eval-Print-Loop. (REPL)
- Numerous modules and functions from Erlang/OTP standard libraries (`kernel`, `stdlib`, `sasl`, etc) are not implemented.

AtomVM bit syntax is restricted to alignment on 8-bit boundaries. Little-endian and signed insertion and extraction of integer values is restricted to 8, 16, and 32-bit values. Only unsigned big and little endian 64-bit values can be inserted into or extracted from binaries.

It is highly unlikely that an existing Erlang program targeted for Erlang/OTP will run unmodified on AtomVM. And indeed, even as AtomVM matures and additional features are added, it is more likely than not that Erlang applications will need to targeted specifically for the AtomVM platform. The intended target environment (small, cheap micro-controllers) differs enough from desktop or server-class systems in both scale and APIs that special care and attention is needed to target applications for such embedded environments.

That being said, many of the features of the BEAM are supported and provide a rich and compelling development environment for embedded devices, which Erlang and Elixir developers will find natural and productive.

4.2 AtomVM Development

This section describes the typical development environment and workflow most AtomVM developers are most likely to use.

4.2.1 Development Environment

In general, for most development purposes, you should be able to get away with an Erlang/OTP development environment, and for Elixir developers, and Elixir development environment. For specific version requirements, see the Release Notes.

We assume most development will take place on some UNIX-like environment (e.g., Linux, FreeBSD, or MacOS). Consult your local package manager for installation of these development environments.

Developers will want to make use of common Erlang or Elixir development tools, such as `rebar3` for Erlang developers or `mix` for Elixir developers.

Developers will need to make use of some AtomVM tooling. Fortunately, there are several choices for developers to use:

1. AtomVM `PackBEAM` executable (described below)
2. `atomvm_rebar3_plugin`, for Erlang development using `rebar3`.
3. `ExAtomVM` Mix plugin, Elixir development using `Mix`.

Some testing can be performed on UNIX-like systems, using the `AtomVM` executable that is suitable for your development environment. AtomVM applications that do not make use of platform-specific APIs are suitable for such tests.

Deployment and testing on micro-controllers is slightly more involved, as these platforms require additional hardware and software, described below.

ESP32 Deployment Requirements

In order to deploy AtomVM applications to and test on the ESP32 platform, developers will need:

- A computer running MacOS or Linux (Windows support is TBD);
- An ESP32 module with a USB/UART connector (typically part of an ESP32 development board);
- A USB cable capable of connecting the ESP32 module or board to your development machine (laptop or PC);
- The `esptool` program, for flashing the AtomVM image and AtomVM programs;
- (Optional, but recommended) A serial console program, such as `minicom` or `screen`, so that you can view console output from your AtomVM application.

STM32 Deployment Requirements

TODO

4.2.2 Development Workflow

For the majority of users, AtomVM applications are written in the Erlang or Elixir programming language. These applications are compiled to BEAM (`.beam`) files using standard Erlang or Elixir compiler tool chains (`erlc`, `rebar3`, `mix`, etc). The generated BEAM files contain byte-code that can be executed by the Erlang/OTP runtime, or by the AtomVM virtual machine.

> Note. In a small number of cases, it may be useful to write parts of an application in the C programming language, as AtomVM nifs or ports. However, writing AtomVM nifs and ports is outside of the scope of this document.

Once Erlang and/or Elixir files are compiled to BEAM files, AtomVM provides tooling for processing and aggregating BEAM files into AtomVM Packbeam (`.avm`) files, using AtomVM tooling, distributed as part of AtomVM, or as provided through the AtomVM community.

AtomVM packbeam files are the applications and libraries that run on the AtomVM virtual machine. For micro-controller devices, they are "flashed" or uploaded to the device; for command-line use of AtomVM (e.g., on Linux, FreeBSD, or MacOS), they are supplied as the first parameter to

the AtomVM command.

The following diagram illustrates the typical development workflow, starting from Erlang or Elixir source code, and resulting in a deployed Packbeam file:

```
*.erl or *.ex                  *.beam
+-------+                   +-------+
|       |+                  |       |+
|       ||+                 |       ||+
|       |||     -------->   |       |||
|       |||  Erlang/Elixir  |       |||
+-------+||     Compiler    +-------+||
 +-------+|                  +-------+|
  +-------+                   +-------+
     ^                           |
     |                           | packbeam
     |                           |
     |                           v
     |                        +-------+
     |                        |       |
     | test                   |       |
     | debug                  |       |
     | fix                    |       |
     |                        +-------+
     |                         app.avm
     |                           |
     |                           | flash/upload
     |                           |
     |                           v
     +-------------------- Micro-controller
                              device
```

The typical compile-test-debug cycle can be summarized in the following steps:

1. Deploy the AtomVM virtual machine to your device
2. Develop an AtomVM application in Erlang or Elixir
    1. Write application
    2. Deploy application to device
    3. Test/Debug/Fix application
    4. Repeat

Deployment of the AtomVM virtual machine and an AtomVM application currently require a USB serial connection. There is currently no support for over-the-air (OTA) updates.

For more information about deploying the AtomVM image and AtomVM applications to your device, see the Getting Started Guide

4.3 Applications

An AtomVM application is a collection of BEAM files, aggregated into an AtomVM "Packbeam" (`.avm`) file, and typically deployed (flashed) to some device. These BEAM files be be compiled from Erlang, Elixir, or any other language that targets the Erlang VM.

> Note. The return value from the `start/0` function is ignored.

Here, for example is one of the smallest AtomVM applications you can write:

```
%% erlang
-module(myapp).
```

```
-export([start/0]).

start() ->
    ok.
```

This particular application doesn't do much, of course. The application will start and immediately terminate, with a return value of `ok`. Typical AtomVM applications will be more complex than this one, and the AVM file that contains the application BEAM files will be considerably larger and more complex than the above program.

Most applications will spawn processes, send and receive messages between processes, and wait for certain conditions to apply before terminating, if they terminate at all. For applications that spawn processes and run forever, you may need to add an empty `receive ... end` block, to prevent the AtomVM from terminating prematurely, e.g.,

```
%% erlang
wait_forever() ->
    receive X -> X end.
```

4.3.1 Packbeam files

AtomVM applications are packaged into Packbeam (`.avm`) files, which contain collections of files, typically BEAM (`.beam`) files that have been generated by the Erlang or Elixir compiler.

At least one BEAM module in this file must contain an exported `start/0` function. The first module in a Packbeam file that contain this function is the entry-point of your application and will be executed when the AtomVM virtual machine starts.

Not all files in a Packbeam need to be BEAM modules – you can embed any type of file in a Packbeam file, for consumption by your AtomVM application.

> Note. The Packbeam format is described in more detail in the AtomVM PackBEAM format.

The AtomVM community has provided several tools for simplifying your experience, as a developer. These tools allow you to use standard Erlang and Elixir tooling (such as `rebar3` and `mix`) to build Packbeam files and deploy then to your device of choice.

4.3.2 `PackBEAM` tool

The `PackBEAM` tool is a command-line application that can be used to create Packbeam files from a collection of input files:

```
shell$ PackBEAM -h
Usage: PackBEAM [-h] [-l] <avm-file> [<options>]
    -h                                                  Print this help menu.
    -i                                                  Include file and line
information.
    -l <input-avm-file>                                 List the contents of an AVM
file.
    [-a] <output-avm-file> <input-beam-or-avm-file>+  Create an AVM file (archive
if -a specified).
```

To create a packbeam file, specify the name of the AVM file to created (by convention, ending in `.avm`), followed by a list of BEAM files:

```
shell$ PackBEAM foo.avm path/to/foo.beam path/to/bar.beam
```

You can also specify another AVM file to include. Thus, for example, to add to BEAM file to an existing AVM file, you might enter:

```
shell$ PackBEAM foo.avm foo.avm path/to/gnu.beam
```

To list the contents of an AVM file, use the `-l` flag:

```
shell% PackBEAM -l foo.avm
foo.beam *
bar.beam
gnu.beam
```

Any BEAM files that export a `start/0` function will contain an asterisk (`*`) in the AVM file contents.

4.3.3 Running AtomVM

AtomVM is executed in different ways, depending on the platform. On most microcontrollers (e.g., the ESP32), the VM starts when the device is powered on. On UNIX platforms, the VM is started from the command-line using the `AtomVM` executable.

AtomVM will use the first module in the supplied AVM file that exports a `start/0` function as the entrypoint for the application.

`AtomVM` program syntax

On UNIX platforms, you can specify a BEAM file or AVM file as the first argument to the executable, e.g.,

```
shell$ AtomVM foo.avm
```

> Note. If you start the `AtomVM` executable with a BEAM file, then the corresponding module may not make any calls to external function in other modules, with the exception of built-in functions and Nifs that are included in the VM.

4.4 Core APIs

The AtomVM virtual machine provides a set of Erlang built-in functions (BIFs) and native functions (NIFs), as well as a collection of Erlang and Elixir libraries that can be used from your applications.

This section provides an overview of these APIs. For more detailed information about specific APIs, please consult the API reference documentation.

4.4.1 Standard Libraries

AtomVM provides a limited implementations of standard library modules, including:

- `base64`
- `gen_server`
- `gen_statem`
- `io` and `io_lib`
- `lists`
- `maps`
- `proplists`
- `supervisor`
- `timer`

In addition AtomVM provides limited implementations of standard Elixir modules, including:

- `List`
- `Tuple`
- `Enum`
- `Kernel`

- `Module`
- `Process`
- `Console`

For detailed information about these functions, please consult the API reference documentation. These modules provide a strict subset of functionality from their Erlang/OTP counterparts. However, they aim to be API-compatible with the Erlang/OTP interfaces, at least for the subset of provided functionality.

4.4.2 Spawning Processes

AtomVM supports the actor concurrency model that is pioneered in the Erlang/OTP runtime. As such, users can spawn processes, send messages to and receive message from processes, and can link or monitor processes to be notified if they have crashed.

To spawn a process using a defined or anonymous function, pass the function to the `spawn/1` function:

```
%% erlang
Pid = spawn(fun run_some_code/0),
```

The function you pass may admit closures, so for example you can pass variables defined outside of the scope of the function to the anonymous function to pass into `spawn/1`:

```
%% erlang
Args = ...
Pid = spawn(fun() -> run_some_code_with_args(Args) end),
```

Alternatively, you can pass a module, function name, and list of arguments to the `spawn/3` function:

```
%% erlang
Args = ...
Pid = spawn(?MODULE, run_some_code_with_args, [Args]),
```

The `spawn_opt/2,4` functions can be be used to spawn a function with additional options that control the behavior of the spawned processs, e.g.,

```
%% erlang
Pid = spawn_opt(fun run_some_code/0, [{min_heap_size, 1342}]),
```

The options argument is a properties list containing optionally the following entries:

Key	Value Type	Default Value	Description
`min_heap_size`	`non_neg_integer()`	none	Minimum heap size of the process. The heap will shrink no smaller than this size.
`max_heap_size`	`non_neg_integer()`	unbounded	Maximum heap size of the process. The heap will grow no larger than this size.
`link`	`boolean()`	`false`	Whether to link the spawned process to the spawning process.

monitor	boolean()	false	Whether to link the spawning process should monitor the spawned process.
atomvm_heap_growth	bounded_free | minimum | fibonacci	bounded_free	Section 9.1.2 to grow the heap of the process.

4.4.3 Console Output

There are several mechanisms for writing data to the console.

For common debugging, many users will find `erlang:display/1` sufficient for debugging:

```
%% erlang
erlang:display({foo, [{bar, tapas}]}).
```

The output parameter is any Erlang term, and a newline will be appended automatically.

Users may prefer using the `io:format/1,2` functions for more controlled output:

```
%% erlang
io:format("The ~p did a ~p~n", [friddle, frop]).
```

Note that the `io_lib` module can be used to format string data, as well.

> Note. Formatting parameters are currently limited to `~p`, `~s`, and `~n`.

4.4.4 Logging

AtomVM supports a subset of the OTP logging facility, allowing users to send log event to log handlers (by default, the console), and to install handlers that handle log events.

To log events, you are encouraged to use the logging macros from the OTP `kernel` application. You can use these macros at compile time, and the generated code can be run in AtomVM.

For example:

```
%% erlang
-include_lib("kernel/include/logger.hrl").
...
?LOG_NOTICE("Something happened that might require your attention: ~p",
[TheThing])
```

By default, this will result in a message displayed on the console, with a timestamp, log level, PID of the process that initiated the log message, the module, function, and function arity, together with the supplied log message:

```
2023-07-04T18:34:56.387 [notice] <0.1.0> test_logger:test_default_logger/0
Something happened that might require your attention: ThatThingThatHappened
```

> Note that log messages need not (and generally should not) include newline separators (`~n`) in log format messages, unless necessary.

Users may provide a format string, with an optional list of arguments. Alternatively, users can provide a map encapsulating a "report" in lieu of a format string. Reports provide a mechanism for supplying a set of structured data directly to log handlers (see below), without necessarily incurring the cost of formatting log messages.

As with OTP, the following ordered log levels (from high to low) are supported:

- `emergency`
- `critical`

- `alert`
- `error`
- `warning`
- `notice`
- `info`
- `debug`

By default, the logging facility drops any messages below `notice` level. To set the default log level for the logging subsystem, see the `logger_manager` section, below.

You can use the `logger` interface directly to log messages at different levels, but in general, the OTP logging macros are encouraged, as log events generated using the OTP macros include additional metadata (such as the location of the log event) you do not otherwise get using the functions in the `logger` module.

For example, the expression

```
logger:notice("Something happened that might require your attention: ~p",
[TheThing])
```

may seem similar to using the `?LOG_NOTICE` macro, but less contextual information will be included in the log event.

For more information about the OTP logging facility, see the Erlang/OTP Logging chapter.

> Note. AtomVM does not currently support programmatic configuration of the logging subsystem. All changes to default behavior should be done via the AtomVM `logger_manager` module (see below).

The `logger_manager`

In order to use the `logger` interface, you will need to first start the AtomVM `logger_manager` service.

> Note. Future versions of AtomVM may automatically start the logging subsystem as part of a kernel application, but currently, this service must be managed manually.

To start the `logger_manager`, use the `logger_manager:start_link/1` function, passing in a configuration map for the logging subsystem.

For example, the default logging framework can be started via:

```
%% erlang
{ok, _Pid} = logger_manager:start_link(#{})
```

> Note. The logger_manager is a registered process, so the returned Pid may be ignored.

The configuration map supplied to the logger_manager may contain the following keys:

Key	Type	Default	Description
`log_level`	`log_level()`	`notice`	Primary log level
`logger`	`logger_config()`	`{handler, default, logger_std_h, undefined}`	Log configuration
`module_level`	`module_level()`	`undefined`	Log level specific to a set of modules

where `log_level()` is defined to be:

```
-type log_level() :: emergency | critical | alert | error | warning | notice |
info | debug.
```

and `logger_config()` is defined as follows:

```
-type handler_id() :: default | atom().
-type handler_config() :: #{
    id => atom(),
    module => module(),
    level => logger:level() | all | none,
    config => term()
}.
-type logger_config() :: [
    {handler, default, undefined} |
    {handler, HandlerId :: handler_id(), Handler :: module(), HandlerConfig ::
handler_config()} |
    {module_level, logger:level(), [module()]}
].
```

You can set the log level for all log handlers by setting the `log_level` in this configuration map. Any messages that are logged at levels "higher" than or equal to the configured log level will be logged by all log handlers.

The standard logger (`logger_std_h`) is included by default, if no default logger is specified (and if the default logger is not disabled – see below). The standard logger will output log events to the console.

You can specify multiple log handlers in the `logger` configuration. If a log entry is allowed for a given log level, then each log handler will handle the log message. For example, you might have a log handler that sends messages over the network to a syslog daemon, or you might have another handler that writes log messages to a file.

You can pass handler configuration int the `config` element of the `handler_config()` you specify when specifying a logger. The value of the `config` element can be any term and is made available to log handlers when events are logged (see below).

If the tuple `{handler, default, undefined}` is included in the logger configuration, the default logger will be disabled.

At most one `default` logger can be specified. If you want to replace the `default` logger (`logger_std_h`), then specify a logger with the handler id `default`.

You can specify different log levels for specific modules. For example, if you want to set the default log level for all handlers to be `notice` or higher, you can set the log level for a given module to `info`, and all `info` and higher messages will be logged for that module or set of modules. Conversely, you can "quiet" a module if it is particularly noisy by setting its level to something relatively high.

For more information about how to configure the logging subsystem, see the Kernel Configuration Parameters section of the OTP Logging chapter.

You can stop the `logger_manager` via the `logger_manager:stop/0` function:

```
%% erlang
ok = logger_manager:stop()
```

Writing your own log handler

Additional loggers can be enabled via handler specifications. A handler module must implement and export the `log/2` function, which takes a log event and a term containing the configuration for the logger handler instance.

For example:

```
%% erlang
-module(my_module).

-export([..., log/2, ...]).

log(LogEvent, HandlerConfig) ->
    %% do something with the log event
```

```
    %% return value is ignored
```

You can specify this handler in the `logger_manager` configuration (see above) via a stanza such as:

```
{handler, my_id, my_module, HandlerConfig}
```

A `LogEvent` is a map structure containing the following fields:

Key	Type	Description
`timestamp`	`integer()`	The time (in microseconds since the UNIX epoch) at which the log event was generated
`level`	`logger:level()`	The log level with which the log event was generated
`pid`	`pid()`	The process id of the Erlang process in which the event was generated
`msg`	`string() | {string(), list()}`	The message format and arguments passed when the event was generated
`meta`	`map()`	Metadata passed when the event was generated.

If the log event was generated using a logging macro, then the `meta` map also contains a `location` field with the following fields:

Key	Type	Description
`file`	`string()`	The path of the file in which the event was generated
`line`	`non_neg_integer()`	The line number in the file in which the event was generated
`mfa`	`{module(), function_name(), arity()}`	The MFA of the function in which the event was generated

The handler config is a map structure containing the id and module of the handler.

an arbitrary term and is passed into the log handler via configuration of the `logger_manager` (see above).

4.4.5 Process Management

You can obtain a list of all processes in the system via `erlang:processes/0`:

```
%% erlang
Pids = erlang:processes().
```

And for each process, you can get detailed process information via the `erlang:process_info/2` function:

```
%% erlang
[io:format("Heap size for Pid ~p: ~p~n", [Pid, erlang:process_info(Pid,
heap_size)]) || Pid <- Pids].
```

The return value is a tuple containing the key passed into the `erlang:process_info/2` function and its associated value.

The currently supported keys are enumerated in the following table:

Key	Value Type	Description
`heap_size`	`non_neg_integer()`	Number of terms (in machine words) used in the process heap
`stack_size`	`non_neg_integer()`	Number of terms (in machine words) used in the process stack

`message_queue_len`	`non_neg_integer()`	Number of unprocessed messages in the process mailbox
`memory`	`non_neg_integer()`	Total number of bytes used by the process (estimate)

See the `word_size` key in the System APIs section for information about how to find the number of bytes used in a machine word on the current platform.

4.4.6 System APIs

You can obtain system information about the AtomVM virtual machine via the `erlang:system_info/1` function, which takes an atom parameter designating the desired datum. Allowable parameters include

- `process_count` The number of processes running in the system.
- `port_count` The number of ports running in the system.
- `atom_count` The number of atoms allocated in the system.
- `word_size` The word size (in bytes) on the current platform (typically 4 or 8).
- `atomvm_version` The version of AtomVM currently running (as a binary).

For example,

```
%% erlang
io:format("Atom Count: ~p~n", [erlang:system_info(atom_count)]).
```

> Note. Additional platform-specific information is supported, depending on the platform type. See below.

Use the `atomvm:platform/0` to obtain the system platform on which your code is running. The return value of this function is an atom who's value will depend on the platform on which your application is running.

```
%% erlang
case atomvm:platform() of
    esp32 ->
        io:format("I am running on an ESP32!~n");
    stm32 ->
        io:format("I am running on an STM32!~n");
    generic_unix ->
        io:format("I am running on a UNIX box!~n")
end.
```

Use `erlang:garbage_collect/0` or `erlang:garbage_collect/1` to force the AtomVM garbage collector to run on a give process. Garbage collection will in general happen automatically when additional free space is needed and is rarely needed to be called explicitly.

The 0-arity version of this function will run the garbage collector on the currently executing process.

```
%% erlang
Pid = ... %% get a reference to some pid
ok = erlang:garbage_collect(Pid).
```

Use the `erlang:memory/1` function to obtain information about allocated memory.

Currently, AtomVM supports the following types:

Type	Description
`binary`	Return the total amount of memory (in bytes) occupied by (reference counted) binaries

> Note. Binary data small enough to be stored in the Erlang process heap are not counted in this measurement.

4.4.7 System Time

AtomVM supports numerous function for accessing the current time on the device.

Use `erlang:timestamp/0` to get the current time since the UNIX epoch (Midnight, Jan 1, 1970, UTC), at microsecond granularity, expressed as a triple (mega-seconds, seconds, and micro-seconds):

```
%% erlang
{MegaSecs, Secs, MicroSecs} = erlang:timestamp().
```

User `erlang:system_time/1` to obtain the seconds, milliseconds or microseconds since the UNIX epoch (Midnight, Jan 1, 1970, UTC):

```
%% erlang
Seconds = erlang:system_time(second).
MilliSeconds = erlang:system_time(millisecond).
MicroSeconds = erlang:system_time(microsecond).
```

User `erlang:monotonic_time/1` to obtain a (possibly not strictly) monotonically increasing time measurement. Use the same time units to convert to seconds, milliseconds, or microseconds:

```
%% erlang
Seconds = erlang:monotonic_time(second).
MilliSeconds = erlang:monotonic_time(millisecond).
MicroSeconds = erlang:monotonic_time(microsecond).
```

> Note. `erlang:monotonic_time/1` should not be used to calculate the wall clock time, but instead should be used by applications to compute time differences in a manner that is independent of the system time on the device, which might change, for example, due to NTP, leap seconds, or similar operations that may affect the wall time on the device.

Use `erlang:universaltime/0` to get the current time at second resolution, to obtain the year, month, day, hour, minute, and second:

```
%% erlang
{{Year, Month, Day}, {Hour, Minute, Second}} = erlang:universaltime().
```

> Note. Setting the system time is done in a platform-specific manner. For information about how to set system time on the ESP32, see the Network Programming Guide.

To convert a time (in seconds, milliseconds, or microseconds from the UNIX epoch) to a date-time, use the `calendar:system_time_to_universal_time/2` function. For example,

```
%% erlang
Milliseconds = ... %% get milliseconds from the UNIX epoch
{{Year, Month, Day}, {Hour, Minute, Second}} =
calendar:system_time_to_universal_time(Milliseconds, millisecond).
```

Valid time units are `second`, `millisecond`, and `microsecond`.

4.4.8 Date and Time

A `datetime()` is a tuple containing a date and time, where a date is a tuple containing the year, month, and day (in the Gregorian calendar), expressed as integers, and a time is an hour, minute, and second, also expressed in integers.

The following Erlang type specification enumerates this type:

```
%% erlang
-type year() :: integer().
-type month() :: 1..12.
-type day() :: 1..31.
-type date() :: {year(), month(), day()}.
-type gregorian_days() :: integer().
```

```
-type day_of_week() :: 1..7.
-type hour() :: 0..23.
-type minute() :: 0..59.
-type second() :: 0..59.
-type time() :: {hour(), minute(), second()}.
-type datetime() :: {date(), time()}.
```

Erlang/OTP uses the Christian epoch to count time units from year 0 in the Gregorian calendar. The, for example, the value 0 in Gregorian seconds represents the date Jan 1, year 0, and midnight (UTC), or in Erlang terms, `{{0, 1, 1}, {0, 0, 0}}`.

> Note. AtomVM is currently limited to representing integers in at most 64 bits, with one bit representing the sign bit. However, even with this limitation, AtomVM is able to resolve microsecond values in the Gregorian calendar for over 292,000 years, likely well past the likely lifetime of an AtomVM application (unless perhaps launched on a deep space probe).

The `calendar` module provides useful functions for converting dates to Gregorian days, and date-times to Gregorian seconds.

To convert a `date()` to the number of days since January 1, year 0, use the `calendar:date_to_gregorian_days/1` function, e.g.,

```
GregorianDays = calendar:date_to_gregorian_days({2023, 7, 23})
```

To convert a `datetime()` to convert the number of seconds since midnight January 1, year 0, use the `calendar:datetime_to_gregorian_seconds/1` function, e.g.,

```
GregorianSeconds = calendar:datetime_to_gregorian_seconds({{2023, 7, 23}, {13, 31,
7}})
```

> Note. The `calendar` module does not support year values before year `0`.

4.4.9 Miscellaneous

Use `atomvm:random/0` to generate a random unsigned 32-bit integer in the range `0..4294967295`:

```
%% erlang
RandomInteger = atomvm:random().
```

Use `atomvm:random_bytes/1` to return a randomly populated binary of a specified size:

```
%% erlang
RandomBinary = erlang:random_bytes(32).
```

Use `base64:encode/1` and `base64:decode/1` to encode to and decode from Base64 format. The input value to these functions may be a binary or string. The output value from these functions is an Erlang binary.

```
%% erlang
Encoded = base64:encode(<<"foo">>).
<<"foo">> = base64:decode(Encoded).
```

You can Use `base64:encode_to_string/1` and `base64:decode_to_string/1` to perform the same encoding, but to return values as Erlang list structures, instead of as binaries.

4.4.10 StackTraces

You can obtain information about the current state of a process via stacktraces, which provide information about the location of function calls (possibly including file names and line numbers) in your program.

Currently in AtomVM, stack traces can be obtained in one of following ways:

- via try-catch blocks

- via catch blocks, when an error has been raised via the `error` Bif.

  Note. AtomVM does not support `erlang:get_stacktrace/0` which was deprecated in Erlang/OTP 21 and 22, stopped working in Erlang/OTP 23 and was removed in Erlang/OTP 24. Support for accessing the current stacktrace via `erlang:process_info/2` may be added in the future.

For example a stack trace can be bound to a variable in the catch clause in a try-catch block:

```
try
    do_something()
catch
    _Class:_Error:Stacktrace ->
        io:format("Stacktrace: ~p~n", [Stacktrace])
end
```

Alternatively, a stack trace can be bound to the result of a `catch` expression, but only when the error is raised by the `error` Bif. For example,

```
{'EXIT', {foo, Stacktrace}} = (catch error(foo)),
io:format("Stacktrace: ~p~n", [Stacktrace])
```

Stack traces are printed to the console in a crash report, for example, when a process dies unexpectedly.

Stacktrace data is represented as a list of tuples, each of which represents a stack "frame". Each tuple is of the form:

```
[{Module :: module(), Function :: atom(), Arity :: non_neg_integer(), AuxData ::
aux_data()}]
```

where `aux_data()` is a (possibly empty) properties list containing the following elements:

```
[{file, File :: string(), line, Line :: pos_integer()}]
```

Stack frames are ordered from the frame "closest" to the point of failure (the "top" of the stack) to the frame furthest from the point of failure (the "bottom" of the stack).

Stack frames will contain file and line information in the AuxData list if the BEAM files (typically embedded in AVM files) include <<"Line">> chunks generated by the compiler. Otherwise, the AuxData will be an empty list.

> Note. Adding line information to BEAM files not only increases the size of BEAM files in storage, but calculation of file and line information can have a non-negligible impact on memory usage. Memory-sensitive applications should consider not including line information in BEAM files.

The PackBEAM tool does not include file and line information in the AVM files it creates, but file and line information can be included via a command line option. For information about the PackBEAM too, see the `PackBEAM` tool.

4.4.11 Reading data from AVM files

AVM files are generally packed BEAM files, but they can also contain non-BEAM files, such as plain text files, binary data, or even encoded Erlang terms.

Typically, these files are included from the `priv` directory in a build tree, for example, when using the `atomvm_rebar3_plugin`, though the `PackBEAM` tool and the `atomvm_packbeam` tool allow you to specify any location for files to include in AVM files.

By convention, these files obey the following path in an AVM file:

```
<application-name>/priv/<file-path>
```

For example, if you wanted to embed `my_file.txt` into your application AVM file (where your

application name is, for example, `my_application`), you would use:

```
my_application/priv/my_file.txt
```

The `atomvm:read_priv/2` function can then be used to extract the contents of this file into a binary, e.g.,

```
%% erlang
MyFileBin = atomvm:read_priv(my_application, <<"my_file.txt">>)
```

> Note. Embedded files may contain path separators, so for example `<<"my_files/my_file.txt">>` would be used if the AVM file embeds `my_file.txt` using the path `my_application/priv/my_files/my_file.txt`

For more information about how to embed files into AVM files, see the `atomvm_rebar3_plugin`.

4.4.12 Code Loading

AtomVM provides a limited set of APIs for loading code and data embedded dynamically at runtime.

To load an AVM file from binary data, use the `atomvm:add_avm_pack_binary/2` function. Supply a reference to the AVM data, together with a (possibly empty) list of options. Specify a `name` option, whose value is an atom, if you wish to close the AVM data at a later point in the program.

For example:

```
%% erlang
AVMData = ... %% load AVM data into memory as a binary
ok = atomvm:add_avm_pack_binary(AVMData, [{name, my_avm}])
```

You can also load AVM data from a file (on the `generic_unix` platform) or from a flash partition (on ESP32 platforms) using the `atomvm:add_avm_pack_file/2` function. Specify a string (or binary) as the path to the AVM file, together with a list of options, such as `name`.

For example:

```
%% erlang
ok = atomvm:add_avm_pack_file("/path/to/file.avm", [{name, my_avm}])
```

On `esp32` platforms, the partition name should be prefixed with the string `/dev/partition/by-name/`. Thus, for example, if you specify `/dev/partition/by-name/main2.avm` as the partition, the ESP32 flash should contain a data partition with the name `main2.avm`

For example:

```
%% erlang
ok = atomvm:add_avm_pack_file("/dev/partition/by-name/main2.avm", [])
```

To close a previous opened AVM by name, use the `atomvm:close_avm_pack/2` function. Specify the name of the AVM pack used to add

```
%% erlang
ok = atomvm:close_avm_pack(my_avm, [])
```

> Note. Currently, the options parameter is ignored, so use the empty list (`[]`) for foward compatibility.

You can load an individual BEAM file using the `code:load_binary/3` function. Specify the Module name (as an atom), as well as the BEAM data you have loaded into memory.

For Example:

```
%% erlang
BEAMData = ... %% load BEAM data into memory as a binary
{module, Module} = code:load_binary(Module, Filename, BEAMData)
```

> Note. The `Filename` parameter is currently ignored.

You can load an individual BEAM file from the file system using the `code:load_abs/1` function. Specify the path to the BEAM file. This path should not include the `.beam` extension, as this extension will be added automatically.

For example:

```
{module, Module} = code:load_abs("/path/to/beam/file/without/beam/extension")
```

> Note. This function is currently only supported on the `generic_unix` platform.

4.4.13 Math

AtomVM supports the following standard functions from the OTP `math` module:

- `cos/1`
- `acos/1`
- `acosh/1`
- `asin/1`
- `asinh/1`
- `atan/1`
- `atan2/2`
- `atanh/1`
- `ceil/1`
- `cosh/1`
- `exp/1`
- `floor/1`
- `fmod/2`
- `log/1`
- `log10/1`
- `log2/1`
- `pow/2`
- `sin/1`
- `sinh/1`
- `sqrt/1`
- `tan/1`
- `tanh/1`
- `pi/0`

The input values for these functions may be `float` or `integer` types. The return value is always a value of `float` type.

Input values that are out of range for the specific mathematical function or which otherwise are invalid or yield an invalid result (e.g., division by 0) will result in a `badarith` error.

> Note. If the AtomVM virtual machine is built with floating point arithmetic support disabled, these functions will result in a `badarg` error.

4.4.14 Cryptographic Operations

You can hash binary date using the `crypto:hash/2` function.

```
%% erlang
crypto:hash(sha, [<<"Some binary">>, $\s, "data"])
```

This function takes a hash algorithm, which may be one of:

```
-type md_type() :: md5 | sha | sha224 | sha256 | sha384 | sha512.
```

and an IO list. The output type is a binary, who's length (in bytes) is dependent on the algorithm chosen:

Algorithm	Hash Length (bytes)
`md5`	16
`sha`	20
`sha224`	32
`sha256`	32
`sha384`	64
`sha512`	64

> Note. The `crypto:hash/2` function is currently only supported on the ESP32 and generic UNIX platforms.

You can also use the legacy `erlang:md5/1` function to compute the MD5 hash of an input binary. The output is a fixed-length binary (16 bytes)

```
%% erlang
Hash = erlang:md5(<<foo>>).
```

On ESP32, you can perform symetric encryption and decryption of any iodata data using `crypto_one_time/4,5` function.

Following ciphers are supported:

Without IV (using `crypto_one_time/4`):

- `aes_128_ecb`
- `aes_192_ecb`
- `aes_256_ecb`

With IV (using `crypto_one_time/5`):

- `aes_128_cbc`
- `aes_192_cbc`
- `aes_256_cbc`
- `aes_128_cfb128`
- `aes_192_cfb128`
- `aes_256_cfb128`
- `aes_128_ctr`
- `aes_192_ctr`
- `aes_256_ctr`

The function is implemented using mbedTLS, so please to its documentation for further details.

Please refer to Erlang crypto documentation for additional details about these two functions.

> Note: mbedTLS doesn't support padding for ciphers other than CCB, so block size must be accounted otherwise output will be truncated.

4.5 ESP32-specific APIs

Certain APIs are specific to and only supported on the ESP32 platform. This section describes these APIs.

4.5.1 System-Level APIs

As noted above, the `erlang:system_info/1` function can be used to obtain system-specific information about the platform on which your application is deployed.

You can request ESP32-specific information using using the following input atoms:

- `esp_free_heap_size` Returns the available free space in the ESP32 heap.
- `esp_largest_free_block` Returns the size of the largest free continuous block in the ESP32 heap.
- `esp_get_minimum_free_size` Returns the smallest ever free space available in the ESP32 heap since boot, this will tell you how close you have come to running out of free memory.
- `esp_chip_info` Returns map of the form `#{features := Features, cores := Cores, revision := Revision, model := Model}`, where `Features` is a list of features enabled in the chip, from among the following atoms: `[emb_flash, bgn, ble, bt]`; `Cores` is the number of CPU cores on the chip; `Revision` is the chip version; and `Model` is one of the following atoms: `esp32`, `esp32_s2`, `esp32_s3`, `esp32_c3`.
- `esp_idf_version` Return the IDF SDK version, as a string.

For example,

```
%% erlang
FreeHeapSize = erlang:system_info(esp_free_heap_size).
```

4.5.2 Non-volatile Storage

AtomVM provides functions for setting, retrieving, and deleting key-value data in binary form in non-volatile storage (NVS) on an ESP device. Entries in NVS survive reboots of the ESP device, and can be used a limited "persistent store" for key-value data.

> Note. NVS storage is limited in size, and NVS keys are restricted to 15 characters. Try to avoid writing frequently to NVS storage, as the flash storage may degrade more rapidly with repeated writes to the medium.

NVS entries are stored under a namespace and key, both of which are expressed as atoms. AtomVM uses the namespace `atomvm` for entries under its control. Applications may read from and write to the `atomvm` namespace, but they are strongly discouraged from doing so, except when explicitly stated otherwise.

To set a value in non-volatile storage, use the `esp:set_binary/3` function, and specify a namespace, key, and value:

```
%% erlang
Namespace = <<"my-namespace">>,
Key = <<"my-key">>,
esp:set_binary(Namespace, Key, <<"some-value">>).
```

To retrieve a value in non-volatile storage, use the `esp:get_binary/2` function, and specify a namespace and key. You can optionally specify a default value (of any desired type), if an entry does not exist in non-volatile storage:

```
%% erlang
```

```
Value = esp:get_binary(Namespace, Key, <<"default-value">>).
```

To delete an entry, use the `esp:erase_key/2` function, and specify a namespace and key:

```
%% erlang
ok = esp:erase_key(Namespace, Key).
```

You can delete all entries in a namespace via the `esp:erase_all/1` function:

```
%% erlang
ok = esp:erase_all(Namespace).
```

Finally, you can delete all entries in all namespaces on the NVS partition via the `esp:reformat/0` function:

```
%% erlang
ok = esp:reformat().
```

Applications should use the `esp:reformat/0` function with caution, in case other applications are making using the non-volatile storage.

> Note. NVS entries are currently stored in plaintext and are not encrypted. Applications should exercise caution if sensitive security information, such as account passwords, are stored in NVS storage.

4.5.3 Restart and Deep Sleep

You can use the `esp:restart/0` function to immediately restart the ESP32 device. This function does not return a value.

```
%% erlang
esp:restart().
```

Use the `esp:reset_reason/0` function to obtain the reason for the ESP32 restart. Possible values include:

- `esp_rst_unknown`
- `esp_rst_poweron`
- `esp_rst_ext`
- `esp_rst_sw`
- `esp_rst_panic`
- `esp_rst_int_wdt`
- `esp_rst_task_wdt`
- `esp_rst_wdt`
- `esp_rst_deepsleep`
- `esp_rst_brownout`
- `esp_rst_sdio`

Use the `esp:deep_sleep/1` function to put the ESP device into deep sleep for a specified number of milliseconds. Be sure to safely stop any critical processes running before this function is called, as it will cause an immediate shutdown of the device.

```
%% erlang
esp:deep_sleep(60*1000).
```

Use the `esp:sleep_get_wakeup_cause/0` function to inspect the reason for a wakeup. Possible return values include:

- `sleep_wakeup_ext0`

- `sleep_wakeup_ext1`
- `sleep_wakeup_timer`
- `sleep_wakeup_touchpad`
- `sleep_wakeup_ulp`
- `sleep_wakeup_gpio`
- `sleep_wakeup_uart`
- `sleep_wakeup_wifi`
- `sleep_wakeup_cocpu`
- `sleep_wakeup_cocpu_trag_trig`
- `sleep_wakeup_bt`
- `undefined` (no sleep wakeup)
- `error` (unknown other reason)

The values matches the semantics of `esp_sleep_get_wakeup_cause`.

```
%% erlang
case esp:sleep_get_wakeup_cause() of
    sleep_wakeup_timer ->
        io:format("Woke up from a timer~n");
    sleep_wakeup_ext0 ->
        io:format("Woke up from ext0~n");
    sleep_wakeup_ext1 ->
        io:format("Woke up from ext1~n");
    _ ->
        io:format("Woke up for some other reason~n")
end.
```

Use the `esp:sleep_enable_ext0_wakeup/2` and `esp:sleep_enable_ext1_wakeup` functions to configure ext0 and ext1 wakeup mechanisms. They follow the semantics of `esp_sleep_enable_ext0_wakeup` and `esp_sleep_enable_ext1_wakeup`.

```
%% erlang
-spec shutdown() -> no_return().
shutdown() ->
    % Configure wake up when GPIO 37 is set to low (M5StickC main button)
    ok = esp:sleep_enable_ext0_wakeup(37, 0),
    % Deep sleep for 1 hour
    esp:deep_sleep(60*60*1000).
```

RTC Memory

On ESP32 systems, you can use (slow) "real-time clock" memory to store data between deep sleeps. This storage can be useful, for example, to store interim state data in your application.

> Note. RTC memory is initialized if the device is reset.

To store data in RTC slow memory, use the `esp:rtc_slow_set_binary/1` function:

```
%% erlang
esp:rtc_slow_set_binary(<<"some binary data">>)
```

To retrieve data in RTC slow memory, use the `esp:rtc_slow_get_binary/0` function:

```
%% erlang
Data = esp:rtc_slow_get_binary()
```

By default, RTC slow memory in AtomVM is limited to 4098 (4k) bytes. This value can be modified at build time using an IDF SDK `KConfig` setting. For instructions about how to build AtomVM, see

the AtomVM Build Instructions.

4.5.4 Miscellaneous

The `freq_hz` function can be used to retrieve the clock frequency of the chip.

- `esp:freq_hz/0`

The `esp:partition_list/0` function can be used to retrieve information about the paritions on an ESP32 flash.

The return type is a list of tuples, each of which contains the partition id (as a binary), partition type and sub-type (both of which are represented as integers), the start of the partition as an address along with its size, as well as a list of properties about the partition, as a properties list.

```
%% erlang
PartitionList = esp:partition_list(),
lists:foreach(
    fun({PartitionId, PartitionType, PartitionSubtype, PartitionAddress,
PartitionSize, PartitionProperties}) ->
        %% ...
    end,
    PartitionList
)
```

Note. The partition properties are currently empty (`[]`).

For information about the encoding of partition types and sub-types, see the IDF SDK partition type definitions.

- `esp:get_mac/1`

The `esp:get_mac/1` function can be used to retrieve the network Media Access Control (MAC) address for a given interface, `wifi_sta` or `wifi_softap`. The return value is a 6-byte binary, in accordance with the IEEE 802 family of specifications.

```
%% erlang
MacAddress = esp:get_mac(wifi_sta)
```

4.6 Peripherals

The AtomVM virtual machine and libraries support APIs for interfacing with peripheral devices connected to the ESP32. This section provides information about these APIs.

4.6.1 GPIO

You can read and write digital values on GPIO pins using the `gpio` module, using the `digital_read/1` and `digital_write/2` functions. You must first set the direction of the pin using the `gpio:set_direction/2` function, using `input` or `output` as the direction parameter.

To read the value of a GPIO pin (`high` or `low`), use `gpio:digital_read/1`:

```
%% erlang
Pin = 2,
gpio:set_direction(Pin, input),
case gpio:digital_read(Pin) of
    high ->
        io:format("Pin ~p is high ~n", [Pin]);
    low ->
        io:format("Pin ~p is low ~n", [Pin])
end.
```

To set the value of a GPIO pin (`high` or `low`), use `gpio:digital_write/2`:

```
%% erlang
Pin = 2,
gpio:set_direction(Pin, output),
gpio:digital_write(Pin, low).
```

Interrupt Handling

You can get notified of changes in the state of a GPIO pin by using the `gpio:set_int/2` function. This function takes a reference to a GPIO Pin and a trigger. Allowable triggers are `rising`, `falling`, `both`, `low`, `high`, and `none` (to disable an interrupt).

When a trigger event occurs, such as a pin rising in voltage, a tuple will be delivered to the process containing the atom `gpio_interrupt` and the pin.

```
%% erlang
Pin = 2,
gpio:set_direction(Pin, input),
GPIO = gpio:open(),
ok = gpio:set_int(GPIO, Pin, rising),
receive
    {gpio_interrupt, Pin} ->
        io:format("Pin ~p is rising ~n", [Pin])
end.
```

Interrupts can be removed by using the `gpio:remove_int/2` function.

Use the `gpio:close/1` function to close the GPIO driver and free any resources in use by it, supplying a reference to a previously opened GPIO driver instance. Any references to the closed GPIO instance are no longer valid after a successful call to this function, and all interrupts will be removed.

```
%% erlang
ok = gpio:close(GPIO).
```

Since only one instance of the GPIO driver is allowed, you may also simply use `gpio:stop/0` to remove all interrupts, free the resouces, and close the GPIO driver port.

```
%% erlang
ok = gpio:stop().
```

4.6.2 I2C

The `i2c` module encapsulates functionality associated with the 2-wire Inter-Integrated Circuit (I2C) interface.

> Note. Information about the ESP32 I2C interface can be found in the IDF SDK I2C Documentation.

The AtomVM I2C implementation uses the AtomVM Port mechanism and must be initialized using the `i2c:open/1` function. The single parameter contains a properties list, with the following elements:

Key	Value Type	Required	Description
`scl_io_num`	`integer()`	yes	I2C clock pin (SCL)
`sda_io_num`	`integer()`	yes	I2C data pin (SDA)
`i2c_clock_hz`	`integer()`	yes	I2C clock frequency (in hertz)
`i2c_num`	`0 .. I2C_NUM_MAX - 1`	no (default: `0`)	I2C port number. `I2C_NUM_MAX` is defined by the device SDK. On ESP32, this value is 1.

For example,

```
%% erlang
I2C = i2c:open([
    {scl_io_num, 21}, {sda_io_num, 22}, {i2c_clock_hz, 40000}
])
```

Once the port is opened, you can use the returned `I2C` instance to read and write bytes to the attached device.

Both read and write operations require the I2C bus address from which data is read or to which data is written. A devices address is typically hard-wired for the specific device type, or in some cases may be changed by the addition or removal of a resistor.

In addition, you may optionally specify a register to read from or write to, as some devices require specification of a register value. Consult your device's data sheet for more information and the device's I2C bus address and registers, if applicable.

There are two patterns for writing data to an I2C device:

1. Queuing `i2c:qwrite_bytes/2,3` write operations between calls to `i2c:begin_transmission/1` and `i2c:end_transmission/1`. In this case, write operations are queued locally and dispatched to the target device when the `i2c:end_transmission/1` operation is called;
2. Writing a byte or sequence of bytes in one `i2c:write_bytes/2,3` operation.

The choice of which pattern to use will depend on the device being communicated with. For example, some devices require a sequence of write operations to be queued and written in one atomic write, in which case the first pattern is appropriate. E.g.,

```
%% erlang
ok = i2c:begin_transmission(I2C),
ok = i2c:qwrite_bytes(I2C, DeviceAddress, Register1, <<"some sequence of
bytes">>),
ok = i2c:qwrite_bytes(I2C, DeviceAddress, Register2, <<"some other of bytes">>),
ok = i2c:end_transmission(I2C),
```

In other cases, you may just need to write a byte or sequence of bytes in one operation to the device:

```
%% erlang
ok = i2c:write_bytes(I2C, DeviceAddress, Register1, <<"write it all in one go">>),
```

Reading bytes is more straightforward. Simply use `i2c:read_bytes/3,4`, specifying the port instance, device address, optionally a register, and the number of bytes to read:

```
%% erlang
{ok, BinaryData} = i2c:read_bytes(I2C, DeviceAddress, Register, Len)
```

To close the I2C driver and free any resources in use by it, use the `i2c:close/1` function, supplying a reference to the I2C driver instance created via `i2c:open/1`:

```
%% erlang
ok = i2c:close(I2C)
```

Once the I2C driver is closed, any calls to `i2c` functions using a reference to the I2C driver instance should return with the value `{error, noproc}`.

4.6.3 SPI

The `spi` module encapsulates functionality associated with the 4-wire Serial Peripheral Interface (SPI) in leader mode.

> Note. Information about the ESP32 SPI leader mode interface can be found in the IDF SDK SPI Documentation.

The AtomVM SPI implementation uses the AtomVM Port mechanism and must be initialized using the `spi:open/1` function. The single parameter to this function is a properties list containing two elements:

- `bus_config` – a properties list containing entries for the SPI bus
- `device_config` – a properties list containing entries for each device attached to the SPI Bus

The `bus_config` properties list contains the following entries:

Key	Value Type	Required	Description
`miso_io_num`	`integer()`	yes	SPI leader-in, follower-out pin (MOSI)
`mosi_io_num`	`integer()`	yes	SPI leader-out, follower-in pin (MISO)
`sclk_io_num`	`integer()`	yes	SPI clock pin (SCLK)

The `device_config` entry is a properties list containing entries for each device attached to the SPI Bus. Each entry in this list contains the user-selected name (as an atom) of the device, followed by configuration for the named device.

Each device configuration is a properties list containing the following entries:

Key	Value Type	Required	Description
`spi_clock_hz`	`integer()`	yes	SPI clock frequency (in hertz)
`mode`	`0..3`	yes	SPI mode, indicating clock polarity (`CPOL`) and clock phase (`CPHA`). Consult the SPI specification and data sheet for your device, for more information about how to control the behavior of the SPI clock.
`spi_cs_io_num`	`integer()`	yes	SPI chip select pin (CS)
`address_len_bits`	`0..64`	yes	number of bits in the address field of a read-/write operation (for example, 8, if the transaction address field is a single byte)
`command_len_bits`	`0..16`	default: 0	number of bits in the command field of a read-/write operation (for example, 8, if the transaction command field is a single byte)

For example,

```
%% erlang
SPIConfig = [
    {bus_config, [
        {miso_io_num, 19},
        {mosi_io_num, 27},
        {sclk_io_num, 5}
    ]},
    {device_config, [
        {my_device_1, [
            {spi_clock_hz, 1000000},
            {mode, 0},
            {spi_cs_io_num, 18},
            {address_len_bits, 8}
        ]}
        {my_device_2, [
            {spi_clock_hz, 1000000},
            {mode, 0},
            {spi_cs_io_num, 15},
            {address_len_bits, 8}
        ]}
    ]}
],
```

```
SPI = spi:open(SPIConfig),
...
```

In the above example, there are two SPI devices, one using pin 18 chip select (named `my_device_1`), and once using pin 15 chip select (named `my_device_2`).

Once the port is opened, you can use the returned `SPI` instance, along with the selected device name, to read and write bytes to the attached device.

To read a byte at a given address on the device, use the `spi:read_at/4` function:

```
%% erlang
{ok, Byte} = spi:read_at(SPI, DeviceName, Address, 8)
```

To write a byte at a given address on the device, use the `spi_write_at/5` function:

```
%% erlang
write_at(SPI, DeviceName, Address, 8, Byte)
```

> Note. The `spi:write_at/5` takes integer values as inputs and the `spi:read_at/4` returns integer values. You may read and write up to 32-bit integer values via these functions.

Consult your local device data sheet for information about various device addresses to read from or write to, and their semantics.

The above functions are optimized for small reads and writes to an SPI device, typically one byte at a time.

The SPI interface also supports a more generic way to read and write from an SPI device, supporting arbitrary-length reads and writes, as well as a number of different "phases" of writes, per the SPI specification.

These phases include:

- Command phase – write of an up-to 16-bit command to the SPI device
- Address Phase – write of an up-to 64-bit address to the SPI device
- Data Phase – read or write of an arbitrary amount of of data to and from the device.

Any one of these phases may be included or omitted in any given SPI transaction.

In order to achieve this level of flexibility, these functions allow users to specify the SPI transaction through a map structure, which includes fields that specify the behavior of an SPI transaction.

The following table enumerates the permissible fields in this structure:

Key	Value Type	Description
`command`	`integer()` (16-bit)	(Optional) SPI command. The low-order `command_len_bits` are written to the device.
`address`	`integer()` (64-bit)	(Optional) Device address. The low-order `address_len_bits` are written to the device.
`write_data`	`binary()`	(Optional) Data to write
`write_bits`	`non_neg_integer()`	Number of bits to write from `write_data'. If not included,then all bits will be written.
`read_bits`	`non_neg_integer()`	Number of bits to read from the SPI device. If not included, then the same number of bits will be read as were written.

To write a blob of data to the SPI device, for example, you would use:

```
WriteData = <<"some binary data">>,
ok = spi:write(SPI, DeviceName, #{write_data => WriteData})
```

To write and simultaneously read back a blob of data to the SPI device, you would use:

```
{ok, ReadData} = spi:write_read(SPI, DeviceName, #{write_data => WriteData})
```

The size of the returned data is the same as the size of the written data, unless otherwise specified by the `read_bits` field.

Use the `spi:close/1` function to close the SPI driver and free any resources in use by it, supplying a reference to a previously opened SPI driver instance. Any references to the closed SPI instance are no longer valid after a successful call to this function.

```
%% erlang
ok = spi:close(SPI).
```

4.6.4 UART

The `uart` module encapsulates functionality associated with the Universal Asynchronous Receiver/-Transmitter (UART) interface supported on ESP32 devices. Some devices, such as NMEA GPS receivers, make use of this interface for communicating with an ESP32.

> Note. Information about the ESP32 UART interface can be found in the IDF SDK UART Documentation.

The AtomVM UART implementation uses the AtomVM Port mechanism and must be initialized using the `uart:open/2` function.

The first parameter indicates the ESP32 UART hardware interface. Legal values are:

```
"UART0" | "UART1" | "UART2"
```

The selection of the hardware interface dictates the default RX and TX pins on the ESP32:

Port	RX pin	TX pin
`UART0`	GPIO_3	GPIO_1
`UART1`	GPIO_9	GPIO_10
`UART2`	GPIO_16	GPIO_17

The second parameter is a properties list, containing the following elements:

Key	Value Type	Required	Default Value	Description
`speed`	`integer()`	no	115200	UART baud rate (bits/sec)
`data_bits`	`5 | 6 | 7 | 8`	no	8	UART data bits
`stop_bits`	`1 | 2`	no	1	UART stop bits
`flow_control`	`hardware | software | none`	no	`none`	Flow control
`parity`	`even | odd | none`	no	`none`	UART parity check

For example,

```
%% erlang
UART = uart:open("UART0", [{speed, 9600}])
```

Once the port is opened, you can use the returned `UART` instance to read and write bytes to the attached device.

To read data from the UART channel, use the `uart:read/1` function. The return value from this function is a binary:

```
%% erlang
Bin = uart:read(UART)
```

To write data to the UART channel, use the `uart_write/2` function. The input data is any Erlang I/O list:

```
%% erlang
uart:write(UART, [<<"any">>, $d, $a, $t, $a, "goes", <<"here">>])
```

Consult your local device data sheet for information about the format of data to be read from or written to the UART channel.

To close the UART driver and free any resources in use by it, use the `uart:close/1` function, supplying a reference to the UART driver instance created via `uart:open/2`:

```
%% erlang
ok = uart:close(UART)
```

Once the UART driver is closed, any calls to `uart` functions using a reference to the UART driver instance should return with the value `{error, noproc}`.

4.6.5 LED Control

The LED Control API can be used to drive LEDs, as well as generate PWM signals on GPIO pins.

The LEDC API is encapsulated in the `ledc` module and is a direct translation of the IDF SDK LEDC API, with a natural mapping into Erlang. This API is intended for users with complex use-cases, and who require low-level access to the LEDC APIs.

The `ledc.hrl` module should be used for common modes, channels, duty cycle resolutions, and so forth.

```
%% erlang
-include("ledc.hrl").

...

%% create a 5khz timer
SpeedMode = ?LEDC_HIGH_SPEED_MODE,
Channel = ?LEDC_CHANNEL_0,
ledc:timer_config([
    {duty_resolution, ?LEDC_TIMER_13_BIT},
    {freq_hz, 5000},
    {speed_mode, ?LEDC_HIGH_SPEED_MODE},
    {timer_num, ?LEDC_TIMER_0}
]).

%% bind pin 2 to this timer in a channel
ledc:channel_config([
    {channel, Channel},
    {duty, 0},
    {gpio_num, 2},
    {speed_mode, ?LEDC_HIGH_SPEED_MODE},
    {hpoint, 0},
    {timer_sel, ?LEDC_TIMER_0}
]).

%% set the duty cycle to 0, and fade up to 16000 over 5 seconds
ledc:set_duty(SpeedMode, Channel, 0).
ledc:update_duty(SpeedMode, Channel).
TargetDuty = 16000.
FadeMs = 5000.
ok = ledc:set_fade_with_time(SpeedMode, Channel, TargetDuty, FadeMs).
```

4.7 Protocols

AtomVM supports network programming on devices that support it, specifically the ESP32 platform,

with its built-in support for WIFI networking, and of course on the UNIX platform.

This section describes the network programming APIs available on AtomVM.

4.7.1 Network (ESP32 only)

The ESP32 supports WiFi connectivity as part of the built-in WiFi and Bluetooth radio (and in most modules, an integrated antenna). The WIFI radio on an ESP32 can operate in several modes:

- STA (Station) mode, whereby it acts as a member of an existing WiFi network;
- AP (Access Point) mode, whereby the ESP32 acts as an access point for other devices; or
- AP+STA mode, whereby the ESP32 behaves both as a member of an existing WiFi network and as an access point for other devices.

AtomVM supports these modes of operation via the `network` module, which is used to initialize the network and allow applications to respond to events within the network, such as a network disconnect or reconnect, or a connection to the ESP32 from another device.

> Note. Establishment and maintenance of network connections on roaming devices is a complex and subtle art, and the AtomVM `network` module is designed to accommodate as many IoT scenarios as possible. This section of the programmer's guide is deliberately brief and only addresses the most basic scenarios. For a more detailed explanation of the AtomVM `network` module and its many use-cases, please refer to the AtomVM Network Programming Guide.

STA mode

To connect your ESP32 to an existing WiFi network, use the `network:wait_for_sta/1,2` convenience function, which abstracts away some of the more complex details of ESP32 STA mode.

This function takes a station mode configuration, as a properties list, and optionally a timeout (in milliseconds) before connecting to the network should fail. The default timeout, if unspecified, is 15 seconds.

The station mode configuration supports the following options:

Key	Value Type	Required	Default Value	Description
`ssid`	`string()` | `binary()`	yes	-	WiFi AP SSID
`psk`	`string()` | `binary()`	yes, if network is encrypted	-	WiFi AP password
`dhcp_hostname`	`string()` | `binary()`	no	`atomvm-<MAC>` where `<MAC>` is the factory-assigned MAC-address of the device	DHCP hostname for the connecting device

> Note. The WiFi network to which you are connecting must support DHCP and IPv4. IPv6 addressing is not yet supported on AtomVM.

If the ESP32 device connects to the specified network successfully, the device's assigned address, netmask, and gateway address will be returned in an `{ok, ...}` tuple; otherwise, an error is returned.

For example:

```
%% erlang
Config = [
    {ssid, <<"myssid">>},
    {psk,  <<"mypsk">>},
    {dhcp_hostname, <<"mydevice">>}
],
case network:wait_for_sta(Config, 15000) of
```

```
    {ok, {Address, _Netmask, _Gateway}} ->
        io:format("Acquired IP address: ~p~n", [Address]);
    {error, Reason} ->
        io:format("Network initialization failed: ~p~n", [Reason])
end
```

Once connected to a WiFi network, you may begin TCP or UDP networking, as described in more detail below.

For information about how to handle disconnections and reconnections to a WiFi network, see the AtomVM Network Programming Guide.

AP mode

To turn your ESP32 into an access point for other devices, you can use the `network:wait_for_ap/1,2` convenience function, which abstracts away some of the more complex details of ESP32 AP mode. When the network is started, the ESP32 device will assign itself the `192.168.4.1` address. Any devices that connect to the ESP32 will take addresses in the `192.168.4/24` network.

This function takes an access point mode configuration, as a properties list, and optionally a timeout (in milliseconds) before starting the network should fail. The default timeout, if unspecified, is 15 seconds.

The access point mode configuration supports the following options:

Key	Value Type	Required	Default Value	Description
`ssid`	`string()` | `binary()`	no	`atomvm-<MAC>` where `<MAC>` is the factory-assigned MAC-address of the device	WiFi AP SSID
`ssid_hidden`	`boolean()`	no	`false`	Whether the AP SSID should be hidden (i.e., not broadcast)
`psk`	`string()` | `binary()`	yes, if network is encrypted	-	WiFi AP password. Warning: If this option is not specified, the network will be an open network, to which anyone who knows the SSID can connect and which is not encrypted.

`ap_max_connections`	`non_neg_integer()`	no	`4`	Maximum number of devices that can be connected to this AP

If the ESP32 device starts the AP network successfully, the `ok` atom is returned; otherwise, an error is returned.

For example:

```
%% erlang
Config = [
    {psk,  <<"mypsk">>}
],
case network:wait_for_ap(Config, 15000) of
    ok ->
        io:format("AP network started at 192.168.4.1~n");
    {error, Reason} ->
        io:format("Network initialization failed: ~p~n", [Reason])
end
```

Once the WiFi network is started, you may begin TCP or UDP networking, as described in more detail below.

For information about how to handle connections and disconnections from attached devices, see the AtomVM Network Programming Guide.

STA+AP mode

For information about how to run the AtomVM network in STA and AP mode simultaneously, see the AtomVM Network Programming Guide.

SNTP

For information about how to use SNTP to synchronize the clock on your device, see the AtomVM Network Programming Guide.

4.7.2 UDP

AtomVM supports network programming using the User Datagram Protocol (UDP) via the `gen_udp` module. This modules obeys the syntax and semantics of the Erlang/OTP `gen_udp` interface.

> Note. Not all of the Erlang/OTP `gen_udp` functionality is implemented in AtomVM. For details, consults the AtomVM API documentation.

To open a UDP port, use the `gen_udp:open/1,2` function. Supply a port number, and if your application plans to receive UDP messages, specify that the port is active via the `{active, true}` property in the optional properties list.

For example:

```
%% erlang
Port = 44404,
case gen_udp:open(Port, [{active, true}, binary]) of
    {ok, Socket} ->
        {ok, SockName} = inet:sockname(Socket)
        io:format("Opened UDP socket on ~p.~n", [SockName])
    Error ->
        io:format("An error occurred opening UDP socket: ~p~n", [Error])
end
```

If the port is active, you can receive UDP messages in your application. They will be delivered as a 5-tuple, starting with the `udp` atom, and containing the socket, address and port from which

the message was sent, as well as the datagram packet, itself, as a list (by default) or a binary. To choose the format, pass `list` or `binary` in options, as with Erlang/OTP.

```
%% erlang
receive
    {udp, _Socket, Address, Port, Packet} ->
        io:format("Received UDP packet ~p from address ~p port ~p~n", [Packet,
Address, Port)])
end,
```

With a reference to a UDP `Socket`, you can send messages to a target UDP endpoint using the `gen_udp:send/4` function. Specify the UDP socket returned from `gen_udp:open/1,2`, the address (as a 4-tuple of octets), port number, and the datagram packet to send:

```
Packet = <<":?
```

> Note. IPv6 networking is not currently supported in AtomVM.

4.7.3 TCP

AtomVM supports network programming using the Transport Connection Protocol (TCP) via the `gen_tcp` module. This modules obeys the syntax and semantics of the Erlang/OTP `gen_tcp` interface.

> Note. Not all of the Erlang/OTP `gen_tcp` functionality is implemented in AtomVM. For details, consults the AtomVM API documentation.

Server-side TCP

Server side TCP requires opening a listening socket, and then waiting to accept connections from remote clients. Once a connection is established, the application may then use a combination of sending and receiving packets over the established connection to or from the remote client.

> Note. Programming TCP on the server-side using the `gen_tcp` interface is a subtle art, and this portion of the documentation will not go into all of the design choices available when designing a TCP application.

Start by opening a listening socket using the `gen_tcp:listen/2` function. Specify the port number on which the TCP server should be listening:

```
%% erlang
case gen_tcp:listen(44405, []) of
    {ok, ListenSocket} ->
        {ok, SockName} = inet:sockname(Socket),
        io:format("Listening for connections at address ~p.~n", [SockName]),
        spawn(fun() -> accept(ListenSocket) end);
    Error ->
        io:format("An error occurred listening: ~p~n", [Error])
end.
```

In this particular example, the server will spawn a new process to wait to accept a connection from a remote client, by calling the `gen_tcp:accept/1` function, passing in a reference to the listening socket. This function will block until a client has established a connection with the server.

When a client connects, the function will return a tuple `{ok, Socket}`, where `Socket` is a reference to the connection between the client and server:

```
%% erlang
accept(ListenSocket) ->
    io:format("Waiting to accept connection...~n"),
    case gen_tcp:accept(ListenSocket) of
        {ok, Socket} ->
            {ok, SockName} = inet:sockname(Socket),
            {ok, Peername} = inet:peername(Socket),
            io:format("Accepted connection.  local: ~p peer: ~p~n", [SockName,
```

```
Peername]),
            spawn(fun() -> accept(ListenSocket) end),
            echo();
        Error ->
            io:format("An error occurred accepting connection: ~p~n", [Error])
    end.
```

Note that immediately after accepting a connection, this example code will spawn a new process to accept any new connections from other clients.

The socket returned from `gen_tcp:accept/1` can then be used to send and receive messages to the connected client:

```
%% erlang
echo() ->
    io:format("Waiting to receive data...~n"),
    receive
        {tcp_closed, _Socket} ->
            io:format("Connection closed.~n"),
            ok;
        {tcp, Socket, Packet} ->
            {ok, Peername} = inet:peername(Socket),
            io:format("Received packet ~p from ~p.  Echoing back...~n", [Packet,
Peername]),
            gen_tcp:send(Socket, Packet),
            echo()
    end.
```

In this case, the server program will continuosuly echo the received input back to the client, until the client closes the connection.

For more information about the `gen_tcp` server interface, consult the AtomVM API Reference Documentation.

Client-side TCP

Client side TCP requires establishing a connection with an endpoint, and then using a combination of sending and receiving packets over the established connection.

Start by opening a connection to another TCP endpoint using the `gen_tcp:connect/3` function. Supply the address and port of the TCP endpoint.

For example:

```
%% erlang
Address = {192, 168, 1, 101},
Port = 44405,
case gen_tcp:connect(Address, Port, []) of
    {ok, Socket} ->
        {ok, SockName} = inet:sockname(Socket),
        {ok, Peername} = inet:peername(Socket),
        io:format("Connected to ~p from ~p~n", [Peername, SockName]);
    Error ->
        io:format("An error occurred connecting: ~p~n", [Error])
end
```

Once a connection is established, you can use a combination of

```
%% erlang
SendPacket = <<":?
```

For more information about the `gen_tcp` client interface, consults the AtomVM API documentation.

Chapter 5

Example Programs

AtomVM includes a collection of useful examples for getting started. This section describes what these examples do, and how to run them, for example, on an ESP32 device.

5.1 Erlang Examples

Erlang examples may be run in the UNIX shell or on supported microcontroller devices.

5.1.1 `hello_world`

This example program prints the string "Hello World" and quits.

Command line

The `hello_world.avm` file will get created as part of a build. This file may be supplied as an argument to the `AtomVM` command:

```
shell$ ./src/AtomVM ./examples/erlang/hello_world.avm
hello_world
Return value: ok
```

5.1.2 `udp_server`

This example program listens on UDP port 44444 and will print information about the received message, including the source IP, (ephemeral) source port, and packet received, to the console.

Command line

The `udp_server.avm` file will get created as part of a build. This file may be supplied as an argument to the `AtomVM` command:

```
shell$ ./src/AtomVM ./examples/erlang/udp_server.avm
Opened UDP socket on "0.0.0.0:44404".
Waiting to receive data...
```

You can send UDP packets to the AtomVM instance using `netcat` (or `nc` on some platforms), in a separate terminal window:

```
shell$ nc -u localhost 44404
```

This command will wait for you to enter a line of text, e.g.,

```
testing 1 2 3
```

In the AtomVM termianl window, you see:

```
Received UDP packet <<116,101,115,116,105,110,103,32,49,32,50,32,51,10>> from
```

```
"127.0.0.1:55261"
Waiting to receive data...
```

Note. Netcat appends a newline character at the end of the input, so the packet binary does not display as printable text.

5.1.3 udp_client

This example program send the packet of data (":?.) to the console, every time it sends a message.

This command may be used in tandem with the `udp_server` program to illustrate sending messages between AtomVM processes over UDP.

Command line

The `udp_client.avm` file will get created as part of a build. This file may be supplied as an argument to the `AtomVM` command:

```
shell$ ./src/AtomVM ./examples/erlang/udp_client.avm
Opened UDP socket on "0.0.0.0:63665".
Sent <<58,-94,-56,-32,54,45>>
Sent <<58,-94,-56,-32,54,45>>
Sent <<58,-94,-56,-32,54,45>>
...
```

If you are running the `udp_server` program, you should see messages like the following printed to the console:

```
Received UDP packet <<58,-94,-56,-32,54,45>> from "127.0.0.1:63665"
Waiting to receive data...
Received UDP packet <<58,-94,-56,-32,54,45>> from "127.0.0.1:63665"
Waiting to receive data...
Received UDP packet <<58,-94,-56,-32,54,45>> from "127.0.0.1:63665"
Waiting to receive data...
...
```

Note. AtomVM does not currently treat characters outside of the printable ASCII character set as printable characters.

5.1.4 tcp_server

This example program listens on TCP port 44404 and accept connections on that port. Once accepted, it will wait for packets to be sent from the client. Once received, the server will print the packet received to the console, and then echo the packet back to the calling client.

Command line

The `tcp_server.avm` file will get created as part of a build. This file may be supplied as an argument to the `AtomVM` command:

```
shell$ ./src/AtomVM ./examples/erlang/tcp_server.avm
Listening on "0.0.0.0:44404".
Waiting to accept connection...
```

You can send TCP packets to the AtomVM instance using `netcat` (or `nc` on some platforms), in a separate terminal window:

```
shell$ nc localhost 44404
```

This will open a TCP connection to the `tcp_server`, and you should see the following on the console:

```
Accepted connection.  local: "127.0.0.1:44404" peer: "127.0.0.1:56628"
Waiting to receive data...
```

```
Waiting to accept connection...
```

The netcat command will wait for you to enter a line of text, e.g.,

```
testing 1 2 3
```

In the AtomVM terminal window, you see:

```
Received packet [116,101,115,116,105,110,103,32,49,32,50,32,51,10] from
"127.0.0.1:56628".  Echoing back...
Waiting to receive data...
```

> Note. Netcat appends a newline character at the end of the input, so the packet binary does not display as printable text.

You may enter as much data as you like, though by default, the packet size will be limited to 128 bytes.

If you stop the `netcat` command (via ^C), you should

```
Connection closed.
```

printed to the AtomVM console.

Note that you can have multiple, concurrent TCP/IP connections to your AtomVM server.

5.1.5 `tcp_client`

This example program send the packet of data (":?

This command may be used in tandem with the `tcp_server` program to illustrate sending messages between AtomVM processes over TCP.

> Note. You will need to change the `Address` variable in the `tcp_client.erl` program in order to test against an AtomVM server running on a different host or device.

Command line

The `tcp_client.avm` file will get created as part of a build. This file may be supplied as an argument to the `AtomVM` command:

```
shell$ ./src/AtomVM ./examples/erlang/tcp_client.avm
Connected to "127.0.0.1:44404" from "127.0.0.1:56741"
Sent <<58,-94,-56,-32,54,45>> to "127.0.0.1:44404"
Received [58,162,200,224,54,45] from "127.0.0.1:44404"
Sent <<58,-94,-56,-32,54,45>> to "127.0.0.1:44404"
Received [58,162,200,224,54,45] from "127.0.0.1:44404"
Sent <<58,-94,-56,-32,54,45>> to "127.0.0.1:44404"
Received [58,162,200,224,54,45] from "127.0.0.1:44404"
...
```

If you are running the `tcp _server` program, you should see messages like the following printed to the console:

```
Accepted connection.  local: "127.0.0.1:44404" peer: "127.0.0.1:56741"
Waiting to receive data...
Waiting to accept connection...
Received packet [58,162,200,224,54,45] from "127.0.0.1:56741".  Echoing back...
Waiting to receive data...
Received packet [58,162,200,224,54,45] from "127.0.0.1:56741".  Echoing back...
Waiting to receive data...
Received packet [58,162,200,224,54,45] from "127.0.0.1:56741".  Echoing back...
Waiting to receive data...
...
```

> Note. AtomVM does not currently treat characters outside of the printable ASCII character set as printable characters.

You may run multiple concurrent instances of the `tcp_client` against a single `tcp_server` instance.

5.2 ESP32 Examples

AtomVM includes examples that are specifically designed for the ESP32 and other microcontrollers.

5.3 Flashing AtomVM Examples for ESP32

In order to run the ESP32 examples, you will need to flash the example AVM files that are created as part of the build to your device.

In the remainder of this document, we assume the `flash.sh` script, located in the `tools/dev` directory of the AtomVM source tree.

> Note. You must set the `ESP_IDF` environment variable to the root directory of the ESP IDF SDK installation on your development machine.

You can control the serial port and baud rate via the `FLASH_SERIAL_PORT` and `FLASH_BAUD_RATE` environment variables, e.g.,

```
shell$ export FLASH_SERIAL_PORT="/dev/tty.SLAB_USBtoUART"
shell$ export FLASH_BAUD_RATE=921600
```

The default values for these variables, if not set, are `/dev/ttyUSB0` and `115200`, respectively.

> Note. Experiment with baud rates (e.g., 921600). You may find you can shorten the flash-debug-flash cycle with higher rates.

You can montor the console output of these examples by issuing the `monitor` target to `make`, in the `src/platforms/esp32` directory of the AtomVM source tree:

```
shell$ make monitor
MONITOR
--- WARNING: Serial ports accessed as /dev/tty.* will hang gdb if launched.
--- Using /dev/cu.SLAB_USBtoUART instead...
--- idf_monitor on /dev/cu.SLAB_USBtoUART 115200 ---
--- Quit: Ctrl+] | Menu: Ctrl+T | Help: Ctrl+T followed by Ctrl+H ---
ets Jun  8 2016 00:22:57
...
```

5.3.1 `blink`

The `blink` example will turn the blue LED on an ESP32 SoC (pin 2) on and off, once every second.

Flash the example program to your device as follows:

```
shell$ ./tools/dev/flash.sh build/examples/erlang/esp32/blink.avm
...
Hard resetting via RTS pin...
```

You should see the blue LED turn on and off on your ESP32 device.

5.3.2 `esp_random`

This demo program illustrates use of the ESP32 `random`, `restart`, and `reset_reason` functions.

The program will generate a random binary of a random size (at most 127 bytes) every 5 seconds. If a 0-length byte sequence is generated (1:128 probability), the ESP will restart.

Flash the example program to your device as follows:

```
shell$ ./tools/dev/flash.sh build/examples/erlang/esp32/esp_random.avm
...
Hard resetting via RTS pin...
```

You should see something like the following output when monitoring the ESP32 output (truncated for brevity):

```
shell$ make monitor
...
Found AVM partition: size: 1048576, address: 0x110000
Booting file mapped at: 0x3f420000, size: 1048576
Starting: esp_random.beam...
---
Reset reason: esp_rst_poweron
Random bytes: <<71,13,221,24,8,15,...,197,120,152,205>>
Random bytes: <<37,155,124,177,44,141,40,106,...,43,48,62,109,2,78,39,107>>
Random bytes:
<<217,210,239,183,...,78,68,253,146,212,71,17,208,219,126,240,218,34,0,152,80,20,166,194,106>>
Random bytes: <<112,77,123,249,162,...,238,237,128,227,58,29,64,74>>
...
<<"">>
ets Jun  8 2016 00:22:57
...
Found AVM partition: size: 1048576, address: 0x110000
Booting file mapped at: 0x3f420000, size: 1048576
Starting: esp_random.beam...
---
esp_rst_sw
Random bytes:
<<155,174,204,143,232,202,136,...,118,177,77,230,10,21,72,91,92,160,198,115,249,217,206,52,102,3
...
```

5.3.3 esp_nvs

This demo program illustrates the use of ESP32 non-volatile storage (NVS).

The program will store the number of times the device has been rebooted, along with the start time, in NVS.

Flash the example program to your device as follows:

```
shell$ ./tools/dev/flash.sh build/examples/erlang/esp32/esp_nvs.avm
...
Hard resetting via RTS pin...
```

You should see the following output when monitoring the ESP32 output (truncated for brevity):

```
shell$ make monitor
...
Found AVM partition: size: 1048576, address: 0x110000
Booting file mapped at: 0x3f420000, size: 1048576
Starting: esp_nvs.beam...
---
Saving count 0 to NVS...
Reset device to increment.
AtomVM finished with return value = ok
going to sleep forever..
```

Hit the reset button on your device, and the ESP device will reboot, and display something like the following:

```
Found AVM partition: size: 1048576, address: 0x110000
Booting file mapped at: 0x3f420000, size: 1048576
```

```
Starting: esp_nvs.beam...
---
Saving count 1 to NVS...
Reset device to increment.
AtomVM finished with return value = ok
going to sleep forever..
```

5.3.4 reformat_nvs

This demo program will reformat the non-volatile storage (NVS) partition.

Flash the example program to your device as follows:

```
shell$ ./tools/dev/flash.sh build/examples/erlang/esp32/reformat_nvs.avm
...
Hard resetting via RTS pin...
```

You should see the following output when monitoring the ESP32 output (truncated for brevity):

```
shell$ make monitor
...
Found AVM partition: size: 1048576, address: 0x110000
Booting file mapped at: 0x3f420000, size: 1048576
Starting: esp_nvs.beam...
---
Warning: Reformatted NVS partition!
AtomVM finished with return value = ok
going to sleep forever..
```

The NVS partition on your ESP device should be reformatted.

> Note. This program will irrevocably delete all existing key-values stored on the NVS partition. Use with caution.

5.3.5 set_network_config

This demo program can be used to set the WIFI credentials in NVS. Setting WIFI credentials in NVS can greatly simplify the task of running ESP programs that require connectivity to WIFI networks.

> Note. Credentials are stored unencrypted and in plaintext and should not be considered secure. Future versions may use encrypted NVS storage.

Edit the `sta_network_config.erl` program and set the `Ssid` binary with your WIFI AP SSID, and `Psk` binary with the password used to access your WIFI network. Save the file, rebuild, and flash to your device:

```
shell$ make
...
shell$ ./tools/dev/flash.sh build/examples/erlang/esp32/sta_network_config.avm
...
Hard resetting via RTS pin...
```

You should see the following output when monitoring the ESP32 output (truncated for brevity):

```
shell$ make monitor
...
Found AVM partition: size: 1048576, address: 0x110000
Booting file mapped at: 0x3f420000, size: 1048576
Starting: set_network_config.beam...
---
{atomvm,sta_ssid,<<"myssid">>}
{atomvm,sta_psk,<<"xxxxxx">>}
AtomVM finished with return value = ok
going to sleep forever..
```

You may now run programs that use your WIFI network (see below) without needing to enter WIFI credentials.

5.3.6 sta_network

The `sta_network` example will connect to your local WiFi network and obtain and IP address. Once a connection is established, a `connected` message will be displayed. Once an IP address is obtained, the device IP address, netmask, and gateway will be displayed on the console.

> Note. AtomVM currently only supports station mode (STA).
>
> Note. AtomVM currently only supports IPv4 addresses.
>
> Note. If you have not set WIFI credentials in NVS (see above), you will need to edit the `examples/erlang/esp32/sta_network.erl` source file and set the `ssid` and `psk` parameters to match your local WiFi network, and then rebuild the example.

Flash the example program to your device as follows:

```
shell$ ./tools/dev/flash.sh build/examples/erlang/esp32/sta_network.avm
...
Hard resetting via RTS pin...
```

You should see the following output when monitoring the ESP32 output (truncated for brevity):

```
shell$ make monitor
...
Starting: sta_network.beam...
---
I (220) wifi: wifi driver task: 3ffc3b54, prio:23, stack:3584, core=0
I (220) wifi: wifi firmware version: d5da5a5
I (220) wifi: config NVS flash: enabled
I (220) wifi: config nano formatting: disabled
I (230) system_api: Base MAC address is not set, read default base MAC address
from BLK0 of EFUSE
I (240) system_api: Base MAC address is not set, read default base MAC address
from BLK0 of EFUSE
I (270) wifi: Init dynamic tx buffer num: 32
I (270) wifi: Init data frame dynamic rx buffer num: 32
I (270) wifi: Init management frame dynamic rx buffer num: 32
I (270) wifi: Init static rx buffer size: 1600
I (280) wifi: Init static rx buffer num: 10
I (280) wifi: Init dynamic rx buffer num: 32
I (290) NETWORK: starting wifi: SSID: [myssid], password: [XXXXXXXX].
I (360) phy: phy_version: 4000, b6198fa, Sep  3 2018, 15:11:06, 0, 0
I (370) wifi: mode : sta (3c:71:bf:84:d9:08)
I (370) NETWORK: SYSTEM_EVENT_STA_START received.
I (490) wifi: n:1 0, o:1 0, ap:255 255, sta:1 0, prof:1
I (1470) wifi: state: init -> auth (b0)
I (1480) wifi: state: auth -> assoc (0)
I (1490) wifi: state: assoc -> run (10)
I (1500) wifi: connected with myssid, channel 1
I (1500) wifi: pm start, type: 1
I (1500) NETWORK: SYSTEM_EVENT_STA_CONNECTED received.
I (3690) event: sta ip: 192.168.1.236, mask: 255.255.255.0, gw: 192.168.1.1
I (3690) NETWORK: SYSTEM_EVENT_STA_GOT_IP: 192.168.1.236
connected
{{192,168,1,236},{255,255,255,0},{192,168,1,1}}
```

5.3.7 udp_server_blink

The `udp_server_blink` example will connect to your local WiFi network and obtain and IP address. It will then start a UDP server on port 44444. When a UDP message is received, the blue LED on the ESP32 SoC (pin 2) will toggle on and off.

Note. AtomVM currently only supports station mode (STA).

Note. AtomVM currently only supports IPv4 addresses.

Note. You will need to edit the `examples/erlang/esp32/udp_server_blink.erl` source file and set the `ssid` and `psk` parameters to match your local WiFi network, and then rebuild the example.

Flash the example program to your device as follows:

```
shell$ ./tools/dev/flash.sh build/examples/erlang/esp32/udp_server_blink.avm
...
Hard resetting via RTS pin...
```

You should see the following output when monitoring the ESP32 output (truncated for brevity):

```
shell$ make monitor
...
Found AVM partition: size: 1048576, address: 0x110000
Booting file mapped at: 0x3f420000, size: 1048576
Starting: udp_server_blink.beam...
---
I (222) wifi: wifi driver task: 3ffc3de8, prio:23, stack:3584, core=0
I (222) wifi: wifi firmware version: d5da5a5
I (222) wifi: config NVS flash: enabled
I (232) wifi: config nano formatting: disabled
I (232) system_api: Base MAC address is not set, read default base MAC address
from BLK0 of EFUSE
I (242) system_api: Base MAC address is not set, read default base MAC address
from BLK0 of EFUSE
I (312) wifi: Init dynamic tx buffer num: 32
I (312) wifi: Init data frame dynamic rx buffer num: 32
I (312) wifi: Init management frame dynamic rx buffer num: 32
I (312) wifi: Init static rx buffer size: 1600
I (322) wifi: Init static rx buffer num: 10
I (322) wifi: Init dynamic rx buffer num: 32
I (332) NETWORK: starting wifi: SSID: [myssid], password: [XXXXXXXX].
I (442) phy: phy_version: 4000, b6198fa, Sep  3 2018, 15:11:06, 0, 0
I (442) wifi: mode : sta (3c:71:bf:84:d9:08)
I (442) NETWORK: SYSTEM_EVENT_STA_START received.
I (572) wifi: n:1 0, o:1 0, ap:255 255, sta:1 0, prof:1
I (1552) wifi: state: init -> auth (b0)
I (1552) wifi: state: auth -> assoc (0)
I (1562) wifi: state: assoc -> run (10)
I (1582) wifi: connected with myssid, channel 1
I (1582) wifi: pm start, type: 1
I (1582) NETWORK: SYSTEM_EVENT_STA_CONNECTED received.
I (2212) event: sta ip: 192.168.1.236, mask: 255.255.255.0, gw: 192.168.1.1
I (2212) NETWORK: SYSTEM_EVENT_STA_GOT_IP: 192.168.1.236
Acquired IP address: "192.168.1.236" Netmask: "255.255.255.0" Gateway:
"192.168.1.1"
Opened UDP socket on "0.0.0.0:44404".
Waiting to receive data...
```

You can send UDP packets to the AtomVM instance using `netcat` (or `nc` on some platforms), in a separate terminal window:

```
shell$ nc -u 192.168.1.236 44404
```

Every time you enter a line of text, the blue LED on the ESP32 SoC (pin 2) should toggle on and off, and you should see output on the console, such as

```
Received UDP packet <<100,115,102,115,100,10>> from "192.168.1.237:53291"
Waiting to receive data...
```

5.3.8 tcp_server_blink

The `tcp_server_blink` example will connect to your local WiFi network and obtain and IP address. It will then start a TCP server on port 44404. When a TCP message is received, the blue LED on the ESP32 SoC (pin 2) will toggle on and off.

> Note. AtomVM currently only supports station mode (STA).
>
> Note. AtomVM currently only supports IPv4 addresses.
>
> Note. You will need to edit the `examples/erlang/esp32/tcp_server_blink.erl` source file and set the `ssid` and `psk` parameters to match your local WiFi network, and then rebuild the example.

Flash the example program to your device as follows:

```
shell$ ./tools/dev/flash.sh build/examples/erlang/esp32/tcp_server_blink.avm
...
Hard resetting via RTS pin...
```

You should see the following output when monitoring the ESP32 output (truncated for brevity):

```
shell$ make monitor
Found AVM partition: size: 1048576, address: 0x110000
Booting file mapped at: 0x3f420000, size: 1048576
Starting: tcp_server_blink.beam...
---
start
I (296) wifi: wifi driver task: 3ffc650c, prio:23, stack:3584, core=0
I (296) wifi: wifi firmware version: 9415913
I (296) wifi: config NVS flash: enabled
I (296) wifi: config nano formatting: disabled
I (296) system_api: Base MAC address is not set, read default base MAC address
from BLK0 of EFUSE
I (306) system_api: Base MAC address is not set, read default base MAC address
from BLK0 of EFUSE
I (346) wifi: Init dynamic tx buffer num: 32
I (346) wifi: Init data frame dynamic rx buffer num: 32
I (346) wifi: Init management frame dynamic rx buffer num: 32
I (346) wifi: Init static rx buffer size: 1600
I (356) wifi: Init static rx buffer num: 10
I (356) wifi: Init dynamic rx buffer num: 32
I (366) NETWORK: starting wifi: SSID: [myssid], password: [XXXXXXXX].
I (446) phy: phy_version: 4008, c9ae59f, Jan 25 2019, 16:54:06, 0, 0
I (446) wifi: mode : sta (3c:71:bf:84:d9:08)
I (446) NETWORK: SYSTEM_EVENT_STA_START received.
I (1176) wifi: n:6 0, o:1 0, ap:255 255, sta:6 0, prof:1
I (2156) wifi: state: init -> auth (b0)
I (2166) wifi: state: auth -> assoc (0)
I (2166) wifi: state: assoc -> run (10)
I (2196) wifi: connected with myssid, channel 6
I (2196) wifi: pm start, type: 1
I (2196) NETWORK: SYSTEM_EVENT_STA_CONNECTED received.
I (2746) event: sta ip: 192.168.1.236, mask: 255.255.255.0, gw: 192.168.1.1
I (2746) NETWORK: SYSTEM_EVENT_STA_GOT_IP: 192.168.1.236
Acquired IP address: "192.168.1.236" Netmask: "255.255.255.0" Gateway:
"192.168.1.1"
Listening on "0.0.0.0:44404".
Waiting to accept connection...
```

You can send TCP packets to the AtomVM instance using `netcat` (or `nc` on some platforms), in a separate terminal window, e.g.,

```
shell$ nc 192.168.1.236 44404
```

On the ESP32 console, you should see:

```
Accepted connection.  local: "192.168.1.236:44404" peer: "192.168.1.237:55275"
Waiting to receive data...
Waiting to accept connection...
```

Every time you enter a line of text, the blue LED on the ESP32 SoC (pin 2) should toggle on and off, and the data you entered should get echoed back to the `netcat` console.

On the ESP32 console, you should see:

```
Received packet [115,100,102,115,100,102,10] from "192.168.1.237:55275".  Echoing
back...
Waiting to receive data...
```

every time a packet is sent to the server.

Chapter 6

Network Programming Guide

One of the exciting features of the ESP32 is its support for WiFi networking, allowing ESP32 micro-controllers to communicate with the outside world over common IP networking protocols, such as TCP or IDP. The ESP32 and ISF SDK supports configuring an ESP32 in station mode (STA), whereby the device connects to an existing access point, as well as "softAP" mode (AP), whereby it functions as an access point, to which other stations can connect. The ESP32 also supports a combined STA+softAP mode, which allows the device to function in both STA and softAP mode simultaneously.

AtomVM provides an Erlang API interface for interacting with the WiFi networking layer on ESP32 devices, providing support for configuring your ESP32 device in STA mode, AP mode, or a combined STA+AP mode, allowing Erlang/Elixir applications to send and receive data from other devices on a network. This interface is encapsulated in the `network` module, which implements a simple interface for connecting to existing WiFi networks or for functioning as a WiFi access point.

Once the network has been set up (in STA or AP mode), AtomVM can use various socket interfaces to interact with the socket layer to create a client or server application. For example, AtomVM supports the `gen_udp` and `gen_tcp` APIs, while AtomVM extensions may support HTTP, MQTT, and other protocols built over low-level networking interfaces.

The AtomVM networking API leverages callback functions, allowing applications to be responsive to changes in the underlying network, which can frequently occur in embedded applications, where devices can easily lose and then regain network connectivity. In such cases, it is important for applications to be resilient to changes in network availability, by closing or re-opening socket connections in response to disconnections and re-connections in the underlying network.

This document describes the basic design of the AtomVM network interfaces, and how to interact programmatically with it.

6.1 Station (STA) mode

In STA mode, the ESP32 connects to an existing WiFi network.

In this case, the input configuration should be a properties list containing a tuple of the form `{sta, <sta-properties>}`, where `<sta-properties>` is a property list containing configuration properties for the device in station mode.

The `<sta-properties>` property list should contain the following entries:

- `{ssid, string() | binary()}` The SSID to which the device should connect.
- `{psk, string() | binary()}` The password required to authenticate to the network, if required.

  Note that the station mode SSID and password *may* be stored in non-volatile storage, in which case these parameters may be skipped. See the "NVS Credentials" section below, for more information about using non-volatile storage to store credentials that persist across device

reboots.

The `network:start/1` will immediately return `{ok, Pid}`, where `Pid` is the process ID of the network server instance, if the network was properly initialized, or `{error, Reason}`, if there was an error in configuration. However, the application may want to wait for the device to connect to the target network and obtain an IP address, for example, before starting clients or services that require network access.

Applications can specify callback functions, which get triggered as events emerge from the network layer, including connection to and disconnection from the target network, as well as IP address acquisition.

Callback functions can be specified by the following configuration parameters:

- `{connected, fun(() -> term())}` A callback function which will be called when the device connects to the target network.
- `{disconnected, fun(() -> term())}` A callback function which will be called when the device disconnects from the target network.
- `{got_ip, fun((ip_info()) -> term())}` A callback function which will be called when the device obtains an IP address. In this case, the IPv4 IP address, net mask, and gateway are provided as a parameter to the callback function.

Note. IPv6 addresses are not yet supported in AtomVM.

Callback functions are optional, but are highly recommended for building robust WiFi applications. The return value from callback functions is ignored, and AtomVM provides no guarantees about the execution context (i.e., BEAM process) in which these functions are invoked.

In addition, the following optional parameters can be specified to configure the AP network:

- `{dhcp_hostname, string()|binary()}` The DHCP hostname as which the device should register (`<<"atomvm-<hexmac>">>`, where `<hexmac>` is the hexadecimal representation of the factory-assigned MAC address of the device).

The following example illustrates initialization of the WiFi network in STA mode. The example program will configure the network to connect to a specified network. Events that occur during the lifecycle of the network will trigger invocations of the specified callback functions.

```
%% erlang
Config = [
    {sta, [
        {ssid, <<"myssid">>},
        {psk,  <<"mypsk">>},
        {connected, fun connected/0,
        {got_ip, fun got_ip/1},
        {disconnected, fun disconnected/0}
        {dhcp_hostname, <<"myesp32">>}
    ]}
],
{ok, Pid} = network:start(Config),
...
```

The following callback functions will be called when the corresponding events occur during the lifetime of the network connection.

```
%% erlang
connected()) ->
    io:format("Connected to AP.~n").

gotIp(IpInfo) ->
    io:format("Got IP: ~p~n", [IpInfo]).

disconnected() ->
```

```
    io:format("Disconnected from AP.~n").
```

In a typical application, the network should be configured and an IP address should be acquired first, before starting clients or services that have a dependency on the network.

6.1.1 Convenience Functions

The `network` module supports the `network:wait_for_sta/1,2` convenience functions for applications that do not need robust connection management. These functions are synchronous and will wait until the device is connected to the specified AP. Supply the properties list specified in the `{sta, [...]}` component of the above configuration, in addition to an optional timeout (in milliseconds).

For example:

```
%% erlang
Config = [
    {ssid, <<"myssid">>},
    {psk,  <<"mypsk">>},
    {dhcp_hostname, <<"mydevice">>}
],
case network:wait_for_sta(Config, 15000) of
    {ok, {Address, _Netmask, _Gateway}} ->
        io:format("Acquired IP address: ~p~n", [Address]);
    {error, Reason} ->
        io:format("Network initialization failed: ~p~n", [Reason])
end
```

6.2 AP mode

In AP mode, the ESP32 starts a WiFi network to which other devices (laptops, mobile devices, other ESP32 devices, etc) can connect. The ESP32 will create an IPv4 network, and will assign itself the address `192.168.4.1`. Devices that attach to the ESP32 in AP mode will be assigned sequential addresses in the `192.168.4.0/24` range, e.g., `192.168.4.2`, `192.168.4.3`, etc.

To initialize the ESP32 device in AP mode, the input configuration should be a properties list containing a tuple of the form `{ap, <ap-properties>}`, where `<ap-properties>` is a property list containing configuration properties for the device in AP mode.

The `<ap-properties>` property list may contain the following entries:

- `{ssid, string() | binary()}` The SSID to which the device should connect.
- `{psk, string() | binary()}` The password required to authenticate to the network, if required. Note that this password must be a minimum of 8 characters.

If the SSID is omitted in configuration, the SSID name `atomvm-<hexmac>` will be created, where `<hexmac>` is the hexadecimal representation of the factory-assigned MAC address of the device. This name should be sufficiently unique to disambiguate it from other reachable ESP32 devices, but it may also be difficult to read or remember.

If the password is omitted, then an *open network* will be created, and a warning will be printed to the console. Otherwise, the AP network will be started using WPA+WPA2 authentication.

> Note that the station mode SSID and password *may* be stored in non-volatile storage, in which case these parameters may be skipped. See the "NVS Credentials" section below, for more information about using non-volatile storage to store credentials that persist across device reboots.

The `network:start/1` will immediately return `{ok, Pid}`, where `Pid` is the process id of the network server, if the network was properly initialized, or `{error, Reason}`, if there was an error in configuration. However, the application may want to wait for the device to to be ready to

accept connections from other devices, or to be notified when other devices connect to this AP.

Applications can specify callback functions, which get triggered as events emerge from the network layer, including when a station connects or disconnects from the AP, as well as when a station is assigned an IP address.

Callback functions can be specified by the following configuration parameters:

- `{ap_started, fun(() -> term())}` A callback function which will be called when the AP endpoint has started and is ready to be connected to.
- `{sta_connected, fun((Mac::binary()) -> term())}` A callback function which will be called when a device connects to the AP. The MAC address of the connected station, as a 6-byte binary, is passed to the callback function.
- `{sta_disconnected, fun((Mac::binary()) -> term())}` A callback function which will be called when a device disconnects from the AP. The MAC address of the disconnected station, as a 6-byte binary, is passed to the callback function.
- `{sta_ip_assigned, fun((ipv4_address()) -> term())}` A callback function which will be called when the AP assigns an IP address to a station. The assigned IP address is passed to the callback function.

Note. IPv6 addresses are not yet supported in AtomVM.

Callback functions are completely optional, but are highly recommended for building robust WiFi applications. The return value from callback functions is ignored, and AtomVM provides no guarantees about the execution context (i.e., BEAM process) in which these functions are invoked.

In addition, the following optional parameters can be specified to configure the AP network:

- `{ssid_hidden, boolean()}` Whether the AP network should be not be broadcast (false, by default)
- `{max_connections, non_neg_integer()}` The maximum number of devices that can connect to this network (by default, 4)

The following example illustrates initialization of the WiFi network in AP mode. The example program will configure the network to connect to start a WiFi network with the name `myssid` and password `mypsk`. Events that occur during the lifecycle of the network will trigger invocations of the specified callback functions.

```
%% erlang
Config = [
    {ap, [
        {ssid, <<"myssid">>},
        {psk,  <<"mypsk">>},
        {ap_started, fun ap_started/0},
        {sta_connected, fun sta_connected/1},
        {sta_ip_assigned, fun sta_ip_assigned/1},
        {sta_disconnected, fun sta_disconnected/1},
    ]}
],
{ok, Pid} = network:start(Config),
...
```

The following callback functions will be called when the corresponding events occur during the lifetime of the network connection.

```
%% erlang
ap_started() ->
    io:format("AP started.~n").

sta_connected(Mac) ->
    io:format("STA connected with mac ~p~n", [Mac]).
```

```
sta_disconnected(Mac) ->
    io:format("STA disconnected with mac ~p~n", [Mac]).

sta_ip_assigned(Address) ->
    io:format("STA assigned address ~p~n", [Address]).
```

In a typical application, the network should be configured and the application should wait for the AP to report that it has started, before starting clients or services that have a dependency on the network.

6.2.1 Convenience Functions

The `network` module supports the `network:wait_for_ap/1,2` convenience functions for applications that do not need robust connection management. These functions are synchronous and will wait until the device is successfully starts an AP. Supply the properties list specified in the `{ap, [...]}` component of the above configuration, in addition to an optional timeout (in milliseconds).

For example:

```
%% erlang
Config = [
    {psk,  <<"mypsk">>}
],
case network:wait_for_ap(Config, 15000) of
    ok ->
        io:format("AP network started at 192.168.4.1~n");
    {error, Reason} ->
        io:format("Network initialization failed: ~p~n", [Reason])
end
```

6.3 STA+AP mode

The `network` module can be started in both STA and AP mode. In this case, the ESP32 device will both connect to an access point in its STA mode, and will simultaneously serve as an access point in its role in AP mode.

In order to enable both STA and AP mode, simply provide valid configuration for both modes in the configuration structure supplied to the `network:start/1` function.

6.4 SNTP Support

You may configure the networking layer to automatically synchronize time on the ESP32 with an NTP server accessible on the network.

To synchronize time with an NTP server, add a property list with the tag `sntp` at the top level configuration passed into the `network:start/1` function. Specify the NTP hostname or IP address with which your device should sync using the `host` property tag. The `host` value can be a string or binary.

You can also specify a callback function that will get called when the clock is synchronized with the SNTP server via the `synchronized` property tag. This function takes a tuple with the updated time in seconds and microseconds.

For example:

```
%% erlang
{sntp, [
    {host, <<"pool.ntp.org">>},
    {synchronized, fun sntp_synchronized/1}
]}
```

where the `sntp_synchronized/1` function is defined as:

```
%% erlang
sntp_synchronized({TVSec, TVUsec}) ->
    io:format("Synchronized time with SNTP server. TVSec=~p TVUsec=~p~n", [TVSec,
TVUsec]).
```

Note. The device must be in STA mode and connected to an access point in order to use an SNTP server on your network or on the internet.

6.5 NVS Credentials

It can become tiresome to enter an SSID and password for every application, and in general it is bad security practice to hard-wire WiFi credentials in your application source code.

You may instead store an STA or AP SSID and PSK in non-volatile storage (NVS) on and ESP32 device under the `atomvm` namespace. The following entries may be specified in non-volatile storage:

namespace	mode	key	type	value
`atomvm`	STA	`sta_ssid`	`binary()`	Station ID
`atomvm`	STA	`sta_psk`	`binary()`	Station password (if applicable)
`atomvm`	AP	`ap_ssid`	`binary()`	Access Point ID
`atomvm`	AP	`ap_psk`	`binary()`	Access Point password (if applicable)

If set in NVS storage, you may remove the corresponding `ssid` and `psk` parameters from the configuration used to initialize the network, and the SSID and PSK configured in NVS will be used, instead. An SSID or PSK defined explicitly in configuration will override any values in NVS.

You can set these credentials once, as follows:

```
esp:nvs_put_binary(atomvm, sta_ssid, <<"myssid">>).
esp:nvs_put_binary(atomvm, sta_psk, <<"mypsk">>).
```

or

```
esp:nvs_put_binary(atomvm, ap_ssid, <<"myssid">>).
esp:nvs_put_binary(atomvm, ap_psk, <<"mypsk">>).
```

With these settings, you can run ESP programs that initialize the network without configuring your SSID and PSK explicitly in source code.

Note. Credentials are stored un-encrypted and in plaintext and should not be considered secure. Future versions may use encrypted NVS storage.

6.6 Stopping the Network

To stop the network and free any resources in use, issue the `stop/0` function:

```
network:stop().
```

Note. This function is currently not well tested.

Chapter 7

Build Instructions

This guide is intended for anyone interested in building the AtomVM virtual machine from source code. You may be interested in building the AtomVM source code if you want to provide bug fixes or enhancements to the VM, or if you want to simply learn more about the platform. In addition, some "downstream" drivers for specific devices may need to be built specifically for the target platform (e.g., ESP32), in which case building the VM from source code is required.

> Note. Many applications do not require building the AtomVM runtime from source code. Instead, you can download pre-built VM images for platforms such as ESP32, and use Erlang and Elixir tooling to build and deploy your applications.

The AtomVM virtual machine itself, including the runtime code execution engine, as well as built-in functions and Nifs is implemented in C. The core standard and AtomVM libraries are implemented in Erlang and Elixir.

The native C parts of AtomVM compile to machine code on MacOS, Linux, and FreeBSD platforms. The C code also compiles to run on the ESP32 and STM32 platforms. Typically, binaries for these platforms are created on a UNIX-like environment (MacOS or Linux, currently) using tool-chains provided by device vendors to cross-compile and target specific device architectures.

The Erlang and Elixir parts are compiled to BEAM byte-code using the Erlang (`erlc`) and Elixir compilers. For information about specific versions of required software, see the Release Notes.

This guide provides information about how to build AtomVM for the various supported platforms (Generic UNIX, ESP32, and STM32).

> Note. In order to build AtomVM AVM files for ESP32 and STM32 platforms, you will also need to build AtomVM for the Generic UNIX platform of your choice.

7.1 Downloading AtomVM

The AtomVM source code is available by cloning the AtomVM github repository:

```
shell$ git clone https://github.com/atomvm/AtomVM
```

> Note. Downloading the AtomVM github repository requires the installation of the `git` program. Consult your local OS documentation for installation of the `git` package.

7.2 Source code organization

Source code is organized as follows:

- `src` Contains the core AtomVM virtual machine source code;
- `lib` Contains the Erlang and Elixir core library source code;
- `tools` Contains AtomVM tooling, including the `PackBEAM` executable, as well as build support

tooling;

- `examples` Contains sample programs for demonstration purposes;
- `tests` Contains test code run as part of test qualification;
- `doc` Contains documentation source code and content.

The `src` directory is broken up into the core platform-independent AtomVM library (`libAtomVM`), and platform-dependent code for each of the supported platforms (Generic UNIX, ESP32, and STM32).

For information about porting to new platforms, see Porting to new platforms, below.

7.3 Building for Generic UNIX

The following instructions apply to unix-like environments, including Linux, FreeBSD, and MacOS.

> Note. The Generic UNIX is useful for running and testing simple AtomVM programs. Not all of the AtomVM APIs, specifically, APIs that are dependent on various device integration, are supported on this platform.

7.3.1 Build Requirements

The following software is required in order to build AtomVM in generic UNIX systems:

- `gcc` or `llvm` tool chains
- `cmake`
- `make`
- `gperf`
- `zlib`
- Erlang/OTP compiler (`erlc`)
- Elixir compiler

Consult Release Notes for currently supported versions of required software.

Consult your local OS documentation for instructions about how to install these components.

7.3.2 Build Instructions

The AtomVM build for generic UNIX systems makes use of the `cmake` tool for generating `make` files from the top level AtomVM directory. With CMake, you generally create a separate directory for all output files (make files, generated object files, linked binaries, etc). A common pattern is to create a local `build` directory, and then point `cmake` to the parent directory for the root of the source tree:

```
shell$ mkdir build
shell$ cd build
shell$ cmake ..
...
```

This command will create all of the required make files for creating the AtomVM binary, tooling, and core libraries. You can create all of these object using the `make` command:

```
shell$ make -j 8
...
```

> Note. You may specify `-j <n>`, where `<n>` is the number of CPUs you would like to assign to run the build in parallel.

Upon completion, the `AtomVM` executable can be found in the `build/src` directory.

The AtomVM core Erlang library can be found in the generated `libs/atomvmlib.avm` AVM file.

Use the `install` target to install the `atomvm` command and associated binary files. On most UNIX systems, these artifacts will be installed in the `/usr/local` directory tree.

Note. On some systems, you may need to run this target with `root` or `sudo` permissions.

```
shell$ sudo make install
```

Once installed, you can use the `atomvm` command to execute an AtomVM application. E.g.,

```
shell$ atomvm /path/to/myapp.avm
```

For users doing incremental development on the AtomVM virtual machine, you may want to run the `AtomVM` binary directly instead of installing the VM on your machine. If you do, you will typically need to also specify the path to the AtomVM core Erlang library. For example,

```
shell$ cd build
shell$ ./src/AtomVM /path/to/myapp.avm ./libs/atomvmlib.avm
```

Special Note for MacOS users

You may build an Apple Xcode project, for developing, testing, and debugging in the Xcode IDE, by specifying the Xcode generator. For example, from the top level AtomVM directory:

```
shell$ mkdir xcode
shell$ cmake -G Xcode ..
...
shell$ open AtomVM.xcodeproj
```

The above commands will build and open an AtomVM project in the Xcode IDE.

7.3.3 Running tests

There are currently two sets of suites of tests for AtomVM:

- Erlang tests (`erlang_tests`) A set of unit tests for basic Erlang functionality, exercising support BEAM opcodes, built-in functions (Bifs) and native functions (Nifs).
- Library tests, exercising functionality in the core Erlang and Elixir libraries.

To run the Erlang tests, run the `test-erlang` executable in the `tests` directory:

```
shell$ ./tests/test-erlang
```

This will run a suite of several score unit tests. Check the status of the executable after running the tests. A non-zero return value indicates a test failure.

To run the Library tests, run the corresponding AVM module in the `tests/libs` directory using the `AtomVM` executable. For example:

```
shell$ ./src/AtomVM ./tests/libs/estdlib/test_estdlib.avm
```

This will run a suite of several unit tests for the specified library. Check the status of the executable after running the tests. A non-zero return value indicates a test failure.

Tests for the following libraries are supported:

- `estdlib`
- `eavmlib`
- `alisp`

7.4 Building for ESP32

Building AtomVM for ESP32 must be done on either a Linux or MacOS build machine.

In order to build a complete AtomVM image for ESP32, you will also need to build AtomVM for the Generic UNIX platform (typically, the same build machine you are suing to build AtomVM for ESP32).

7.4.1 Build Requirements

The following software is required in order to build AtomVM for the ESP32 platform:

- Espressif Xtensa tool chains
- Espressif IDF SDK (consult Release Notes for currently supported versions)
- `cmake`

Instructions for downloading and installing the Espressif IDF SDK and tool chains are outside of the scope of this document. Please consult the IDF SDKGetting Started guide for more information.

7.4.2 Build Instructions

To activate the ESP-IDF build environment change directories to the tree root of your local ESP-IDF:

```
shell$ cd <ESP-IDF-ROOT-DIR>
shell$ . ./export.sh
```

> Note: If you followed Espressif's installation guide the ESP-IDF directory is `${HOME}/esp/esp-idf`

Change directories to the `src/platforms/esp32` directory under the AtomVM source tree root:

```
shell$ cd <atomvm-source-tree-root>
shell$ cd src/platforms/esp32
```

Start by updating the default build configuration of local `sdkconfig` file via the `idf.py reconfigure` command:

```
shell$ idf.py set-target esp32
shell$ idf.py reconfigure
```

> Note. For those familiar with esp-idf the build can be customized using `menuconfig` instead of `reconfigure`
>
> shell$ idf.py menuconfig
>
> This command will bring up a curses dialog box where you can make adjustments such as not including AtomVM components that are not desired in a particular build. You can also change the behavior of a crash in the VM to print the error and reboot, or halt after the error is printed. Extreme caution should be used when changing any non AtomVM settings. You can quit the program by typing `Q`. Save the changes, and the program will exit.

You can now build AtomVM using the build command:

```
shell$ idf.py build
...
```

This command, once completed, will create the Espressif bootloader, partition table, and AtomVM binary. The last line of the output should read something like the following:

```
To flash all build output, run 'idf.py flash' or:
python /path/to/esp-idf-sdk/components/esptool_py/esptool/esptool.py --chip esp32
```

```
--port /dev/ttyUSB0 --baud 115200 --before default_reset --after hard_reset
write_flash
-z --flash_mode dio --flash_freq 40m --flash_size detect
0x1000
/path-to-atomvm-source-tree/Atomvm/src/platforms/esp32/build/bootloader/bootloader.bin
0x10000
/path-to-atomvm-source-tree/Atomvm/src/platforms/esp32/build/atomvm-esp32.bin
0x8000
/path-to-atomvm-source-tree/Atomvm/src/platforms/esp32/build/partition_table/partition-table.bin
```

At this point, you can run `idf.py flash` to upload the 3 binaries up to your ESP32 device, and in some development scenarios, this is a preferable shortcut.

However, first, we will build a single binary image file containing all of the above 3 binaries, as well as the AtomVM core libraries. See Section 7.4.6, below. But first, it is helpful to understand a bit about how the AtomVM partitioning scheme works, on the ESP32.

7.4.3 Running tests

Tests for ESP32 are run on the desktop (or CI) using qemu.

Install or compile Espressif's fork of qemu. Espressif provides binaries for Linux amd64 and it's also bundled in espressif/idf:5.1 docker image.

Also install Espressif pytest's extensions for embedded testing with:

```
shell$ cd <ESP-IDF-ROOT-DIR>
shell$ . ./export.sh
shell$ pip install pytest==7.0.1 \
        pytest-embedded==1.2.5 \
        pytest-embedded-serial-esp==1.2.5 \
        pytest-embedded-idf==1.2.5 \
        pytest-embedded-qemu==1.2.5
...
```

Change directory to the `src/platforms/esp32/test` directory under the AtomVM source tree root:

```
shell$ cd <atomvm-source-tree-root>
shell$ cd src/platforms/esp32/test
```

Build tests using the build command:

```
shell$ idf.py build
...
```

Note. This eventually compiles host AtomVM to be able to build and pack erlang test modules.

Run tests using the command:

```
shell$ pytest --embedded-services=idf,qemu -s
...
```

ESP32 tests are erlang modules located in `src/platforms/esp32/test/main/test_erl_sources/` and executed from `src/platforms/esp32/test/main/test_main.c`.

7.4.4 Flash Layout

The AtomVM Flash memory is partitioned to include areas for the above binary artifacts created from the build, as well areas for runtime information used by the ESP32 and compiled Erlang/Elixir code.

The flash layout is roughly as follows (not to scale):

```
+-----------------+  ------------- 0x0000
```

```
|    secure       |
|     boot        | 4KB
|                 |
+-----------------+  ------------- 0x1000
|                 |             ^
|   boot loader   | 28KB        |
|                 |             |
+-----------------+             |
| partition table | 3KB         |
+-----------------+             |
|                 |             |
|       NVS       | 24KB        |
|                 |             |
+-----------------+             |
|     PHY_INIT    | 4KB         |
+-----------------+             | AtomVM
|                 |             | binary
|                 |             | image
|                 |             |
|     AtomVM      |             |
|     Virtual     | 1.75MB      |
|     Machine     |             |
|                 |             |
|                 |             |
+-----------------+             |
|     lib.avm     | 256KB       v
+-----------------+  ------------- 0x210000
|                 |             ^
|                 |             |
|     main.avm    | 1MB+        | Erlang/Elixir
|                 |             | Application
|                 |             |
|                 |             v
+-----------------+  ------------- end
```

The following table summarizes the partitions created on the ESP32 when deploying AtomVM:

Partition	Offset	Length	Description
Secure Boot	0x00	4kB	Initialization vectors and other data needed for ESP32 secure boot.
Bootloader	0x1000	28kB	The ESP32 bootloader, as built from the IDF-SDK. AtomVM does not define its own bootloader.
Partition Table	0x8000	3kB	The AtomVM-defined partition table.
NVS	0x9000	24kB	Space for non-volatile storage.
PHY_INIT	0xF000	4kB	Initialization data for physical layer radio signal data.
AtomVM virtual machine	0x10000	1.75mB	The AtomVM virtual machine (compiled from C code).
lib.avm	0x1D0000	256k	The AtomVM BEAM library, compiled from Erlang and Elixir files in the AtomVM source tree.
main.avm	0x210000	1mB	The user application. This is where users flash their compiled Erlang/Elixir code

7.4.5 The `lib.avm` and `main.avm` partitions

The `lib.avm` and `main.avm` partitions are intended to store Erlang/Elixir libraries (compiled down to BEAM files, and assembled as AVM files).

The `lib.avm` partition is intended for core Erlang/Elixir libraries that are built as part of the AtomVM build. The release image of AtomVM (see below) includes both the AtomVM virtual machine and the `lib.avm` partition, which includes the BEAM files from the `estdlib` and `eavmlib` libraries.

In contrast, the `main.avm` partition is intended for user applications. Currently, the `main.avm` partition starts at address `0x210000`, and it is to that location to which application developers should flash their application AVM files.

The AtomVM search path for BEAM modules starts in the `main.avm` partition and falls back to `lib.avm`. Users should not have a need to override any functionality in the `lib.avm` partition, but if necessary, a BEAM module of the same name in the `main.avm` partition will be loaded instead of the version in the `lib.avm` partition.

> Note. The location of the `main.avm` partition may change over time, depending on the relative sizes of the AtomVM binary and `lib.avm` partitions.

7.4.6 Building a Release Image

The `<atomvm-source-tree-root>/tools/release/esp32` directory contains the `mkimage.sh` script that can be used to create a single AtomVM image file, which can be distributed as a release, allowing application developers to develop AtomVM applications without having to build AtomVM from scratch.

> Note. Before running the `mkimage.sh` script, you must have a complete build of both the esp32 project, as well as a full build of the core Erlang libraries in the `libs` directory. The script configuration defaults to assuming that the core Erlang libraries will be written to the `build/libs` directory in the AtomVM source tree. You should pass the `--build_dir <path>` option to the `mkimage.sh` script, with `<path>` pointing to your AtomVM build directory, if you target a different build directory when running CMake.

Running this script will generate a single `atomvm-<sha>.img` file in the `build` directory of the esp32 source tree, where `<sha>` is the git hash of the current checkout. This image contains the ESP32 bootloader, AtomVM executable, and the `eavmlib` and `estdlib` Erlang libraries in one file, which can then be flashed to address `0x1000`.

The `mkimage.sh` script is run from the src/platform/esp32 directory as follows:

```
shell$ ./build/mkimage.sh
Writing output to
/home/frege/AtomVM/src/platforms/esp32/build/atomvm-esp32-0.6.0-dev+git.602e6bc.img
=============================================
Wrote bootloader at offset 0x1000 (4096)
Wrote partition-table at offset 0x8000 (32768)
Wrote AtomVM Virtual Machine at offset 0x10000 (65536)
Wrote AtomVM Core BEAM Library at offset 0x110000 (1114112)
```

Users can then use the `esptool.py` directly to flash the entire image to the ESP32 device, and then flash their applications to the `main.app` partition at address `0x210000`,

But first, it is a good idea to erase the flash, e.g.,

```
shell$ esptool.py --chip esp32 --port /dev/ttyUSB0 erase_flash
esptool.py v2.1
Connecting........_
Chip is ESP32D0WDQ6 (revision 1)
Uploading stub...
Running stub...
Stub running...
Erasing flash (this may take a while)...
Chip erase completed successfully in 5.4s
Hard resetting...
```

Flashing Release Images

After preparing a release image you can use the `flashimage.sh`, which is generated with each build that will flash the full image using the correct flash offset for the chip the build was configured for.

```
shell$ ./build/flashimage.sh
```

To perform this action manually you can use the `./build/flash.sh` tool (or `esptool.py` directly, if you prefer):

```
shell$ FLASH_OFFSET=0x1000 ./build/flash.sh
./build/atomvm-esp32-0.6.0-dev+git.602e6bc.img
esptool.py v2.8-dev
Serial port /dev/tty.SLAB_USBtoUART
Connecting........_
Chip is ESP32D0WDQ6 (revision 1)
Features: WiFi, BT, Dual Core, Coding Scheme None
Crystal is 40MHz
MAC: 30:ae:a4:1a:37:d8
Uploading stub...
Running stub...
Stub running...
Changing baud rate to 921600
Changed.
Configuring flash size...
Auto-detected Flash size: 4MB
Wrote 1163264 bytes at 0x00001000 in 15.4 seconds (603.1 kbit/s)...
Hash of data verified.
Leaving...
Hard resetting via RTS pin...
```

Note. Flashing the full AtomVM image will delete all entries in non-volatile storage. Only flash the full image if you have a way to recover and re-write any such data, if you need to retain it.

7.4.7 Flashing Applications

Applications can be flashed using the `flash.sh` script in the esp32 build directory:

```
shell$ ./build/flash.sh ../../../build/examples/erlang/esp32/blink.avm
%%
%% Flashing examples/erlang/esp32/blink.avm (size=4k)
%%
esptool.py v2.8-dev
Serial port /dev/tty.SLAB_USBtoUART
Connecting........_
Chip is ESP32D0WDQ6 (revision 1)
Features: WiFi, BT, Dual Core, Coding Scheme None
Crystal is 40MHz
MAC: 30:ae:a4:1a:37:d8
Uploading stub...
Running stub...
Stub running...
Changing baud rate to 921600
Changed.
Configuring flash size...
Auto-detected Flash size: 4MB
Wrote 16384 bytes at 0x00210000 in 0.2 seconds (611.7 kbit/s)...
Hash of data verified.
Leaving...
Hard resetting via RTS pin...
```

Note. Since the Erlang core libraries are flashed to the ESP32 device, it is not necessary to include core libraries in your application AVM files. Users may be interested in using downstream development tools, such as the Elixir ExAtomVM Mix task, or the Erlang AtomVM Rebar3 Plugin for doing day-to-day development of applications for the AtomVM platform.

Flashing the core libraries

If you are doing development work on the core Erlang/Elixir libraries and wish to test changes that do not involve the `C` code in the core VM you may flash `atomvmlib.avm` to the avm.lib partition (offset 0x1D0000) by using the `flash.sh` script in the esp32 build directory as follows:

```
shell$ build/flash.sh -l ../../../build/libs/atomvmlib.avm
%%
%% Flashing ../../../build/libs/atomvmlib.avm (size=116k)
%%
esptool.py v4.5.1
Serial port /dev/ttyUSB0
Connecting.....
Detecting chip type... Unsupported detection protocol, switching and trying
again...
Connecting.....
Detecting chip type... ESP32
Chip is ESP32-D0WD (revision v1.0)
Features: WiFi, BT, Dual Core, 240MHz, VRef calibration in efuse, Coding Scheme
None
Crystal is 40MHz
MAC: 1a:57:c5:7f:ac:5b
Uploading stub...
Running stub...
Stub running...
Changing baud rate to 921600
Changed.
Configuring flash size...
Auto-detected Flash size: 8MB
Flash will be erased from 0x001d0000 to 0x001ecfff...
Wrote 131072 bytes at 0x001d0000 in 1.8 seconds (582.1 kbit/s)...
Hash of data verified.

Leaving...
Hard resetting via RTS pin...
```

7.4.8 Adding custom Nifs, Ports, and third-party components

While AtomVM is a functional implementation of the Erlang virtual machine, it is nonetheless designed to allow developers to extend the VM to support additional integrations with peripherals and protocols that are not otherwise supported in the core virtual machine.

AtomVM supports extensions to the VM via the implementation of custom native functions (Nifs) and processes (AtomVM Ports), allowing users to extend the VM for additional functionality, and you can add your own custom Nifs, ports, and additional third-party components to your ESP32 build by adding them to the `components` directory, and the ESP32 build will compile them automatically.

> For more information about building components for the IDF SDK, consult the IDF SDK Build System documentation.

The instructions for adding custom Nifs and ports differ in slight detail, but are otherwise quite similar. In general, they involve:

1. Adding the custom Nif or Port to the `components` directory of the AtomVM source tree;
2. Adding the component to the corresponding `main/component_nifs.txt` or `main/component_ports.txt` files;
3. Building the AtomVM binary.

> Note. The Espressif SDK and tool chains do not, unfortunately, support dynamic loading of shared libraries and dynamic symbol lookup. In fact, dynamic libraries are not supported at all on the ESP32 using the IDF SDK; instead, any code that is needed at runtime must be statically linked into the application.

Custom Nifs and Ports are available through third parties. Follow the instructions provided with these custom components for detailed instruction for how to add the Nif or Port to your build.

More detailed instructions follow, below, for implementing your own Nif or Port.

Adding a custom AtomVM Nif

To add support for a new peripheral or protocol using custom AtomVM Nif, you need to do the following:

- Choose a name for your nif (e.g, "my_nif"). Call this `<moniker>`.
- In your source code, implement the following two functions:
  - `void <moniker>_nif_init(GlobalContext *global);`
    - This function will be called once, when the application is started.
  - `const struct Nif *<moniker>_nif_get_nif(const char *nifname);`
    - This function will be called to locate the Nif during a function call.

Example:

```
void my_nif_init(GlobalContext *global);
const struct Nif *my_nif_get_nif(const char *nifname);
```

Note. Instructions for implementing Nifs is outside of the scope of this document.

- Add your `<moniker>` to the `main/component_nifs.txt` file in the `src/platforms/esp32` directory.

Note. The `main/component_nifs.txt` file will not exist until after the first clean build.

Adding a custom AtomVM Port

To add support for a new peripheral or protocol using an AtomVM port, you need to do the following:

- Choose a name for your port (e.g, "my_port"). Call this `<moniker>`.
- In your source code, implement the following two functions:
  - `void <moniker>_init(GlobalContext *global);`
    - This function will be called once, when the application is started.
  - `Context *<moniker>_create_port(GlobalContext *global, term opts);`
    - This function will be called to locate the Nif during a function call.

Example:

```
void my_port_init(GlobalContext *global);
Context *my_port_create_port(GlobalContext *global, term opts);
```

Note. Instructions for implementing Ports is outside of the scope of this document.

- Add your `<moniker>` to the `main/component_ports.txt` file in the `src/platforms/esp32` directory.

Note. The `main/component_ports.txt` file will not exist until after the first clean build.

7.5 Building for STM32

7.5.1 Prerequisites

- `arm-none-eabi/arm-elf` toolchain compatible with your system
- `cmake`
- `make`
- `git`
- `libopencm3`

7.5.2 Setup libopencm3

Before building for the first time you need to have a compiled clone of the libopencm3 libraries, from inside the AtomVM/src/platforms/stm32 directory:

```
$ git clone https://github.com/libopencm3/libopencm3.git
$ cd libopencm3 && make -j4 && cd ..
```

> Note: You can put libopencm3 wherever you want on your PC as long as you update LIBOPENCM3_DIR to point to it. This example assumes it has been cloned into /opt/libopencm3 and built. From inside the AtomVM/src/platforms/stm32 directory:``` cmake -DCMAKE_TOOLCHAIN_FILE=../cmake/arm-toolchain.cmake -DLIBOPENCM3_DIR=/opt/libopencm3 ..

7.5.3 Build AtomVM

```
$ mkdir build
$ cd build
$ cmake -DCMAKE_TOOLCHAIN_FILE=../cmake/arm-toolchain.cmake
$ make
```

7.5.4 Changing device

The default build is based on the STM32F4Discovery board chip (`stm32f407vgt6`). If you want to target a different chip, pass the `-DDEVICE` flag when invoking cmake. For example, to use the STM32F429Discovery, pass `-DDEVICE=stm32f429zit6`

If you are building for a different target board the `CLOCK_FREQUENCY` definition in main.c will need to be changed to match the clock frequency (in hertz) of your cpu.

The rcc_clock_setup_XXX_XXX will also need to be changed to match your particular chip-set. Consult ST's documentation for appropriate settings.

7.5.5 Configure logging with `cmake`

The default maximum log level is `LOG_INFO`. To change the maximum level displayed pass `-DAVM_LOG_LEVEL_MAX="{level}"` to `cmake`, with one of `LOG_ERROR`, `LOG_WARN`, `LOG_INFO`, or `LOG_DEBUG` (listed from least to most verbose). Log messages can be completely disabled by using `-DAVM_LOG_DISABLE=on`.

For log entries colorized by log level pass `-DAVM_ENABLE_LOG_COLOR=on` to cmake. With color enable there is a very small performance penalty (~1ms per message printed), the log entries are colored as follows:

Message Level	Color
ERROR	Red
WARN	Orange
INFO	Green
DEBUG	Blue

By default only `ERROR` messages contain file and line number information. This can be included with all log entries by passing `-DAVM_ENABLE_LOG_LINES=on` to cmake, but it does incur a significant performance penalty and is only suggested for debugging during development.

7.5.6 Printing

AtomVM is built with standard `newlib` to support `long long` integers (`signed` and `unsigned`). If you are building for a device with extremely limited flash space the `nano` version of `newlib` can be used instead. This may be done by passing `-DAVM_NEWLIB_NANO=on`. If the `nano newlib` is used logs will be automatically disabled, this is because many of the VM low level log messages will include `%ull` formatting and will cause buffer overflows and crash the VM if logging is not disabled for `nano newlib` builds. The total flash savings of using `nano newlib` and disabling logs is just under 40kB.

By default, stdout and stderr are printed on USART2. On the STM32F4Discovery board, you can see them using a TTL-USB with the TX pin connected to board's pin PA2 (USART2 RX). Baudrate is 115200 and serial transmission is 8N1 with no flow control.

If building for a different target USART and gpio pins may need to be adjusted in `main.c`.

7.6 Building for Raspberry Pi Pico

7.6.1 Prerequisites

- `cmake`
- `ninja`
- `Erlang/OTP`
- `Elixir` (optional)

7.6.2 AtomVM build steps

```
cd src/platforms/rp2040/
mkdir build
cd build
cmake .. -G Ninja
ninja
```

You may want to build with option `AVM_REBOOT_ON_NOT_OK` so Pico restarts on error.

7.6.3 libAtomVM build steps

Build of standard libraries is part of the generic unix build.

From the root of the project:

```
mkdir build
cd build
cmake .. -G Ninja
ninja
```

7.6.4 Running tests

Tests for Pico/RP2040 are run on the desktop (or CI) using rp2040js. Running tests currently require nodejs 20.

Change directory to the `src/platforms/rp2040/tests` directory under the AtomVM source tree root:

```
shell$ cd <atomvm-source-tree-root>
shell$ cd src/platforms/rp2040/tests
```

Install the emulator and required Javascript dependencies:

```
shell$ npm install
```

We are assuming tests were built as part of regular build of AtomVM. Run them with the commands:

```
shell$ npx tsx run-tests.ts ../build/tests/rp2040_tests.uf2
../build/tests/test_erl_sources/rp2040_test_modules.uf2
```

7.7 Building for NodeJS/Web

Two different builds are possible, depending on link options: for NodeJS and for the web browser.

7.7.1 Prerequisites

- emscripten SDK
- `cmake`
- Erlang/OTP
- Elixir (optional)

7.7.2 Building for NodeJS

This is the default. Execute the following commands:

```
cd src/platforms/emscripten/
mkdir build
cd build
emcmake cmake ..
emmake make -j
```

AtomVM can then be invoked as on Generic Unix with node:

```
node ./src/AtomVM.js
```

7.7.3 Running tests with NodeJS

NodeJS build currently does not have dedicated tests. However, you can run AtomVM library tests that do not depend on unimplemented APIs.

Build them first by building AtomVM for Generic Unix (see above.) Then execute the tests with:

```
cd src/platforms/emscripten/build/
node ./src/AtomVM.js ../../../../build/tests/libs/eavmlib/test_eavmlib.avm
node ./src/AtomVM.js ../../../../build/tests/libs/alisp/test_alisp.avm
```

7.7.4 Building for the web

Execute the following commands:

```
cd src/platforms/emscripten/
mkdir build
cd build
emcmake cmake .. -DAVM_EMSCRIPTEN_ENV=web
emmake make -j
```

7.7.5 Running tests with Cypress

AtomVM WebAssembly port on the web uses SharedArrayBuffer feature which is restricted by browsers. Tests require an HTTP server that returns the proper HTTP headers.

Additionally, tests require Cypress. Plus, because of a current bug in Cypress, tests only run with Chrome-based browsers except Electron (Chromium, Chrome or Edge).

Build first AtomVM for Generic Unix (see above). This will include the web server.

Then run the web server with:

```
cd build
./src/AtomVM examples/emscripten/wasm_webserver.avm
```

In another terminal, compile specific test modules that are not part of examples.

```
cd src/platforms/emscripten/build/
make emscripten_erlang_test_modules
```

Then run tests with Cypress with:

```
cd src/platforms/emscripten/tests/
npm install cypress
npx cypress run --browser chrome
```

You can alternatvely specify: `chromium` or `edge` depending on what is installed.

Alternatively, on Linux, you can run tests with docker:

```
cd src/platforms/emscripten/tests/
docker run --network host -v $PWD:/mnt -w /mnt cypress/included:12.17.1 --browser
chrome
```

Or you can open Cypress to interactively run selected test suites.

```
cd src/platforms/emscripten/tests/
npm install cypress
npx cypress open
```

Chapter 8

AtomVM Internals

8.1 What is an Abstract Machine?

AtomVM is an "abstract" or "virtual" machine, in the sense that it simulates, in software, what a physical machine would do when executing machine instructions. In a normal computing machine (e.g., a desktop computer), machine code instructions are generated by a tool called a compiler, allowing an application developer to write software in a high-level language (such as C). (In rare cases, application developers will write instructions in assembly code, which is closer to the actual machine instructions, but which still requires a translation step, called "assembly", to translate the assembly code into actual machine code.) Machine code instructions are executed in hardware using the machine's Central Processing Unit (CPU), which is specifically designed to efficiently execute machine instructions targeted for the specific machine architecture (e.g., Intel x86, ARM, Apple M-series, etc.) As a result, machine code instructions are typically tightly packed, encoded instructions that require minimum effort (on the part of the machine) to unpack an interpret. These a low level instructions unsuited for human interpretation, or at least for most humans.

AtomVM and virtual machines generally (including, for example, the Java Virtual Machine) perform a similar task, except that i) the instructions are not machine code instructions, but rather what are typically called "bytecode" or sometimes "opcode" instructions; and ii) the generated instructions are themselves executed by a runtime execution engine written in software, a so-called "virtual" or sometimes "abstract" machine. These bytecode instructions are generated by a compiler tailored specifically for the virtual machine. For example, the `javac` compiler is used to translate Java source code into Java VM bytecode, and the `erlc` compiler is used to translate Erlang source code into BEAM opcodes.

AtomVM is an abstract machine designed to implement the BEAM instruction set, the 170+ (and growing) set of virtual machine instructions implemented in the Erlang/OTP BEAM.

> Note that there is no abstract specification of the BEAM abstract machine and instruction set. Instead, the BEAM implementation by the Erlang/OTP team is the definitive specification of its behavior.

At a high level, the AtomVM abstract machine is responsible for:

- Loading and execution of the BEAM opcodes encoded in one or more BEAM files;
- Managing calls to internal and external functions, handling return values, exceptions, and crashes;
- Creation and destruction of Erlang "processes" within the AtomVM memory space, and communication between processes via message passing;
- Memory management (allocation and reclamation) of memory associated with Erlang "processes"
- Pre-emptive scheduling and interruption of Erlang "processes"
- Execution of user-defined native code (Nifs and Ports)

- Interfacing with the host operating system (or facsimile)

This document provides a description of the AtomVM abstract machine, including its architecture and the major components and data structures that form the system. It is intended for developers who want to get involved in bug fixing or implementing features for the VM, as well as for anyone interested in virtual machine internals targeted for BEAM-based languages, such as Erlang or Elixir.

8.2 AtomVM Data Structures

This section describes AtomVM internal data structures that are used to manage the load and runtime state of the virtual machine. Since AtomVM is written in C, this discussion will largely be in the context of native C data structures (i.e., `structs`). The descriptions will start at a fairly high level but drill down to some detail about the data structures, themselves. This narrative is important, because memory is limited on the target architectures for AtomVM (i.e., micro-controllers), and it is important to always be aware of how memory is organized and used in a way that is as space-efficient as possible.

8.2.1 The GlobalContext

We start with the top level data structure, the `GlobalContext` struct. This object is a singleton object (currently, and for the foreseeable future), and represents the root of all data structures in the virtual machine. It is in essence in 1..1 correspondence with instances of the virtual machine.

> Note. Given the design of the system, it is theoretically possible to run multiple instances of the AtomVM in one process space. However, no current deployments make use of this capability.

In order to simplify the exposition of this structure, we break the fields of the structure into manageable subsets:

- Process management – fields associated with the management of Erlang (lightweight) "processes"
- Atoms management – fields associated with the storage of atoms
- Module Management – fields associated with the loading of BEAM modules
- Reference Counted Binaries – fields associated with the storage of binary data shared between processes
- Other data structures

These subsets are described in more detail below.

> Note. Not all fields of the `GlobalContext` structure are described in this document.

Process Management

As a BEAM implementation, AtomVM must be capable of spawning and managing the lifecycle of Erlang lightweight processes. Each of these processes is encapsulated in the `Context` structure, described in more detail in subsequent sections.

The `GlobalContext` structure maintains a list of running processes and contains the following fields for managing the running Erlang processes in the VM:

- `processes_table` the list of all processes running in the system
- `waiting_processes` the subset of processes that are waiting to run (e.g., waiting for a message or timeout condition).
- `running_processes` the subset of processes that are currently running.
- `ready_processes` the subset of processes that are ready to run.

Processes are in either `waiting_processes`, `running_processes` or `ready_processes`. A running process can technically be moved to the ready list while running to signify that if it yields, it will be eligible for being run again, typically if it receives a message. Also, native handlers (ports) are never moved to the `running_processes` list but are in the `waiting_processes` list when they run (and can be moved to `ready_processes` list if they are made ready while running).

Each of these fields are doubly-linked list (ring) structures, i.e, structs containing a `prev` and `next` pointer field. The `Context` data structure begins with two such structures, the first of which links the `Context` struct in the `processes_table` field, and the second of which is used for either the `waiting_processes`, the `ready_processes` or the `running_processes` field.

> Note. The C programming language treats structures in memory as contiguous sequences of fields of given types. Structures have no hidden pramble data, such as you might find in C++ or who knows what in even higher level languages. The size of a struct, therefore, is determined simply by the size of the component fields.

The relationship between the `GlobalContext` fields that manage BEAM processes and the `Context` data structures that represent the processes, themselves, is illustrated in the following diagram:

Error opening image file: cannot identify image file '/home/runner/-work/AtomVM/AtomVM/build/doc/src/_static/globalcontext-processes.svg'

> Note. The `Context` data structure is described in more detail below.

Module Management

An Aside: What's in a HashTable?

8.2.2 Modules

8.2.3 Contexts

8.3 Runtime Execution Loop

8.4 Module Loading

8.5 Function Calls and Return Values

8.6 Exception Handling

8.7 The Scheduler

In SMP builds, AtomVM runs one scheduler thread per core. Scheduler threads are actually started on demand. The number of scheduler threads can be queried with `erlang:system_info/1` and be modified with `erlang:system_flag/2`. All scheduler threads are considered equal and there is no notion of main thread except when shutting down (main thread is shut down last).

Each scheduler thread picks a ready process and execute it until it yields. Erlang processes yield when they are waiting (for a message) and after a number of reductions elapsed. Native processes yield when they are done consuming messages (when the handler returns).

Once a scheduler thread is done executing a process, if no other thread is waiting into `sys_poll_events`, it calls `sys_poll_events` with a timeout that correspond to the time to wait for next execution. If there are ready processes, the timeout is 0. If there is no ready process, this scheduler thread will wait into `sys_poll_event` and depending on the platform implementation, the CPU usage can drop.

If there already is one thread in `sys_poll_events`, other scheduler threads pick the next ready process and if there is none, wait. Other scheduler threads can also interrupt the wait in `sys_poll_events` if a process is made ready to run. They do so using platform function `sys_signal`.

8.8 Mailboxes and signals

Erlang processes receive messages in a mailbox. The mailbox is the interface with other processes.

When a sender process sends a message to a recipient process, the message is first enqueued into an outer mailbox. The recipient process eventually moves all messages from the outer mailbox to the inner mailbox. The reason for the inner and outer mailbox is to use lock-free data structures using atomic CAS operations.

Sometimes, Erlang processes need to query information from other processes but without sending a regular message, for example when using `process_info/1,2` nif. This is handled by signals. Signals are special messages that are enqueued in the outer mailbox of a process. Signals are processed by the recipient process when regular messages from the outer mailbox are moved to the inner mailbox. Signal processing code is part of the main loop and transparent to recipient processes. Both native handlers and erlang processes can receive signals. Signals are also used to run specific operation on other processes that cannot be done from another thread. For example, signals are used to perform garbage collection on another process.

When an Erlang process calls a nif that requires such an information from another process such as `process_info/1,2`, the nif returns a special value and set the Trap flag on the calling process. The calling process is effectively blocked until the other process is scheduled and the information is sent back using another signal message. This mechanism can also be used by nifs that want to block until a condition is true.

8.9 Stacktraces

Stacktraces are computed from information gathered at load time from BEAM modules loaded into the application, together with information in the runtime stack that is maintained during the execution of a program. In addition, if a BEAM file contains a `Line` chunk, additional information is added to stack traces, including the file name (as defined at compile time), as well as the line number of a function call.

> Note. Adding line information to a BEAM file adds non-trivial memory overhead to applications and should only be used when necessary (e.g., during the development process). For applications to make the best use of memory in tightly constrained environments, packagers should consider removing line information all together from BEAM files and rely instead on logging or other mechanisms for diagnosing problems in the field.

Newcomers to Erlang may find stacktraces slightly confusing, because some optimizations taken by the Erlang compiler and runtime can result in stack frames "missing" from stack traces. For example, tail-recursive function calls, as well as function calls that occur as the last expression in a function clause, don't involve the creation of frames in the runtime stack, and consequently will not appear in a stacktrace.

8.9.1 Line Numbers

Including file and line number information in stacktraces adds considerable overhead to both

the BEAM file data, as well as the memory consumed at module load time. The data structures used to track line numbers and file names are described below and are only created if the associated BEAM file contains a `Line` chunk.

The line-refs table

The line-refs table is an array of 16-bit integers, mapping line references (as they occur in BEAM instructions) to the actual line numbers in a file. (Internally, BEAM instructions do not reference line numbers directly, but instead are indirected through a line index). This table is stored on the `Module` structure.

This table is populated when the BEAM file is loaded. The table is created from information in the `Line` chunk in the BEAM file, if it exists. Note that if there is no `Line` chunk in a BEAM file, this table is not created.

The memory cost of this table is `num_line_refs * 2` bytes, for each loaded module, or 0, if there is no `Line` chunk in the associated BEAM file.

The filenames table

The filenames table is a table of (usually only 1?) file name. This table maps filename indices to `ModuleFilename` structures, which is essentially a pointer and a length (of type `size_t`). This table generally only contains 1 entry, the file name of the Erlang source code module from which the BEAM file was generated. This table is stored on the `Module` structure.

Note that a `ModuleFilename` structure points to data directly in the `Line` chunk of the BEAM file. Therefore, for ports of AtomVM that memory-map BEAM file data (e.g., ESP32), the actual file name data does not consume any memory.

The memory cost of this table is `num_filenames * sizeof(struct ModuleFilename)`, where `struct ModuleFilename` is a pointer and length, for each loaded module, or 0, if there is no `Line` chunk in the associated BEAM file.

The line-ref-offsets list

The line-ref-offsets list is a sequence of `LineRefOffset` structures, where each structure contains a ListHead (for list book-keeping), a 16-bit line-ref, and an unsigned integer value designating the code offset at which the line reference occurs in the code chunk of the BEAM file. This list is stored on the `Module` structure.

This list is populated at code load time. When a line reference is encountered during code loading, a `LineRefOffset` structure is allocated and added to the line-ref-offsets list. This list is used at a later time to find the line number at which a stack frame is called, in a manner described below.

The memory cost of this list is `num_line_refs * sizeof(struct LineRefOffset)`, for each loaded module, or 0, if there is no `Line` chunk in the associated BEAM file.

8.9.2 Raw Stacktraces

8.10 AtomVM WebAssembly port

WebAssembly or Wasm port of AtomVM relies on Emscripten SDK and library. Even when SMP is disabled (with `-DAVM_DISABLE_SMP=On`), it uses pthread library to sleep when Erlang processes are not running (to not waste CPU cycles).

8.10.1 NodeJS environment build

The NodeJS environment build of this port is relatively straightforward, featuring NODERAWFS which means it can access files directly like node does.

8.10.2 Web environment build

The Web environment build of this port is slightly more complex.

Regarding files, `main` function can load modules (beam or AVM packages) using FetchAPI, which means they can be served by the same HTTP server. This is a fallback and users can preload files using Emscripten `file_packager` tool.

The port also uses Emscripten's proxy-to-pthread feature which means AtomVM's `main` function is run in a web worker. The rationale is the browser thread (or main thread) with WebAssembly cannot run a loop such as AtomVM's schedulers. Web workers typically cannot manipulate the DOM and do other things that only the browser's main thread can do. For this purpose, Erlang processes can call `emscripten:run_script/2` function which dispatches the Javascript to execute to the main thread, waiting for completion (with `[main_thread]`) or not waiting for completion (with `[main_thread, async]`). Waiting for completion of a script on the main thread does not block the Erlang scheduler, other Erlang processes can be scheduled. Execution of Javascript on the worker thread, however, does block the scheduler.

Javascript code can also send messages to Erlang processes using `call` and `cast` functions from `main.c`. These functions are actually wrapped in `atomvm.pre.js`. Usage is demonstrated by `call_cast.html` example.

Cast is straightforward: the message is enqueued and picked up by the scheduler. It is freed when it is processed.

Call allows Javascript code to wait for the result and is based on Javascript promises (related to `async/await` syntax).

- A promise is created (in the browser's main thread) in a map to prevent Javascript garbage collection (this is done by Emscripten's promise glue code).
- An Erlang resource is created to encapsulate the promise so it is properly destroyed when garbage collected
- A message is enqueued with the resource as well as the registered name of the target process and the content of the message
- C code returns the handle of the promise (actually the index in the map) to Javascript Module.-call wrapper.
- The `Module.call` wrapper converts the handle into a Promise object and returns it, so Javascript code can await on the promise.
- A scheduler dequeues the message with the resource, looks up the target process and sends it the resource as a term
- The target process eventually calls `emscripten:promise_resolve/1,2` or `emscripten:promise_reject/1,2` to resolve or reject the promise.
- The `emscripten:promise_resolve/1,2` and `emscripten:promise_reject/1,2` nifs dispatch a message in the browser's main thread.
- The dispatched function retrieves the promise from its index, resolves or rejects it, with the value passed to `emscripten:promise_resolve/2` or `emscripten:promise_reject/2` and destroys it.

Values currently can only be integers or strings.

If the scheduler cannot find the target process, the promise is rejected with "noproc" as a value. As the promise is encapsulated into an Erlang resource, if the resource object's reference count reaches 0, the promise is rejected with "noproc" as the value.

Chapter 9

Memory Management

Like most managed execution environments, AtomVM provides automated memory management for compiled Erlang/Elixir applications that run on the platform, allowing developers to focus on the logic of application programs, instead of the minutiae of managing the allocation and disposal of memory in the process heap of the program.

Because Erlang/Elixir, and the BEAM, specifically, is a shared-nothing, concurrency-based language, AtomVM can manage memory independently, for each unit of concurrency, viz., the Erlang process. While there is some global state, internally, that AtomVM manages (e.g., to manage all running processes in the system), memory management for each individual process can be performed independently of any other process.

AtomVM internally uses a "Context" structure, to manage aspects of a process (including memory management), and we use "execution context" and "Erlang process" interchangeably in this document. As usual, an Erlang process should be distinguished from the Operating System (OS) process in which Erlang processes run.

For any given execution context, there are three regions of memory that are relevant: i) the stack, ii) the heap, and iii) registers. The stack and heap actually occupy one region of memory allocated in the OS process heap (via malloc or equiv), and grow in opposite directions towards each other. Registers in AtomVM are a fixed size array of 16 elements.

The fundamental unit of memory that occupies space in the stack, heap, and registers is the `term`, which is typedef'd internally to be an integral type that fits in a single word of machine memory (i.e., a C `int`). Various tricks are used, described below, to manage and reference multi-word terms, but in general, a term (or in some cases, a term pointer) is intended to fit into a single word or memory.

This document describes the memory layout for each execution context (i.e., Erlang/Elixir process), how memory is allocated and used, how terms are represented internally, and how AtomVM makes room for more terms, as memory usage increases and as terms go out of scope and are no longer used by the application, and can hence be garbage collected.

9.1 The Context structure

9.1.1 The Heap and Stack

The heap and stack for each AtomVM process are stored in a single allocated block of memory (e.g., via the `malloc` C function) in the heap space of the AtomVM program, and the AtomVM runtime manages the allocation of portions of this memory during the execution of a program. The heap starts at the bottom of the block of memory, and grows incrementally towards the top of the allocated block, as memory is allocated in the program. Each word in the heap and stack (or in some cases, a sequence of words) represent a term that has been allocated.

The heap contains all of the allocated terms in an execution context. In some cases, the terms occupy more than one word of memory (e.g., a tuple), but in general, the heap contains a record of memory in use by the program.

The heap grows incrementally, as memory is allocated, and terms are allocated sequentially, in increasing memory addresses. There is, therefore, no memory fragmentation, properly speaking, at least insofar as a portion of memory might be in use and then freed. However, it is possible that previously allocated blocks of memory in the context heap are no longer referenced by the program. In this case, the allocated blocks are "garbage", and are reclaimed at the next garbage collection. The actual growth of the heap is controlled by a heap growth strategy (`atomvm_heap_growth` spawn option) as described Section 9.1.2.

> Note. It is possible for the AtomVM heap, as provided by the underlying operating system, to become fragmented, as the execution context stack and heap are allocated via `malloc` or equiv. But that is a different kind of fragmentation that does not refer to the allocated block used by an individual AtomVM process.

The stack grows from the top of the allocated block toward the heap in decreasing addresses. Terms in the stack, as opposed to the heap, are either single-word terms, i.e., simple terms like small integers, process ids, etc, or *pointers* to terms in the heap. In either case, they only occupy one word of memory.

The region between the stack and heap is the free space available to the Erlang/Elixir process.

The following diagram illustrates an allocated block of memory that stores terms (or term pointers) in the heap and stack:

```
+================================+ <- heap_start --
|             word[0]            |      ^      ^
+--------------------------------+      |      |
|             word[1]            |      |      |
+--------------------------------+      |      |
|             word[2]            |      | heap |
+--------------------------------+      |      |
|               ...              |      |      |
+--------------------------------+      |      |
|                                |      v      |
+--------------------------------+ <- heap_ptr |
|                                |      ^      |
|                                |      |      |
|                                |      |      |
|                                |      | free |
|                                |      |      |
|                                |      |      |
|                                |      v      |
+--------------------------------+ <- e ----   |
|                                |      ^      |
+--------------------------------+      |      |
|                                |      |      |
+--------------------------------+      | stack|
|                                |      |      |
+--------------------------------+      |      |
|           word[n-1]            |      v      v
+================================+ <- stack_base --
```

The initial size of the allocated block for the stack and heap in AtomVM is 8 words. As heap and stack allocations grow, eventually, the amount of free space will decrease to the point where a garbage collection is required. In this case, a new but larger block of memory is allocated by the AtomVM OS process, and terms are copied from the old stack and heap to the new stack and heap. Garbage collection is described in more detail below.

9.1.2 Heap growth strategies

AtomVM aims at minimizing memory footprint and several heap growth strategies are available. The heap is grown or shrunk when an allocation is required and the current execution context allows for a garbage collection (that will move data structures), allows for shrinking or forces shrinking (typically in the case of a call to `erlang:garbage_collect/0,1`).

Each strategy is set at the process level.

Default strategy is bounded free (`{atomvm_heap_growth, bounded_free}`). In this strategy, when more memory is required, the allocator keeps the free amount between fixed boundaries (currently 16 and 32 terms). If no allocation is required but free space is larger than boundary, a garbage collection is triggered. After copying data to a new heap, if the free space is larger than the maximum, the heap is shrunk within the boundaries.

With minimum strategy (`{atomvm_heap_growth, minimum}`), when an allocation can happen, it is always adjusted to have the free space at 0.

With fibonacci strategy (`{atomvm_heap_growth, fibonacci}`), heap size grows following a variation of fibonacci until a large value and then grows by 20%. If free space is larger than 75% of heap size, the heap is shrunk. This strategy is inspired from Erlang/OTP's implementation.

9.1.3 Registers

Registered are allocated in an array of 16 terms (words) and are referenced by the `x` field in the Context data structure:

```
+---------+---------+---------+--------+
|   x[0]  |   x[1]  |   ...   |  x[15] |
+---------+---------+---------+--------+
```

Like terms in the stack, terms in registers are either single-word terms, i.e., simple terms like small integers, process ids, etc, or *pointers* to terms in the heap, in a manner described in more detail below. In either case, they only occupy one word of memory.

Registers are used as part of the BEAM instruction set to store and retrieve values that are passed between BEAM instruction opcodes.

9.1.4 Process Dictionary

AtomVM processes support a process dictionary, or map of process-specific data, as supported via the `erlang:put/2` and `erlang:get/1` functions.

The Process Dictionary contains a list of key-value pairs, where each key and value is a single-word term, either a simple term like an atom or pid, or a reference to an allocated object in the process heap. (see below)

9.1.5 Heap Fragments

AtomVM makes use of heap fragments in some edge cases, such as loading external terms from the literals table in a BEAM file. Heap fragments are individually allocated blocks of memory that contain may contain multi-word term structures. The data in heap fragments are copied into the heap during a garbage collection event, and then deleted, so heap fragments are generally short lived. However, during execution of a program, there may be references to term structures in such fragments from the stack, registers, the process dictionary, or from nested terms in the process heap.

9.1.6 Mailbox

Each Erlang process contains a process mailbox, which is a linked-list structure of messages. Each message in this list contains a term structure, which is a copy of a term sent to it, e.g., via the `erlang:send/2` operation, or `!` operator.

The representation of terms in a message is identical to that in the heap and heap fragments. Messages are allocated like fragments and they actually become heap fragments of the receiving process when the message is read off the mailbox (e.g., via `receive ... end`). Messages (and their term contents) are moved to the main heap as part of regular garbage collection of the process, and the fragment is freed.

9.1.7 Memory Graph

Memory is allocated in the execution context heap, and structured types, such as tuples and lists, generally include references to the blocks of memory that have been previously allocated.

For example, if we look at the memory allocated for the term

```
{foo, [{bar, self()}]}
```

we would generally see something like the following in the execution context heap:

```
|           ...             |
|                           |
+---------------------------+
|          tuple            |<--+
+---------------------------+   |
|          bar              |   |
+---------------------------+   |
|          <0.1.0>          |   |
+---------------------------+   |
|          []               |<- | --+
+---------------------------+   |   |
|          tuple ptr        |---+   |
+---------------------------+       |
|          tuple            |       |
+---------------------------+       |
|          foo              |       |
+---------------------------+       |
|          list ptr         |-------+
+---------------------------+
|                           |
|           ...             |
0123456789012345678901234567890123456789012345678 9
```

The tuple `{bar, self()}` is allocated in a block, and the list `[{bar, self()}]` (or, technically, `[{bar, self()} | []]`) contains elements that *point* to it elements (in this case, `[]` and `{bar, self()}` – note that in general, in AtomVM, the address of the tail of a list occupies the first byte in the list – more details on that below). Finally, the tuple `{foo, [{bar, self()}]}` contains the atom `foo` and a *pointer* to the list it contains.

In this way, the set of allocated blocks in the execution context heap forms a directed graph of objects, whose nodes are structured terms (lists, tuples, etc) and whose leaves are simple terms, like atoms, pids, and so forth. Note that because BEAM-based languages such as Erlang and Elixir are true functional programming languages, these directed graphs have no cycles.

The stack, registers, and process dictionary contain pointers to terms in the heap. We call these terms "root" nodes, and any term in the heap that is referenced by a root node, or any term that is so referenced by such a term, is in the path of a root node. Some terms in the heap are not in the path of a root node. We call these terms "garbage".

Note that the values in the stack and register root nodes change over time as the result of the execution of Erlang opcodes, and are dependent on the BEAM output of the Erlang compiler, along with inputs to the program being executed. Thus, a term in the process heap may become garbage, once it is no longer reachable from the root set. But once garbage, the term will always remain garbage, at least until it is reclaimed during a garbage collection event. For more information about how the garbage collector works, see the Garbage Collection section, below.

9.2 Simple Terms

The fundamental unit of memory in AtomVM is the `term` object, which is designed to fit either into a single machine work (single-word terms), or into multiple words (so called "boxed terms" and lists).

This section enumerates the AtomVM term types, and how they are represented in memory.

> Note. The term type is overloaded in some cases to store raw pointers to memory addresses, but this is rare and well controlled.

The following term types take up a single word, referred to as "immediates" in the BEAM documentation[1]. The low-order bits of the word are used to represent the type of the term, and the high order bits represent the term contents, in a manner described in the following sections.

9.2.1 Atoms

An atom is represented as a single word, with the low-order 6 bits having the value `0xB` (`001011b`). The high order word-size-6 bits are used to represent the index of the atom in the global atom table:

```
                           |< 6  >|
+=========================+======+
|       atom index        |001011| <- 0xB
+=========================+======+
|                                |
|<---------- word-size --------->|
```

There may therefore only be `2^{word-size-6}` atoms in an AtomVM program (e.g., on a 32-bit platform, `67,108,864`). Plenty to work with!

> Note. The global atom table is a table of all allocated atoms, and is generally (at least in the limit, as modules are loaded) a fixed size table. Management of the global atom table is outside of the scope of this document.

9.2.2 Integers

An integer is represented as a single word, with the low-order 4 bits having the value `0xF` (`1111b`). The high order word-size-6 bits are used to represent the integer value:

```
                             |< 4>|
+============================+====+
|       integer value        |1111| <- 0xF
+============================+====+
|                                |
|<---------- word-size --------->|
```

The magnitude of an integer is therefore limited to `2^{word-size - 4}` in an AtomVM program (e.g., on a 32-bit platform, `+- 134,217,728`).

> Note. Arbitrarily large integers (bignums) are not currently supported in AtomVM.

9.2.3 nil

The special value `nil` (typically the tail of the tail … of the tail of a list, or `[]`) is the special value 0x3B:

```
+================================+
|000         ...       0000111011| <- 0x3B
+================================+
|                                |
|<---------- word-size --------->|
```

9.2.4 Pids

A Pid is represented as a single word, with the low order 4 bits indicating the Pid term type (`0x03`), and (for now), the high order `word-size - 4` bits store the local process id:

```
                             |< 4>|
+============================+====+
```

```
|     local process id      |0011| <- 0x3
+===========================+====+
|                                |
|<---------- word-size --------->|
```

There may therefore only be `2^{word-size - 4}` Pids in an AtomVM program (e.g., on a 32-bit platform, `268,435,456`).

> Note. Global process IDs are not currently supported, but they may be in the future, which may result in segmentation of the high order `word-size - 4` bits.

9.3 Boxed terms

Some term types cannot fit in a single word, and must therefore used a sequence of contiguous words to represent the term contents. These terms are called "Boxed" terms. Boxed terms use the low-order 6 bits of the first byte (`boxed[0]`) to represent the term type, and the high order `word-size - 6` bits to represent the remaining size (in words) of the boxed term, not including the first word.

9.3.1 Boxed term pointers

Before discussing the different types of boxed terms in detail, let us first see how boxed terms are referenced from the stack, registers, process dictionary, and from embedded terms in the heap. We call such references to boxed terms boxed term pointers.

A boxed term pointer is a single-word term that contains the address of the referenced term in the high-order `word-size - 2` bits, and `0x2` (`10b`) in the low-order 2 bits.

```
                              |2 |
+============================+==+
|       term address         |10| <- term pointer type (2 bits)
+============================+==+
|                                |
|<---------- word-size --------->|
```

Because terms (and hence the heap) are always aligned on boundaries that are divisible by the word size, the low-order 2 bits of a term address are always 0. Consequently, the high-order word-size - 2 (`1,073,741,824`, on a 32-bit platform) are sufficient to address any term address in the AtomVM address space, for 32-bit and greater machine architectures.

9.3.2 References

A reference (e.g., created via `erlang:make_ref/0`) stores a 64-bit incrementing counter value (a "ref tick"). On 64 bit machines, a Reference takes up two words – the boxed header and the 64-bit value, which of course can fit in a single word. On 32-bit platforms, the high-order 28 bits are stored in `boxed[1]`, and the low-order 32 bits are stored in `boxed[2]`:

```
                          |< 6  >|
+========================+======+
|        boxed-size      |010000| boxed[0]
+------------------------+------+
|     high-order ref-ticks      | boxed[1]
+===============================+
|      low-order ref-ticks      | boxed[2] (32-bit only)
+= = = = = = = = = = = = = = = ==+
|                                |
|<---------- word-size --------->|
```

9.3.3 Tuples

Tuples are represented as boxed terms containing a boxed header (`boxed[0]`), a type tag of `0x00`

(`000000b`), followed by a sequence of `n`-many words, which may either (copies of) single-word terms, or boxed term pointers, where `n` is the arity of the tuple:

```
                          |< 6  >|
+=========================+======+
|    boxed-size (n)       |000000| boxed[0]
+-------------------------+------+
|             element-1          | boxed[1]
+--------------------------------+
|             element-2          | boxed[2]
+--------------------------------+
|               ...              | boxed[i]
+--------------------------------+
|             element-n          | boxed[n]
+================================+
|                                |
|<---------- word-size --------->|
```

9.3.4 Maps

Maps are represented as boxed terms containing a boxed header (`boxed[0]`), a type tag of `0x3C` (`111100b`), followed by:

- a term pointer to a tuple of arity `n` containing the keys in the map;
- a sequence of `n`-many words, containing the values of the map corresponding (in order) to the keys in the reference tuple.

The keys and values are single word terms, i.e., either immediates or pointers to boxed terms or lists.

```
        +=========================+======+
+-----> |    boxed-tuple (n)      |000000|
|       +-------------------------+------+
|       |             key-1              |
|       +--------------------------------+
|       |             key-2              |
|       +--------------------------------+
|       |               ...              |
|       +--------------------------------+
|       |             key-n              |
|       +================================+
|       |                                |
|                         ...
|       |                         |< 6  >|
|       +=========================+======+
|       |    boxed-size (n)       |111100| boxed[0]
|       +-------------------------+------+
+-----------------<   keys               | boxed[1]
        +--------------------------------+
        |             value-1            | boxed[2]
        +--------------------------------+
        |               ...              | ...
        +--------------------------------+
        |             value-n            | boxed[2 + n]
        +================================+
        |                                |
        |<---------- word-size --------->|
```

The tuple of keys may or may not be contiguous with the boxed term holding the map itself (and in general will not be, after garbage collection). In addition, maps that are modified [sic] via the `:=` operator (or via `=>`, when the key already exists in the source map) share the keys tuple, for space efficiency.

9.3.5 Binaries

Binaries are stored in several different ways, depending on their size and the kinds of data to which they refer.

Binary data less than 64 bytes in length are stored in the process heap, as so-called Heap Binaries.

Binary data greater or equal to 64 bytes is stored in two manners, depending on whether the data stored is constant data (e.g., literal binary data compiled directly into a BEAM file), or dynamically allocated data, e.g., as the result of a call to the `erlang:list_to_binary/1` Nif.

Non-const binaries are stored outside of the heap in dynamically allocated memory and are reference-counted, whereby references to dynamically allocated blocks are tracked from pointers in heap storage. This way, large blocks of binary data can be efficiently shared between processes; only a relatively small term that contains a reference to the dynamically allocated storage needs to be copied. When the reference count of non-literal binary reaches 0, the dynamically allocated memory is free'd.

Const binaries share similar features to non-const binaries in the process heap; however, instead of pointing to dynamically allocated memory that requires reference counting and memory management, the boxed term in the process heap points directly to constant memory (e.g., a term literal stored in a memory-mapped BEAM file). This is especially useful in memory constrained applications, such as the ESP32 micro-controller, where the BEAM file contents are not read into memory, but are instead directly mapped from flash storage.

Finally, a special kind of binary is used in the heap to maintain the state of a match context, when, for example, matching binary terms using Erlang bit syntax. Like non-const binaries, creation and destruction of match context binaries will affect the reference count on the binaries to which they refer.

The following sub-sections describe these storage mechanisms and memory management in more detail.

Heap Binaries

Heap binaries are represented as boxed terms containing a boxed header (`boxed[0]`), a type tag of `0x024` (`100100b`), followed by the size in bytes of the binary, and then a sequence of `n`-many words, which contains the sequence of `size`-many bytes (`<= word-size * n`):

```
                           |< 6  >|
+=========================+======+
|    boxed-size (n)       |100100| boxed[0]
+-------------------------+------+
|         size (in bytes)        | boxed[1]
+--------------------------------+
| byte-1, byte-2, byte-3,  ...   | boxed[2]
+--------------------------------+
|               ...              | boxed[i]
+-------------------+------------+
| ..., byte-{size-1}| -unused-   | boxed[n+1]
+===================+============+
|                                |
|<---------- word-size --------->|
```

> Note. If the number of bytes in a binary is not evenly divisible by the machine word size, then the remaining sequence of bytes in the last word are unused.

Reference Counted Binaries

Reference counted binaries are represented as boxed terms containing a boxed header (`boxed[0]`), a type tag of `0x020` (`100000b`), followed by the size in bytes of the binary data, a word containing a set of flags, and then a pointer to the off-heap data.

Currently, only the low-order bit of the flags field is used. A `0` value of indicates that the referenced binary is non-literal.

The off-heap data is a block of allocated data, containing:

- a ListHead structure, used to maintain a list of dynamically allocated data (mostly for book-keeping purposes);
- a reference count (unsigned integer);
- the size of the stored data;
- the stored data, itself.

All of the above data is allocated in a single block, so that it can be easily `free`'d when no longer referenced.

The reference count is initialized to 1, under the principle that that reference count is incremented for any occurrence of boxed terms that reference the same data in any heap space, including process heaps, mailbox messages, heap fragments, and so forth. Decrementing reference counts and `free`'ing data in off-heap storage is discussed in more detail below, in the Garbage Collection section.

```
                                |< 6  >|
+--> +=========================+======+
|    |    boxed-size (5)       |100000| boxed[0]
|    +-------------------------+------+
|    |         size (in bytes)        | boxed[1]
|    +--------------------------------+
|    |             flags             0| boxed[2]
|    +--------------------------------+                    off-heap storage
|    |              ptr >-------------- boxed[3] ---> +----------------------+
------
|    +--------------------------------+               |         prev         |
^
|    |              cdr               | boxed[4]      +----------------------+
| ListHead
|    +--------------------------------+               |         next         |
v
+-----------------< car               | boxed[5]      +----------------------+
------
     +==================================+               |   reference-count    |
     |<---------- word-size --------->|               +----------------------+
                                                       |         size         |
                                                       +----------------------+
                                                       |         data         |
                                                        ...
                                                       |                      |
                                                       +----------------------+
```

> Note. The size of a reference counted binary is stored both in the process heap (in the boxed term), as well as in the off-heap storage. The size count in the off-heap storage is needed in order to report the amount of data in use by binaries (e.g., via `erlang:memory/0,1`).

In addition, a reference-counted boxed term contains a cons-cell appended to the end of the boxed term, which is used by the garbage collector for tracking references. The `car` of this cell points to the boxed term, itself, and the `cdr` points to the "previous" cons cell associated with a reference counted binary in the heap, if there is one, or the empty list (`nil`), otherwise. The cons cell forms an element in the "Mark and Sweep Object" (MSO) list, used to reclaim unreferenced storage during a garbage collection event.. See the Garbage Collection section, below, for more information about the critical role of this structure in the process of reclaiming unused memory in the AtomVM virtual machine.

Const Binaries

Const binaries are stored in the same manner as Reference Counted binaries, with the following exceptions:

- The low order bit of the flags field (`boxed[2]`) is `1`, to indicate that the reference binary is

constant;

- The ptr field (`boxed[3]`) points directly to the constant storage (e.g., literal data stored in a memory-mapped BEAM file);
- The trailing cons cell elements are unused, as dynamic memory management for static storage is unnecessary. These values are initialized to `nil`.

This heap structure has the following representation:

```
                           |< 6  >|
+=========================+======+
|    boxed-size (5)       |100000| boxed[0]
+-------------------------+------+
|         size (in bytes)        | boxed[1]
+--------------------------------+
|             flags             1| boxed[2]
+--------------------------------+                        static storage
|              ptr >-------------- boxed[3] -------> +----------------------+
+--------------------------------+                    |          data        |
|             unused             | boxed[4]          |                      |
+--------------------------------+                                ...
|             unused             | boxed[5]          |                      |
+================================+                   +----------------------+
|<---------- word-size --------->|
```

Match Binaries

Match binaries are represented as boxed terms containing a boxed header (`boxed[0]`), a type tag of `0x04` (`000100b`), and the following elements:

- a reference to either a binary or another match binary that refers to a binary;
- an offset in the referenced binary used by the match opcodes;
- a saved state used for backtracking unmatched clause heads;

Like a reference counted binary, a match binary includes a trailing cons cell, whose `car` element points to the actual referenced binary (if the referenced binary is a reference-counted binary), and whose `cdr` points to the "previous" cons cell associated with a reference counted binary in the heap.

> Note. If the referenced binary is not reference-counted, the trailing cons cell elements are unused and are initialized to `nil`.

```
some
binary                            |< 6  >|
^       +=========================+======+
|       |    boxed-size (5)       |100100| boxed[0]
|       +-------------------------+------+
|       |      match-or-binary-ref       | boxed[1]
|       +--------------------------------+
|       |             offset             | boxed[2]
|       +--------------------------------+
|       |             saved              | boxed[3]
|       +--------------------------------+
|       |              cdr               | boxed[4]
|       +--------------------------------+
+--------------------< car               | boxed[5]
        +================================+
        |<---------- word-size --------->|
```

A reference to a reference-counted binary counts as a reference, in which case the creation or copying of a match binary results in the increment of the reference-counted binary's reference count, and the garbage collection of a match binary results in a decrement (and possible `free`ing) of a reference-counted binary. The trailing cons cell becomes an element of the context (or message) MSO list,

and plays a critical role in garbage collection. See the garbage collection section below for more information about the role of this structure.

Sub-Binaries

Sub-binaries are represented as boxed terms containing a boxed header (`boxed[0]`), a type tag of `0x28` (`001000b`)

A sub-binary is a boxed term that points to a reference-counted binary, recording the offset into the binary and the length (in bytes) of the sub-binary. An invariant for this term is that the `offset + length` is always less than or equal to the length of the referenced binary.

```
some
refc
binary                            |< 6  >|
^       +=========================+======+
|       |    boxed-size (3)       |001000| boxed[0]
|       +-------------------------+------+
|       |              len                | boxed[1]
|       +---------------------------------+
|       |             offset              | boxed[2]
|       +---------------------------------+
+----------------< binary-ref             | boxed[3]
        +=================================+
        |<---------- word-size --------->|
```

Note than when a sub-binary is copied between processes (e.g., via `erlang:send`, or `!`), the sub-binary boxed term, as well as the boxed-term that manages the reference-counted binary is copied, as well. Thus, sending a sub-binary to another process will result in an increment of the reference count on the referenced binary, and similarly, garbage collection of the sub-binary will result in a decrement of the referenced binary's reference count.

A sub-binary may be created from both const (literal) and non-const reference-counted binaries. For performance reasons, sub-binaries do not reference heap binaries.

Sub-binaries are created via the `binary:part/3` and `binary:split/2` Nifs, as well as via the `/binary` bit syntax specifier.

9.4 Lists

A list is, very simply, a cons cell, i.e., a sequence of two words, whose first word is a term (single word or term pointer) representing the tail (`cdr`) of the list, and the second of which represents the head (`car`) of the list.

```
+================================+
|               tail             | list_elem[0]
+--------------------------------+
|               head             | list_elem[1]
+================================+
|                                |
|<---------- word-size --------->|
```

> Note. Lists are typically terminated with the empty list (`[]`), represented by the nil term, described above. However, nothing in Erlang requires that a sequence of cons cells is `nil`-terminated.

Unlike boxed terms, the low-order two bits of list pointers are `0x1` (`01b`):

```
+==============================+==+
|       term address           |01| <- list pointer type (2 bits)
+==============================+==+
|                                |
```

```
|<---------- word-size --------->|
```

9.4.1 Strings

Strings are just lists of integers, but they are efficiently allocated at creation time so that a contiguous block of cons cells are created in the heap. They otherwise have the same properties of a list described above.

```
+================================+
|    address-of-next-cons     |01| elem[1]
+--------------------------------+
|             int-value          |
+--------------------------------+
|    address-of-next-cons     |01| elem[2]
+--------------------------------+
|             int-value          |
+--------------------------------+
|                ...          |01| elem[i]
+--------------------------------+
|                ...             |
+--------------------------------+
|                nil             | elem[n]
+--------------------------------+
|             int-value          |
+================================+
|                                |
|<---------- word-size --------->|
```

Note. String elements may not remain contiguous after a garbage collection event.

9.4.2 Functions

Functions are represented as boxed terms containing a boxed header (`boxed[0]`), a type tag of `0x14` (`010100b`), followed by the raw memory address of the Module data structure in which the function is defined, and the function index (so that the function can be located).

In addition, if there are any terms that are used outside of the scope of the function (i.e., closures), these terms are copied from registers into the function objects

```
                          |< 6  >|
+========================+======+
|    boxed-size (n)       |010100| boxed[0]
+------------------------+------+
|         module address         | boxed[1]
+--------------------------------+
|         function index         | boxed[2]
+--------------------------------+
|            closure_1           | boxed[3]
+- - - - - - - - - - - - - - - - +
|               ...              |
+- - - - - - - - - - - - - - - - +
|            closure_k           | boxed[n-1]
+= = = = = = = = = = = = = = = = +
|                                |
|<---------- word-size --------->|
```

9.5 Special Stack Types

Some terms are only used in the stack.

9.5.1 Continuation Pointer

A continuation pointer is a raw address. Because words are aligned on word boundaries, the low order two bits of a continuation pointer are always `0x0` (`(00000000)b`):

```
+================================+
|            raw address      |00|
+================================+
```

9.5.2 Catch Labels

A catch label is used to indicate a position in code to which to jump in a try-catch expression. The term occupies a single term, with the low order 6 bits having the value `0x1B`, the high order 8 bits holding the module index (`m_i`), and the middle 18 bits holding the catch label index (`l_i`):

```
|<   8  >|<     18       >|< 6  >|
+========+================+======+
|   m_i  |     l_i        |011011|
+========+================+======+
|                                |
|<---------- word-size --------->|
```

Module and catch label indices are stored outside of the process heap and are outside of the scope of this document.

9.6 Garbage Collection

Garbage collection refers to the process of removing no-longer referenced term data stored in the heap, making room for new storage, as the program requires. AtomVM implements Tracing Garbage Collection, as does Erlang Garbage Collection. Unlike some garbage collection systems (e.g., as implemented by the Java Virtual Machine), garbage collection in Erlang-based systems, is performed independently on the heap allocated for each active Erlang process; there is no single shared heap for all running Erlang processes.

A given process heap and stack occupy a single region of malloc'd memory, and it is the job of the Erlang VM to manage memory within the allocated regions. Because this region is fixed, every allocation in the heap or stack results in less free space for the Erlang process. When free space reaches a limit, AtomVM will run a garbage collection event, which will allocate a new block of memory to hold the new heap and stack (the actual allocation depends on the heap growth strategy Section 9.1.2), and then copy terms from the old heap and stack to the new heap and stack. Any terms that no longer have references from term pointers in the old stack or registers are not copied to the new stack, and are therefore "collected" as garbage. In addition, any objects in the old heap that reference objects in shared memory (see reference counted binaries, above) are also managed as part of this process, in a manner described below.

```
                                +---------+ ------
                                |   new   |     ^
                                |   heap  |     |
                                +---------+     |
        ---- +----------+       |         |     |
         ^   |          |       |         |     |
         |   |   old    |       |         |     |  new
old      |   |   heap   |       |         |     |  malloc'd
malloc'd|    |          | ===>  |         |     |  region
region   |   +==========+  gc   |  free   |     |
         |   |   old    |       |         |     |
         v   |   stack  |       |         |     |
        ---- +----------+       |         |     |
```

```
                              +---------+    |
                              |   new   |    |
                              |   stack |    v
                              +---------+ -----


+---+---+---+-------------------+---+
| 0 | 1 | 2 |                   | 15|
+---+---+---+-------------------+---+
registers


process dictionary
+--------+--------+
|   k1   |   v1   |
+--------+--------+
|   k2   |   v2   |
+--------+--------+
|       ...       |
```

Terms stored in the stack, registers, and process dictionary are either single-word terms (like atoms or pids) or term references, i.e., single-word terms that point to boxed terms or list cells in the heap. These terms constitute the "roots" of the memory graph of all "reachable" terms in the process.

9.6.1 When does garbage collection happen?

Garbage collection typically occurs as the result of a request for an allocation of a multi-word term in the heap (e.g., a tuple, list, or binary, among other types), and when there is currently insufficient space in the free space between the current heap and the current stack to accommodate the allocation.

Garbage collection is a *synchronous* operation in each Context (Erlang process), but conceptually no other execution contexts are impacted (i.e., no global locks, other than those required for memory allocation in the OS process heap).

> Note. Currently, AtomVM does not support symmetric multi-processing, or execution of multiple processes in parallel on separate machine cores.

9.6.2 Garbage Collection Steps

Garbage collection in AtomVM can be broken down into the following phases:

- Allocation of a new block of memory to store the new heap and stack;
- A "shallow copy" of all root terms (from the stack, registers, and process dictionary) into the heap, as well as updates to the references in the stack, registers, and process dictionary;
- An iterative "scan and copy" of the new heap, until all "live" terms are copied to the new heap;
- A sweep of the "Mark Sweep Object" list;
- Deletion of the old heap.

The following subsections describe these phases in more detail.

Allocation

Garbage collection typically occurs as the result of a request for space on an Erlang process's heap. The amount of space requested is dependent on the kind of term being allocated, but in general, AtomVM will check the amount of free space in the heap, and if it is below the amount of requested space plus some extra (currently, 16 words), then a garbage collection will occur, with the requested allocation space being the current size of the heap, plus the requested size, plus an extra 16 words.

Allocation is a straightforward `malloc` in the (operating system) process heap of the requested set of words. This block of storage will become the "new heap", as opposed to the existing, or "old heap".

Shallow Copy

The garbage collector starts by traversing the current root set, i.e., the terms contained in the stack, registers, and keys and values in the process dictionary, and performs a "shallow copy" of the terms that are in or referenced from these root terms from the old heap to the new heap, while at the same time updating the values in the root set, as some of these values may be pointers into the old heap, and therefore need to be updated to pointers in the new heap.

A shallow copy of a term depends on the type of the term being copied. If the term is a single-word term, like an atom or pid, then the term only resides in the root set, itself, and nothing needs to be copied from the old heap to the new heap. (The term *may* occur in the heap elsewhere, but as an element of another term, like a tuple, for example.)

On the other hand, if the term in the root set points to a boxed term in the old heap, then three things happen:

- The boxed term is copied from the old heap to the new heap. Note that if the term being copied contains pointers to other boxed terms in the old heap, the pointers are *not* updated (yet); they will be as part of the iterative scan and copy (see below);
- The first word of the existing boxed term that was copied is *over-written* with a marker value (`0x2b`) in the old heap, and the second word is over-written with the address of the copied boxed term in the new heap.
- The term in the root set is updated with the address of the copied boxed term in the new heap.

This process is best illustrated with a motivating example:

```
{foo, <<1,2,3,4,...,1024>>}
```

Suppose this term resides in the old heap, and some `register[i]` is a root term pointer to this tuple in the heap:

```
+-> |              |        |             |
|   |              |        |             |
|   |              |        |             |  USED
|   |              |        |             |
          ...                      ...
|   |              |        |             |
|   +--------------+        +=============+ <-- heap
|   |     tuple    |<---+   |             |     addr
|   +--------------+    |   |             |
|   |   atom foo   |    |   |             |
|   +--------------+    |   |             |
+----< refc binary |    |   |             |  FREE
    +--------------+    |   |             |
    |              |    |   |             |
    |     ...      |    |   |     ...     |
    |              |    |   |             |
        old heap        |       new heap
                        |
            ---+----------------+---
           ... |     old-ptr    | ...
            ---+----------------+---
                  register[i]
```

The boxed term is copied to the new heap, overwritten with the marked header `0x2b`, along with a pointer to the new term, and the root term is updated with the same address:

```
+-> |              | <--------+   |             |
|   |              |          |   |             |
|   |              |          |   |             |
|   |              |          |   |             |
```

```
           ...                          ...
|   |              |          |   |             |
|   +--------------+ >>>>>>>>>|>> +-------------+
|   |     0x2b     |   +--+---|-> |   tuple     | USED
|   +--------------+   |  |   |   +-------------+
|   |     ptr  --------+  |   |   |  atom foo   |
|   +--------------+      |   |   +-------------+
+----< refc binary |      |   +----< refc binary|
    +--------------+ >>>>>|>>>>>>>+=============+ <-- new
    |              |      | COPY  |             |   heap
    |     ...      |      |       |             |   addr
         old heap         |       |             |
                          |       |             | FREE
                          |
                          |
             ---+----------------+---
            ... |     new-ptr    | ...
             ---+----------------+---
                   register[i]
```

Note that the first term of the tuple (atom `foo`) is copied to the new heap, but the pointer to the refc binary is out of date – it still points to a value in the old heap. This will be corrected in the iterative scan and copy phase, below.

After a shallow copy of the root set, all terms immediately reachable from the root set have been copied to the new heap, and any boxed terms they reference have been marked as being moved. The new heap consists of a set of contiguous copied boxed terms from the old heap, starting from the base address of the heap, to some higher address in the heap, but less than or equal to the maximum heap size on the new heap.

Iterative Scan and Copy

The iterative scan and copy phase works as follows:

- Starting with the newly created region used in the shallow copy phase in the new heap, iterate over every term in the region (call this the "scan&copy" region);
- If any term in this region is a reference to a term on the old heap that has *not* been marked as copied, perform a shallow copy of it (as described above) to the new heap, but starting at the next free address below the region being iterated over;
- Note that after iterating over all such terms in the scan and copy region, all terms are "complete", in that there are no references to boxed terms in the old heap in that region. We have, however, created a new region which may have references to boxed terms in the old heap;
- So we repeat the process on the new region, which will complete the current scan&copy region, but which in turn may create a new region of copied terms;
- The process is repeated until no new regions have been introduced.

The following sequence of iterative additions to the new heap illustrates this process:

```
+---------------+ ===> +---------------+ ===> +---------------+ ...
|   scan&copy   |      |    complete   |      |    complete   |
|   region      |      |    region     |      |    region     |
|               |      |               |      |               |
|               |      |               |      |               |
|               |      |               |      |               |
|               |      |               |      |               |
|               |      |               |      |               |
+---------------+      +---------------+      +---------------+
                       |     newly     |      |   scan&copy   |
                       |    copied     |      |   region      |
                       |     terms     |      |               |
                       +---------------+      +---------------+
```

```
                                                |newlycpiedterms|
                                                +---------------+



 ... ===> +---------------+ ===> +---------------+
          |    complete   |      |    complete   |
          |    region     |      |    region     |
          |               |      |               |
          |               |      |               |
          |               |      |               |
          |               |      |               |
          |               |      |               |
          |               |      |               |
          |               |      |               |
          |               |      |               |
          |               |      |               |
          +---------------+      |               |
          |  scan&copy    |      |               |
          +---------------+      +---------------+
```

At the end of the iterative scan and copy, all reachable terms in the old heap will be copied to the new heap, and no boxed terms in the old heap will contain pointers to terms in the old heap. Any terms that have not been copied to the new heap are "garbage", as there are no longer any paths to them from the root set, and can therefore be destroyed,

MSO Sweep

As mentioned in the section above on binaries, AtomVM supports reference-counted binaries, whereby binaries of a sufficiently large size (>64 bytes) are allocated outside of the process heap, and are instead referenced from boxed terms in the heap. This way, binaries, which are immutable objects, can be shared between processes without incurring the time and space cost of a large data copy.

In order to manage the memory associated with such binaries, AtomVM tracks references to these off-heap binaries via the "Mark and Sweep Object" list, a list that keeps track of which boxed terms in the process heap have a reference to an off-heap binary. When such a boxed term is copied (e.g., from a heap to a mailbox on a `send`, or from a mailbox to a heap on a `receive`), the reference count is incremented on the off-heap binary.

The MSO list is formed via the cons cells that are appended to reference counted binary boxed terms in the process heap. The list is initially empty (nil), but as reference counted binaries are added to the process heap, they are pre-pended to the MSO list for the process (on the mailbox message, as reference-counted binaries in the mailbox need to be managed, as well).

The following diagram illustrates a set of two reference counted binaries in a process heap:

```
        |                   |
+-----> +-------------------+
|       |       refc        |
|       |      binary       |
|       |                   |
|       |                   |
|       +-------------------+ <----+
|       |        nil        |      |
|       +-------------------+      |
+---------------< car       |      |
        +-------------------+      |
        |                   |      |
                ...
        |                   |      |
+-----> +-------------------+      |
|       |       refc        |      |
|       |      binary       |      |
|       |                   |      |
```

```
|       |                   |      |
|       +-------------------+ <----|-------+
|       |        cdr >-------------+       |
|       +-------------------+              |
+--------------< car        |              |
        +-------------------+              |
        |                   |              |
        |                   |              |
                 ...                       |
                                           |
                                   +-------^--------+
                                   |   mso_list     |
                                   +----------------+
```

After the new heap has been scanned and copied, as described above, the MSO list is traversed to determine if any reference-counted binaries are no longer referenced from the process heap. If any reference counted binaries in the heap have not been marked as moved from the old heap, they are, effectively, no longer referenced from the root set, and the reference count on the corresponding off-heap binary can be decremented. Furthermore, when the reference count reaches 0, the binaries can then be deleted.

> Note. Const binaries, while they have slots for entry into the MSO list, nonetheless are never "stitched" into the MSO list, as the binary data they point to is const, endures for the lifecycle of the program, and is never deleted. Match binaries, on the other hand, do count as references, and can therefore be stitched into the MSO list. However, when they are, the reference counted binaries they point to are the actual binaries in the process heap, not the match binaries, as with the case of refc binaries on the process heap.

Deletion

Once all terms have been copied from the old heap to the new heap, and once the MSO list has been swept for unreachable references, the old heap is simply discarded via the `free` function.

Chapter 10

Packbeam Format

AtomVM makes use of the packbeam format for aggregating beam and other file types into a single file that is used as the code base for an AtomVM application. Typically, on an embedded device, packbeam files are uploaded (e.g., via serial connection) to a specific location on flash media. The AtomVM runtime will locate the entrypoint into the application, and use the beam and other files flashed to the local media to run the uploaded application.

AtomVM provides a simple tool for generating packbeam files, but other tools have emerged for manipulation packbeam files using standard Erlang and Elixir tool chains, notably `Mix` and `rebar3`.

This document describes the packbeam format, so that both AtomVM and upstream/downstream tooling have a reference document on which to base implementations.

10.1 Overview

Packbeam files are binary-encoded aggregations of BEAM and plain data files. At a high level, a packbeam file consists of a packbeam header, followed by a sequence of files (beam or otherwise), each of which is prefixed with a header, including data about the file (name, size, flags, etc).

All binary integer values are 32-bit, in network order (big-endian). Headers and encoded files are padded when necessary and aligned on 4-byte boundaries.

At present, the AtomVM runtime treats data in packbeam files as *read-only* data. There is no support for modifying the contents on an AtomVM file by the runtime.

10.2 Packbeam Header

All AtomVM files begin with the packbeam header, a fixed 24-byte sequence of octets:

```
0x23, 0x21, 0x2f, 0x75,
0x73, 0x72, 0x2f, 0x62,
0x69, 0x6e, 0x2f, 0x65,
0x6e, 0x76, 0x20, 0x41,
0x74, 0x6f, 0x6d, 0x56,
0x4d, 0x0a, 0x00, 0x00
```

The ASCII encoding of this sequence is

```
#!/usr/bin/env AtomVM\n
```

followed by two nil (`0x00`) bytes.

The packbeam header is followed by a sequence of 0 or more encoded files. The number of files in a packbeam file is not indicated in the packbeam header; however, packbeam files do contain a special `end` file header, marking the end of the sequence of encoded files.

10.3 File encodings

Each embedded file in a packbeam file contains a file header, followed by the file contents.

10.3.1 File Header

The file header consists of the following 4 fields:

- `size` (32 bit, big-endian)
- `flags` (32-bit, big endian)
- `reserved` (32-bit, big-endian, currently unused)
- `module_name` (null-terminated sequence of bytes)

The `size` field indicates the size (in bytes) of the encoded file following the header. This size includes the file content length, in addition to any padding that may have been added to the file, in order for it to align on a 4-byte boundary.

Currently, the two low-order bits of the `flags` field are used. `0x02` indicates the file is a BEAM file, and `0x01` indicates that the file contains a `start/0` function, and is therefore suitable as an entry-point to start code execution.

> When AtomVM starts, it will scan the BEAM files in the AtomVM file, from start to finish, with which it is initialized to find the entrypoint to start code execution. It will start execution on the first BEAM file with a `start/0` function, i.e., whose flags mask against `0x03`. It is conventional, but not required, for the first file in an AtomVM file to be a BEAM file that has a `start/0` entrypoint.

The `reserved` field is currently unused.

The `module_name` is variable length, null terminated sequence of characters. Because the module name is variable-length, the header may be padded with null characters (`0x00`), in order to align the start of the file contents on a 4-byte boundary.

10.3.2 Example

The following BEAM header indicates a BEAM file with a length of 308 bytes (`0x00000134`), with a `start/0` entrypoint (`0x00000003`), and named `mylib.beam` (`0x6D796C69 622E6265 616D00`). The header has a 1-byte padding of null (`0x00`) characters.

```
00000134 00000003 00000000 6D796C69 622E6265 616D0000
```

10.3.3 BEAM files

BEAM files obey IFF encoding as detailed here, but certain information in BEAM files is stripped out in order to minimize the amount of data stored on flash.

The following BEAM chunks are included in BEAM files:

- `AtU8`
- `Code`
- `ExpT`
- `LocT`
- `ImpT`
- `LitU`
- `FunT`

- `StrT`
- `LitT`

Any other chunks are stripped out of the BEAM files before insertion into AVM files.

In addition, data in the literals table (`LitT`) are uncompressed before insertion into AVM files, as the AtomVM runtime does not include support for `zlib` decompression.

BEAM files may be padded at the end with a sequence of 1-3 null (`0x00`) characters, in order to align on 4-byte boundaries.

> Note. The `module_name` field in the file header will only contain the "base" name of the BEAM file, i.e., the file name stripped of any path information.

10.3.4 Normal Files

Normal files (e.g., text files, data files, etc.) can be stored in packbeam AVM files, as well as BEAM files. For example, a normal file might contain static configuration information, or data that is interpreted at runtime.

Normal files contain a 32-bit big-endian size prefix, indicating the size of the file data (without padding). Note that the `size` field in the file header includes the size of the data with padding, if applicable.

The AtomVM runtime provides access to data files via the `atomvm:read_priv/2` NIF. This function will create a path name formed by the `App` (atom) and `Path` (string) terms provided by this function, separated by `"/priv/"`. For example, the expression

```
atomvm:read_priv(mylib, "sample.txt")
```

yields a `binary` containing the contents of `mylib/priv/sample.txt`, if it exists, in the AtomVM packbeam file.

As a consequence, normal files should be included in packbeam files using module names that obey the above patterns.

> Note. Normal file names may encode virtual directory names, such as `mylib/priv/another/sample/text/file`. There is no requirement that the `Path` component of a normal file be a simple file name.

10.3.5 `end` file

Packbeam files end with a special `end` header. The `size` field of the `end` header is 0 bytes.

Example

The following sequence of bytes encodes the `end` header:

```
00000000 00000000 00000000 656E6400
```

Chapter 11

API Reference Documentation

11.1 Erlang Libraries

11.1.1 estdlib

The estdlib library

Modules

Module base64

- Description
- Function Index
- Section 9.4.2

An implementation of a subset of the Erlang/OTP base64 interface.

Description

This module is designed to be API-compatible with the Erlang/OTP base64 module, with the following exceptions:

- No support for decoding data with whitespace in base64 data
- No support for mime decoding functions

Function Index

Function Details

decode/1

`Data`: the data to decode

returns: the base-64 data decoded, as a binary

Base-64 decode a binary or string, outputting a binary.

This function will raise a badarg exception if the supplied data is not valid base64-encoded data.

decode_to_string/1

`Data`: the data to decode

returns: the base-64 data decoded, as a string

Base-64 decode a binary or string, outputting a string.

This function will raise a badarg exception if the supplied data is not valid base64-encoded data.

encode/1

`Data`: the data to encode

returns: the base-64 data encoded, as a binary

Base-64 encode a binary or string, outputting a binary.

encode_to_string/1

`Data`: the data to encode

returns: the base-64 data encoded, as a string

Base-64 encode a binary or string, outputting a string.

Module binary

- Description
- Function Index
- Section 9.4.2

An implementation of a subset of the Erlang/OTP binary interface.

Function Index

Function Details

at/2

`Binary`: binary to get a byte from`Index`: 0-based index of the byte to return

returns: value of the byte from the binary

Get a byte from a binary by index.

part/3

`Binary`: binary to extract a subbinary from`Pos`: 0-based index of the subbinary to extract`Len`: length, in bytes, of the subbinary to extract.

returns: a subbinary from Binary

Get the part of a given binary. A negative length can be passed to count bytes backwards.

split/2

`Binary`: binary to split`Pattern`: pattern to perform the split

returns: a list composed of one or two binaries

Split a binary according to pattern. If pattern is not found, returns a singleton list with the passed binary. Unlike Erlang/OTP, pattern must be a binary.

Module calendar

- Description
- Data Types
- Function Index
- Section 9.4.2

A partial implementation of the Erlang/OTP calendar functions.

Description

This module provides an implementation of a subset of the functionality of the Erlang/OTP calendar functions.

All dates conform to the Gregorian calendar. This calendar was introduced by Pope Gregory XIII in

1582 and was used in all Catholic countries from this year. Protestant parts of Germany and the Netherlands adopted it in 1698, England followed in 1752, and Russia in 1918 (the October revolution of 1917 took place in November according to the Gregorian calendar).

The Gregorian calendar in this module is extended back to year 0. For a given date, the gregorian day is the number of days up to and including the date specified.

Data Types

date()

datetime()

day()

day_of_week()

gregorian_days()

hour()

minute()

month()

second()

time()

year()

Function Index

Function Details

date_to_gregorian_days/1

`Date`: the date to get the gregorian day count of

returns: Days number of days

Equivalent to `date_to_gregorian_days(Year, M, D)`.

Year cannot be abbreviated.

For example, 93 denotes year 93, not 1993. The valid range depends on the underlying operating system. The date tuple must denote a valid date.

date_to_gregorian_days/3

`Year`: ending year`M`: ending month`D`: ending day

returns: Days number of days

Computes the number of gregorian days starting with year 0 and ending at the specified date.

datetime_to_gregorian_seconds/1

returns: Days number of days

Computes the number of gregorian days starting with year 0 and ending at the specified date.

day_of_the_week/1

`Date`: the date for which to retreive the weekday

returns: Weekday day of the week

Equivalent to `day_of_the_week(Y, M, D)`.

Computes the day of the week from the specified date tuple {Year, Month, Day}. Returns the day of the week as 1: Monday, 2: Tuesday, and so on.

day_of_the_week/3

`Y`: year of the desired day`M`: month of the desired day`D`: year of the desired day

returns: Weekday day of the week

Computes the day of the week from the specified Year, Month, and Day. Returns the day of the week as 1: Monday, 2: Tuesday, and so on.

system_time_to_universal_time/2

`Time`: the time, as an integer, in the specified unit`TimeUnit`: the time unit

Convert an integer time value to a date and time in UTC.

Module code

- Description
- Function Index
- Section 9.4.2

An implementation of a subset of the Erlang/OTP code interface.

Function Index

Function Details

load_abs/1

`Filename`: path to the beam to open, without .beams suffix

returns: A tuple with the name of the module

Load a module from a path. Error return result type is different from Erlang/OTP.

load_binary/3

`Module`: name of the module to load`Filename`: path to the beam (unused)`Binary`: binary of the module to load

returns: A tuple with the name of the module

Load a module from a binary. Error return result type is different from Erlang/OTP. Also unlike Erlang/OTP, no check is performed to verify that `Module` matches the name of the loaded module.

Module crypto

- Data Types
- Function Index
- Section 9.4.2

Data Types

cipher_iv()

cipher_no_iv()

crypto_opt()

crypto_opts()

digest()

hash_algorithm()

padding()

Function Index

Function Details

crypto_one_time/4

`Cipher`: a supported cipher`Key`: the encryption / decryption key`Data`: to be crypted or encrypted`FlagOrOptions`: either just true for encryption (or false for decryption), or a proplist for any additional option

returns: Returns crypted or encrypted data.

Encrypted/decrypt data using given cipher and key

crypto_one_time/5

`Cipher`: a supported cipher that makes use of IV`Key`: the encryption / decryption key`IV`: an initialization vector`Data`: to be crypted or encrypted`FlagOrOptions`: either just true for encryption (or false for decryption), or a proplist for any additional option such as padding.

returns: Returns crypted or encrypted data.

Encrypted/decrypt data using given cipher, key, IV.

hash/2

`Type`: the hash algorithm`Data`: the data to hash

returns: Returns the result of hashing the supplied data using the supplied hash algorithm.

Hash data using a specified hash algorithm.

Module erlang

- Description
- Data Types
- Function Index
- Section 9.4.2

An implementation of the Erlang/OTP erlang module, for functions that are not already defined as NIFs.

Data Types

demonitor_option()

float_format_option()

heap_growth_strategy()

mem_type()

spawn_option()

time_unit()

timestamp()

Function Index

Function Details

apply/3

`Module`: Name of module`Function`: Exported function name`Args`: Parameters to pass to function (max 6)

returns: Returns the result of Module:Function(Args).

Returns the result of applying Function in Module to Args. The applied function must be exported from Module. The arity of the function is the length of Args. Example:

```
> apply(lists, reverse, [[a, b, c]]).
[c,b,a]
> apply(erlang, atom_to_list, ['AtomVM']).
"AtomVM"
```

If the number of arguments are known at compile time, the call is better written as Module:Function(Arg1, Arg2, …, ArgN).

atom_to_binary/2

`Atom`: Atom to convert`Encoding`: Encoding for conversion

returns: a binary with the atom's name

Convert an atom to a binary. Only latin1 encoding is supported.

atom_to_list/1

`Atom`: Atom to convert

returns: a string with the atom's name

Convert an atom to a string.

binary_to_atom/2

`Binary`: Binary to convert to atom`Encoding`: encoding for conversion

returns: an atom from passed binary

Convert a binary to atom. Only latin1 encoded is supported.

binary_to_integer/1

`Binary`: Binary to parse for integer

returns: the integer represented by the binary

Parse the text in a given binary as an integer.

binary_to_list/1

`Binary`: Binary to convert to list

returns: a list of bytes from the binary

Convert a binary to a list of bytes.

binary_to_term/1

`Binary`: binary to decode

returns: A term decoded from passed binary

Decode a term that was previously encodes with `term_to_binary/1` This function should be mostly compatible with its Erlang/OTP counterpart. Unlike modern Erlang/OTP, resources are currently serialized as empty binaries and cannot be unserialized.

demonitor/1

`Monitor`: reference of monitor to remove

returns: `true`

Remove a monitor

demonitor/2

`Monitor`: reference of monitor to remove

returns: `true`

Remove a monitor, with options. If `flush`, monitor messages are flushed and guaranteed to not be received. If `info`, return `true` if monitor was removed, `false` if it was not found. If both options are provivded, return `false` if flush was needed.

display/1

`Term`: term to print

returns: `true`

Print a term to stdout.

erase/1

`Key`: key to erase from the process dictionary

returns: the previous value associated with this key or undefined

Erase a key from the process dictionary.

exit/1

`Reason`: reason for exit

Raises an exception of class `exit` with reason `Reason`. The exception can be caught. If it is not, the process exits. If the exception is not caught the signal is sent to linked processes. In this case, if `Reason` is `kill`, it is not transformed into `killed` and linked processes can trap it (unlike `exit/2`).

exit/2

`Process`: target process`Reason`: reason for exit

returns: `true`

Send an exit signal to target process. The consequences of the exit signal depends on `Reason`, on whether `Process` is self() or another process and whether target process is trapping exit. If `Reason` is not `kill` nor `normal`:

- If target process is not trapping exits, it exits with `Reason`
- If traget process is trapping exits, it receives a message `{'EXIT', From, Reason}` where `From` is the caller of `exit/2`.

If `Reason` is `kill`, the target process exits with `Reason` changed to `killed`. If `Reason` is `normal` and `Process` is not `self()`:

- If target process is not trapping exits, nothing happens.
- If traget process is trapping exits, it receives a message `{'EXIT', From, normal}` where `From` is the caller of `exit/2`.

If `Reason` is `normal` and `Process` is `self()`:

- If target process is not trapping exits, it exits with `normal`.
- If traget process is trapping exits, it receives a message `{'EXIT', From, normal}` where `From` is the caller of `exit/2`.

float_to_binary/1

`Float`: Float to convert

returns: a binary with a text representation of the float

Convert a float to a binary.

float_to_binary/2

`Float`: Float to convert`Options`: Options for conversion

returns: a binary with a text representation of the float

Convert a float to a binary.

float_to_list/1

`Float`: Float to convert

returns: a string with a text representation of the float

Convert a float to a string.

float_to_list/2

`Float`: Float to convert`Options`: Options for conversion

returns: a string with a text representation of the float

Convert a float to a string.

fun_to_list/1

`Fun`: function to convert to a string

returns: a string representation of the function

Create a string representing a function.

function_exported/3

`Module`: module to test`Function`: function to test`Arity`: arity to test

returns: `true` if Module exports a Function with this Arity

Determine if a function is exported

garbage_collect/0

returns: `true`

Run a garbage collect in current process

garbage_collect/1

`Pid`: pid of the process to garbage collect

returns: `true` or `false` if the process no longer exists

Run a garbage collect in a given process. The function returns before the garbage collect actually happens.

get/1

`Key`: key in the process dictionary

returns: value associated with this key or undefined

Return a value associated with a given key in the process dictionary

get_module_info/1

`Module`: module to get info for

returns: A list of module info tuples

Get info for a given module. This function is not meant to be called directly but through `Module:module_info/0` exported function.

get_module_info/2

`Module`: module to get info for`InfoKey`: info to get

returns: A term representing info for given module

Get specific info for a given module. This function is not meant to be called directly but through `Module:module_info/1` exported function. Supported info keys are `module`, `exports`, `compile` and `attributes`.

group_leader/0

returns: Pid of group leader or self() if no group leader is set.

Return the pid of the group leader of caller.

group_leader/2

`Leader`: pid of process to set as leader`Pid`: pid of process to set a Leader

returns: `true`

Set the group leader for a given process.

integer_to_binary/1

`Integer`: integer to convert to a binary

returns: a binary with a text representation of the integer

Convert an integer to a binary.

integer_to_binary/2

`Integer`: integer to convert to a binary`Base`: base for representation

returns: a binary with a text representation of the integer

Convert an integer to a binary.

integer_to_list/1

`Integer`: integer to convert to a string

returns: a string representation of the integer

Convert an integer to a string.

integer_to_list/2

`Integer`: integer to convert to a string`Base`: base for representation

returns: a string representation of the integer

Convert an integer to a string.

iolist_to_binary/1

`IOList`: IO list to convert to binary

returns: a binary with the bytes of the IO list

Convert an IO list to binary.

is_map/1

`Map`: the map to test

returns: `true` if `Map` is a map; `false`, otherwise.

Return `true` if `Map` is a map; `false`, otherwise.

This function may be used in a guard expression.

is_process_alive/1

`Pid`: pid of the process to test

returns: `true` if the process is alive, `false` otherwise

Determine if a process is alive

link/1

`Pid`: process to link to

returns: `true`

Link current process with a given process.

list_to_atom/1

`String`: string to convert to an atom

returns: an atom from the string

Convert a string into an atom. Unlike Erlang/OTP 20+, atoms are limited to ISO-8859-1 characters. The VM currently aborts if passed unicode characters. Atoms are also limited to 255 characters. Errors with system_limit_atom if the passed string is longer.

See also: list_to_existing_atom/1.

list_to_binary/1

`IOList`: iolist to convert to binary

returns: a binary composed of bytes and binaries from the list

Convert a list into a binary. Errors with `badarg` if the list is not an iolist.

list_to_existing_atom/1

`String`: string to convert to an atom

returns: an atom from the string

Convert a string into an atom. This function will error with badarg if the atom does not exist

See also: list_to_atom/1.

list_to_integer/1

`String`: string to convert to integer

returns: an integer value from its string representation

Convert a string (list of characters) to integer. Errors with `badarg` if the string is not a representation of an integer.

list_to_tuple/1

`List`: list to convert to tuple

returns: a tuple with elements of the list

Convert a list to a tuple with the same size.

localtime/0

returns: A tuple representing the current local time.

Return the current time and day for system local timezone.

See also: universaltime/0.

make_ref/0

returns: a new reference

Create a new reference

map_get/2

`Key`: the key to get`Map`: the map from which to get the value

returns: the value in `Map` associated with `Key`, if it exists.

throws `{badkey, Key} | {badmap, Map}`

Get the value in `Map` associated with `Key`, if it exists.

This function throws a `{badkey, Key}` exception if 'Key' does not occur in `Map` or a `{badmap, Map}` if `Map` is not a map.

This function may be used in a guard expression.

map_is_key/2

`Key`: the key`Map`: the map

returns: `true` if `Key` is associated with a value in `Map`; `false`, otherwise.

throws `{badmap, Map}`

Return `true` if `Key` is associated with a value in `Map`; `false`, otherwise.

This function throws a `{badmap, Map}` exception if `Map` is not a map.

This function may be used in a guard expression.

map_size/1

`Map`: the map

returns: the size of the map

throws `{badmap, Map}`

Returns the size of (i.e., the number of entries in) the map

This function throws a `{badmap, Map}` exception if `Map` is not a map.

This function may be used in a guard expression.

max/2

`A`: any term`B`: any term

returns: `A` if `A > B`; `B`, otherwise.

Return the maximum value of two terms

Terms are compared using > and follow the ordering principles defined in https://www.erlang.org/-doc/reference_manual/expressions.html#term-comparisons

md5/1

`Data`: data to compute hash of, as a binary.

returns: the md5 hash of the input Data, as a 16-byte binary.

Computes the MD5 hash of an input binary, as defined by https://www.ietf.org/rfc/rfc1321.txt

memory/1

`Type`: the type of memory to request

returns: the amount of memory (in bytes) used of the specified type

Return the amount of memory (in bytes) used of the specified type

min/2

`A`: any term`B`: any term

returns: `A` if `A < B`; `B`, otherwise.

Return the minimum value of two terms

Terms are compared using < and follow the ordering principles defined in https://www.erlang.org/-doc/reference_manual/expressions.html#term-comparisons

monitor/2

`Type`: type of monitor to create`Pid`: pid of the object to monitor

returns: a monitor reference

Create a monitor on a process or on a port. When the process or the port terminates, the following message is sent to the caller of this function:

```
{'DOWN', MonitorRef, Type, Pid, Reason}
```

Unlike Erlang/OTP, monitors are only supported for processes and ports.

monotonic_time/1

`Unit`: time unit

returns: monotonic time in the specified units

Return the monotonic time in the specified units.

Monotonic time varies from system to system, and should not be used to determine, for example the wall clock time.

Instead, monotonic time should be used to compute time differences, where the function is guaranteed to return a (not necessarily strictly) monotonically increasing value.

For example, on ESP32 system, monotonic time is reported as the difference from the current time and the time the ESP32 device was started, whereas on UNIX systems the value may vary among UNIX systems (e.g., Linux, macOS, FreeBSD).

open_port/2

`PortName`: Tuple {spawn, Name} identifying the port`Options`: Options, meaningful for the port

returns: A pid identifying the open port

Open a port. Unlike Erlang/OTP, ports are identified by pids.

pid_to_list/1

`Pid`: pid to convert to a string

returns: a string representation of the pid

Create a string representing a pid.

process_flag/2

`Flag`: flag to change`Value`: new value of the flag

returns: Previous value of the flag

Set a flag for the current process. When `trap_exit` is true, exit signals are converted to messages

```
{'EXIT', From, Reason}
```

and the process does not exit if `Reason` is not `normal`.

process_info/2

`Pid`: the process pid.`Key`: key used to find process information.

`Pid`: the process pid.`Key`: key used to find process information.

`Pid`: the process pid.`Key`: key used to find process information.

`Pid`: the process pid.`Key`: key used to find process information.

`Pid`: the process pid.`Key`: key used to find process information.

`Pid`: the process pid.`Key`: key used to find process information.

returns: process information for the specified pid defined by the specified key.

Return process information.

This function returns information about the specified process. The type of information returned is dependent on the specified key.

The following keys are supported:

- **heap_size** the number of words used in the heap (integer), including the stack but excluding fragments
- **total_heap_size** the number of words used in the heap (integer) including fragments
- **stack_size** the number of words used in the stack (integer)
- **message_queue_len** the number of messages enqueued for the process (integer)
- **memory** the estimated total number of bytes in use by the process (integer)
- **links** the list of linked processes

Specifying an unsupported term or atom raises a bad_arg error.

processes/0

returns: A list of pids of all processes

Return a list of all current processes. Compared to Erlang/OTP, this function also returns native processes (ports).

put/2

`Key`: key to add to the process dictionary`Value`: value to store in the process dictionary

returns: the previous value associated with this key or undefined

Store a value with a given key in the process dictionary.

ref_to_list/1

`Ref`: reference to convert to a string

returns: a string representation of the reference

Create a string representing a reference.

register/2

`Name`: name of the process to register`Pid`: pid of the process to register

returns: `true`

Register a name for a given process. Processes can be registered with several names. Unlike Erlang/OTP, ports are not distinguished from processes. Errors with `badarg` if the name is already registered.

send/2

`Pid`: process to send the message to`Message`: message to send

returns: the sent message

Send a message to a given process

send_after/3

`Time`: time in milliseconds after which to send the message.`Dest`: Pid or server name to which to send the message.`Msg`: Message to send to Dest after Time ms.

returns: a reference that can be used to cancel the timer, if desired.

Send Msg to Dest after Time ms.

spawn/1

`Function`: function to create a process from

returns: pid of the new process

Create a new process

spawn/3

`Module`: module of the function to create a process from`Function`: name of the function to create a process from`Args`: arguments to pass to the function to create a process from

returns: pid of the new process

Create a new process by calling exported Function from Module with Args.

spawn_link/1

`Function`: function to create a process from

returns: pid of the new process

Create a new process and link it.

spawn_link/3

`Module`: module of the function to create a process from`Function`: name of the function to create a process from`Args`: arguments to pass to the function to create a process from

returns: pid of the new process

Create a new process by calling exported Function from Module with Args and link it.

spawn_opt/2

`Function`: function to create a process from`Options`: additional options.

returns: pid of the new process

Create a new process.

spawn_opt/4

`Module`: module of the function to create a process from`Function`: name of the function to create a process from`Args`: arguments to pass to the function to create a process from`Options`: additional options.

returns: pid of the new process

Create a new process by calling exported Function from Module with Args.

start_timer/3

`Time`: time in milliseconds after which to send the timeout message.`Dest`: Pid or server name to which to send the timeout message.`Msg`: Message to send to Dest after Time ms.

returns: a reference that can be used to cancel the timer, if desired.

Start a timer, and send {timeout, TimerRef, Msg} to Dest after Time ms, where TimerRef is the reference returned from this function.

system_flag/2

`Key`: key used to change system flag.`Value`: value to change

returns: previous value of the flag.

Update system flags.

This function allows to modify system flags at runtime.

The following key is supported on SMP builds:

- **schedulers_online** the number of schedulers online

Specifying an unsupported atom key will result in a bad_arg error. Specifying a term that is not an atom will result in a bad_arg error.

system_info/1

`Key`: key used to find system information.

returns: system information defined by the specified key.

Return system information.

This function returns information about the system on which AtomVM is running. The type of information returned is dependent on the specified key.

The following keys are supported on all platforms:

- **process_count** the number of processes running in the node (integer)
- **port_count** the number of ports running in the node (integer)
- **atom_count** the number of atoms currently allocated (integer)
- **system_architecture** the processor and OS architecture (binary)
- **version** the version of the AtomVM executable image (binary)
- **wordsize** the number of bytes in a machine word on the current platform (integer)
- **schedulers** the number of schedulers, equal to the number of online processors (integer)
- **schedulers_online** the current number of schedulers (integer)

The following keys are supported on the ESP32 platform:

- **esp32_free_heap_size** the number of (noncontiguous) free bytes in the ESP32 heap (integer)
- **esp_largest_free_block** the number of the largest contiguous free bytes in the ESP32 heap (inte-

ger)(integer)

- **esp_get_minimum_free_size** the smallest number of free bytes in the ESP32 heap since boot (integer)

Additional keys may be supported on some platforms that are not documented here.

Specifying an unsupported atom key will results in returning the atom 'undefined'.

Specifying a term that is not an atom will result in a bad_arg error.

system_time/1

`Unit`: Unit to return system time in

returns: An integer representing system time

Get the current system time in provided unit.

term_to_binary/1

`Term`: term to encode

returns: A binary encoding passed term.

Encode a term to a binary that can later be decoded with `binary_to_term/1`. This function should be mostly compatible with its Erlang/OTP counterpart. Unlike modern Erlang/OTP, resources are currently serialized as empty binaries.

timestamp/0

returns: A tuple representing the current timestamp.

Return the timestamp in `{MegaSec, Sec, MicroSec}` format. This the old format returned by `erlang:now/0`. Please note that the latter which is deprecated in Erlang/OTP is not implemented by AtomVM.

See also: monotonic_time/1, system_time/1.

universaltime/0

returns: A tuple representing the current universal time.

Return the current time and day for UTC.

See also: localtime/0.

unlink/1

`Pid`: process to unlink from

returns: `true`

Unlink current process from a given process.

unregister/1

`Name`: name to unregister

returns: `true`

Lookup a process by name. Unlike Erlang/OTP, ports are not distinguished from processes. Errors with `badarg` if the name is not registered.

whereis/1

`Name`: name of the process to locate

returns: `undefined` or the pid of the registered process

Lookup a process by name.

Module erts_debug

- Description

- Function Index
- Section 9.4.2

An implementation of a subset of the Erlang/OTP erts_debug interface.

Function Index

Function Details

flat_size/1

`Term`: term to get the size of

returns: A size

Return the size, in terms, of a given term.

Module gen_event

- Function Index
- Section 9.4.2

Function Index

Function Details

add_handler/3

```
add_handler(EventMgrRef, Handler, Args) -> any()
```

delete_handler/3

```
delete_handler(EventMgrRef, Handler, Args) -> any()
```

notify/2

```
notify(EventMgrRef, Event) -> any()
```

start/0

```
start() -> any()
```

start/2

```
start(EventMgrName, Options) -> any()
```

start_link/0

```
start_link() -> any()
```

start_link/2

```
start_link(EventMgrName, Options) -> any()
```

stop/1

```
stop(EventManagerRef) -> any()
```

sync_notify/2

```
sync_notify(EventMgrRef, Event) -> any()
```

Module gen_server

- Description
- Data Types
- Function Index
- Section 9.4.2

An implementation of the Erlang/OTP gen_server interface.

This module defines the `gen_server` behaviour. Required callback functions: `init/1`, `handle_call/3`, `handle_cast/2`.

Description

This module implements a strict subset of the Erlang/OTP gen_server interface, supporting operations for local creation and management of gen_server instances.

This module is designed to be API-compatible with gen_server, with exceptions noted below.

Caveats:

- Support only for locally named procs
- No support for abcast
- No support for enter_loop
- No support for format_status
- No support for multi_call

Data Types

from()

options()

server_ref()

Function Index

Function Details

call/2

Equivalent to `call(ServerRef, Request, 5000)`.

Send a request to a gen_server instance, and wait for a reply.

call/3

`ServerRef`: a reference to the gen_server acquired via start`Request`: the request to send to the gen_server`TimeoutMs`: the amount of time in milliseconds to wait for a reply

returns: the reply sent back from the gen_server; {error, Reason}, otherwise.

Send a request to a gen_server instance, and wait for a reply.

This function will send the specified request to the specified gen_server instance, and wait at least Timeout milliseconds for a reply from the gen_server.

cast/2

`ServerRef`: a reference to the gen_server acquired via start`Request`: the request to send to the gen_server

returns: ok | {error, Reason}

Send a request to a gen_server instance.

This function will send the specified request to the specified gen_server instance, but will not wait for a reply.

init_it/4

```
init_it(Starter, Module, Args, Options) -> any()
```

init_it/5

```
init_it(Starter, Name, Module, Args, Options) -> any()
```

reply/2

`From`: the client to whom to send the reply`Reply`: the reply to send to the client

returns: an arbitrary term, that should be ignored

Send a reply to a calling client.

This function will send the specified reply back to the specified gen_server client (e.g, via call/3). The return value of this function can be safely ignored.

start/3

`Module`: the module in which the gen_server callbacks are defined`Args`: the arguments to pass to the module's init callback`Options`: the options used to create the gen_server

returns: the gen_server pid, if successful; {error, Reason}, otherwise.

Start an un-named gen_server.

This function will start a gen_server instance.

Note. *The Options argument is currently ignored.*

start/4

`ServerName`: the name with which to register the gen_server`Module`: the module in which the gen_server callbacks are defined`Args`: the arguments to pass to the module's init callback`Options`: the options used to create the gen_server

returns: the gen_server pid, if successful; {error, Reason}, otherwise.

Start a named gen_server.

This function will start a gen_server instance and register the newly created process with the process registry. Subsequent calls may use the gen_server name, in lieu of the process id.

Note. *The Options argument is currently ignored.*

start_link/3

`Module`: the module in which the gen_server callbacks are defined`Args`: the arguments to pass to the module's init callback`Options`: the options used to create the gen_server

returns: the gen_server pid, if successful; {error, Reason}, otherwise.

Start and link an un-named gen_server.

This function will start a gen_server instance.

Note. *The Options argument is currently ignored.*

start_link/4

`ServerName`: the name with which to register the gen_server`Module`: the module in which the gen_server callbacks are defined`Args`: the arguments to pass to the module's init callback`Options`: the options used to create the gen_server

returns: the gen_server pid, if successful; {error, Reason}, otherwise.

Start and link a named gen_server.

This function will start a gen_server instance and register the newly created process with the process registry. Subsequent calls may use the gen_server name, in lieu of the process id.

Note. *The Options argument is currently ignored.*

stop/1

Equivalent to `stop(ServerRef, normal, infinity)`.

Stop a previously started gen_server instance.

stop/3

`ServerRef`: a reference to the gen_server acquired via start`Reason`: reason to be supplied to callback

function`Timeout`: ms to wait for successful stop

returns: ok, if the gen_server stopped; {error, Reason}, otherwise.

Stop a previously started gen_server instance.

This function will stop a gen_server instance, providing the supplied Reason to the gen_server's terminate/2 callback function. If the gen_server is named, then the gen_server name may be used to stop the gen_server.

Module gen_statem

- Description
- Data Types
- Function Index
- Section 9.4.2

An implementation of the Erlang/OTP gen_statem interface.

This module defines the `gen_statem` behaviour. Required callback functions: `init/1`, `callback_mode/0`.

Description

This module implements a strict susbset of the Erlang/OTP gen_statem interface, supporting operations for local creation and management of gen_statem instances.

This module is designed to be API-compatible with gen_statem, with exceptions noted below.

Caveats:

- No support for start_link
- Support only for locally named gen_statem instances
- Support only for state function event handlers
- No support for keep_state or repeat_state return values from Module:StateName/3 callbacks
- No support for postpone or hibernate state transition actions
- No support for state enter calls
- No support for multi_call

Data Types

options()

server_ref()

Function Index

Function Details

call/2

Equivalent to `call(ServerRef, Request, infinity)`.

Send a request to a gen_statem instance, and wait for a reply.

call/3

`ServerRef`: a reference to the gen_statem acquired via start`Request`: the request to send to the gen_statem`Timeout`: the amount of time in milliseconds to wait for a reply

returns: the reply sent back from the gen_statem; {error, Reason}, otherwise.

Send a request to a gen_statem instance, and wait for a reply..

This function will send the specified request to the specified gen_statem instance, and wait at least Timeout milliseconds for a reply from the gen_statem.

cast/2

`ServerRef`: a reference to the gen_statem acquired via start`Request`: the request to send to the gen_statem

returns: ok | {error, Reason}

Send a request to a gen_statem instance.

This function will send the specified request to the specified gen_statem instance, but will not wait for a reply.

reply/2

`Client`: the client to whom to send the reply`Reply`: the reply to send to the client

returns: an arbitrary term, that should be ignored

Send a reply to a calling client.

This function will send the specified reply back to the specified gen_statem client (e.g, via call/3). The return value of this function can be safely ignored.

start/3

`Module`: the module in which the gen_statem callbacks are defined`Args`: the arguments to pass to the module's init callback`Options`: the options used to create the gen_statem

returns: the gen_statem pid, if successful; {error, Reason}, otherwise.

Start an un-named gen_statem.

This function will start a gen_statem instance.

Note. *The Options argument is currently ignored.*

start/4

`ServerName`: the name with which to register the gen_statem`Module`: the module in which the gen_statem callbacks are defined`Args`: the arguments to pass to the module's init callback`Options`: the options used to create the gen_statem

returns: the gen_statem pid, if successful; {error, Reason}, otherwise.

Start a named gen_statem.

This function will start a gen_statem instance and register the newly created process with the process registry. Subsequent calls may use the gen_statem name, in lieu of the process id.

Note. *The Options argument is currently ignored.*

start_link/3

`Module`: the module in which the gen_statem callbacks are defined`Args`: the arguments to pass to the module's init callback`Options`: the options used to create the gen_statem

returns: the gen_statem pid, if successful; {error, Reason}, otherwise.

Start an un-named gen_statem.

This function will start a gen_statem instance.

This version of the start function will link the started gen_statem process to the calling process.

Note. *The Options argument is currently ignored.*

start_link/4

`ServerName`: the name with which to register the gen_statem`Module`: the module in which the gen_statem callbacks are defined`Args`: the arguments to pass to the module's init callback`Options`: the options used to create the gen_statem

returns: the gen_statem pid, if successful; {error, Reason}, otherwise.

Start a named gen_statem.

This function will start a gen_statem instance and register the newly created process with the process registry. Subsequent calls may use the gen_statem name, in lieu of the process id.

This version of the start function will link the started gen_statem process to the calling process.

Note. *The Options argument is currently ignored.*

stop/1

Equivalent to `stop(ServerRef, normal, infinity)`.

Stop a previously started gen_statem.

stop/3

`ServerRef`: a reference to the gen_statem acquired via start

returns: ok, if the gen_statem stopped; {error, Reason}, otherwise.

Stop a previously started gen_statem instance.

This function will stop a gen_statem instance, providing the supplied Reason to the . If the gen_statem is a named gen_statem, then the gen_statem name may be used to stop the gen_statem.

Module gen_tcp

- Description
- Data Types
- Function Index
- Section 9.4.2

A partial implementation of the Erlang/OTP gen_tcp interface.

Description

This module provides an implementation of a subset of the functionality of the Erlang/OTP gen_tcp interface. It is designed to be API-compatible with gen_tcp, with exceptions noted below.

This interface may be used to send and receive TCP packets, as either binaries or strings. Active and passive modes are supported for receiving data.

Caveats:

- Limited support for socket tuning parameters
- No support for **controlling_process/2**

Note. *Port drivers for this interface are not supportedon all AtomVM platforms.*

Data Types

connect_option()

listen_option()

option()

packet()

reason()

Function Index

Function Details

accept/1

`ListenSocket`: the listening socket.

returns: a connection-based (tcp) socket that can be used for reading and writing

Accept a connection on a listening socket.

accept/2

`ListenSocket`: the listening socket.`Timeout`: amount of time in milliseconds to wait for a connection

returns: a connection-based (tcp) socket that can be used for reading and writing

Accept a connection on a listening socket.

close/1

`Socket`: the socket to close

returns: ok.

Close the socket.

connect/3

`Address`: the address to which to connect`Port`: the port to which to connect`Options`: options for controlling the behavior of the socket (see below)

returns: {ok, Socket} | {error, Reason}

Connect to a TCP endpoint on the specified address and port.

If successful, this function will return a Socket which can be used with the send/2 and recv/2 and recv/3 functions in this module.

The following options are supported:

- **active** Active mode (default: true)
- **buffer** Size of the receive buffer to use in active mode (default: 512)
- **binary** data is received as binaries (as opposed to lists)
- **list** data is received as lists (default)

If the socket is connected in active mode, then the calling process will receive messages of the form {tcp, Socket, Packet} when data is received on the socket. If active mode is set to false, then applications need to explicitly call one of the recv operations in order to receive data on the socket.

controlling_process/2

`Socket`: the socket to which to assign the pid`Pid`: Pid to which to send messages

returns: ok | {error, Reason}.

Assign a controlling process to the socket. The controlling process will receive messages from the socket.

This function will return `{error, not_owner}` if the calling process is not the current controlling process.

By default, the controlling process is the process associated with the creation of the Socket.

listen/2

`Port`: the port number on which to listen. Specify 0 to use an OS-assigned port number, which can then be retrieved via the inet:port/1 function.`Options`: A list of configuration parameters.

returns: a listening socket, which is appropriate for use in accept/1

Create a server-side listening socket.

This function is currently unimplemented

recv/2

Equivalent to `recv(Socket, Length, infinity)`.

Receive a packet over a TCP socket from a source address/port.

recv/3

`Socket`: the socket over which to receive a packet`Length`: the maximum length to read of the received packet`Timeout`: the amount of time to wait for a packet to arrive

returns: {ok, Packet} | {error, Reason}

Receive a packet over a TCP socket from a source address/port.

This function is used when the socket is not created in active mode. The received packet data returned from this call, and should be of length no greater than the specified length. This function will return {error, closed} if the server gracefully terminates the server side of the connection.

This call will block until data is received or a timeout occurs.

Note. *Currently, the Timeout parameter isignored.*

send/2

`Socket`: The Socket obtained via connect/3`Packet`: the data to send

returns: ok | {error, Reason}

Send data over the specified socket to a TCP endpoint.

If successful, this function will return the atom ok; otherwise, an error with a reason.

Module gen_udp

- Description
- Data Types
- Function Index
- Section 9.4.2

An implementation of the Erlang/OTP gen_udp interface.

Description

This module provides an implementation of a subset of the functionality of the Erlang/OTP gen_udp interface. It is designed to be API-compatible with gen_udp, with exceptions noted below.

This interface may be used to send and receive UDP packets, as either binaries or strings. Active and passive modes are supported for receiving data.

Caveats:

- Currently no support for IPv6
- Currently limited support for socket tuning parameters
- Currently no support for closing sockets

Note. *Port drivers for this interface are not supportedon all AtomVM platforms.*

Data Types

option()

packet()

reason()

Function Index

Function Details

close/1

`Socket`: the socket to close

returns: ok

Close the socket.

controlling_process/2

`Socket`: the socket to which to assign the pid`Pid`: Pid to which to send messages

returns: ok | {error, Reason}.

Assign a controlling process to the socket. The controlling process will receive messages from the socket.

This function will return `{error, not_owner}` if the calling process is not the current controlling process.

By default, the controlling process is the process associated with the creation of the Socket.

open/1

Equivalent to `open(PortNum, [])`.

Create a UDP socket. This function will instantiate a UDP socket that may be used to send or receive UDP messages.

open/2

`Options`: A list of configuration parameters.

returns: an opaque reference to the socket instance, used in subsequent commands.

throws `bad_arg`

Create a UDP socket. This function will instantiate a UDP socket that may be used to send or receive UDP messages. This function will raise an exception with the bad_arg atom if there is no socket driver supported for the target platform.

Note. *The Params argument is currently ignored.*

recv/2

Equivalent to `recv(Socket, Length, infinity)`.

Receive a packet over a UDP socket from a source address/port.

recv/3

`Socket`: the socket over which to receive a packet`Length`: the maximum length to read of the received packet`Timeout`: the amount of time to wait for a packet to arrive

returns: {ok, {Address, Port, Packet}} | {error, Reason}

Receive a packet over a UDP socket from a source address/port. The address and port of the received packet, as well as the received packet data, are returned from this call. This call will block until data is received or a timeout occurs.

Note. *Currently Length and Timeout parameters areignored.*

Note. *Currently the length of the received packetis limited to 128 bytes.*

send/4

`Socket`: the socket over which to send a packet`Address`: the target address to which to send the packet`Packet`: the packet of data to send

returns: ok | {error, Reason}

Send a packet over a UDP socket to a target address/port.

Note. *Currently only ipv4 addresses are supported.*

Module inet

- Data Types
- Function Index
- Section 9.4.2

Data Types

address()

hostname()

ipv4_address()

octet()

port_number()

socket()

Function Index

Function Details

close/1

`Socket`: the socket to close

returns: ok.

Close the socket.

peername/1

`Socket`: the socket

returns: The address and port of the remote end of an established connection.

The address and port representing the "remote" end of a connection. This function should be called on a running socket instance.

port/1

`Socket`: the socket from which to obtain the port number

returns: the port number associated with the local socket

Retrieve the actual port number to which the socket is bound. This function is useful if the port assignment is done by the operating system.

sockname/1

`Socket`: the socket

returns: The address and port of the local end of an established connection.

The address and port representing the "local" end of a connection. This function should be called on a running socket instance.

Module io

- Description
- Function Index
- Section 9.4.2

An implementation of the Erlang/OTP io interface.

Description

This module implements a strict subset of the Erlang/OTP io interface.

Function Index

Function Details

format/1

Equivalent to format(Format, []).

format/2

`Format`: format string`Args`: format argument

returns: string

Format string and data to console. See io_lib:format/2 for information about formatting capabilities.

get_line/1

`Prompt`: prompt for user input

returns: string

Read string from console with prompt.

put_chars/1

`Chars`: character(s) to write to console

returns: ok

Writes the given character(s) to the console.

Module io_lib

- Description
- Function Index
- Section 9.4.2

An implementation of the Erlang/OTP io_lib interface.

Description

This module implements a strict subset of the Erlang/OTP io_lib interface.

Function Index

Function Details

format/2

`Format`: format string`Args`: format argument

returns: string

Format string and data to a string. Approximates features of OTP io_lib:format/2, but only supports ~p and ~n format specifiers. Raises `badarg` error if the number of format specifiers does not match the length of the Args.

Module lists

- Description
- Function Index
- Section 9.4.2

An implementation of the Erlang/OTP lists interface.

Description

This module implements a strict susbset of the Erlang/OTP lists interface.

Function Index

Function Details

all/2

`Fun`: the predicate to evaluate`List`: the list over which to evaluate elements

returns: true if Fun(E) evaluates to true, for all elements in List

Evaluates to true iff Fun(E) =:= true, for all E in List

any/2

`Fun`: the predicate to evaluate`List`: the list over which to evaluate elements

returns: true if Fun(E) evaluates to true, for at least one in List

Evaluates to true iff Fun(E) =:= true, for some E in List

delete/2

`E`: the member to delete`L`: the list from which to delete the value

returns: the result of removing E from L, if it exists in L; otherwise, L.

Remove E from L

duplicate/2

`Count`: the number of times to duplicate the element`Elem`: the element to duplicate

returns: a list made of Elem duplicate Count times

Duplicate an element

filter/2

`Pred`: the predicate to apply to elements in List`List`: list

returns: all values in L for which Pred is true.

Filter a list by a predicate, returning the list of elements for which the predicate is true.

flatten/1

`L`: the list to flatten

returns: flattened list

recursively flattens elements of L into a single list

foldl/3

`Fun`: the function to apply`List`: the list over which to fold

returns: the result of folding Fun over L

Fold over a list of terms, from left to right, applying Fun(E, Accum) to each successive element in List

foldr/3

Equivalent to `foldl(Fun, Acc0, reverse(List))`.

Fold over a list of terms, from right to left, applying Fun(E, Accum) to each successive element in List

foreach/2

`Fun`: the predicate to evaluate`List`: the list over which to evaluate elements

returns: ok

Applies given fun to each list element

join/2

`Sep`: the separator`List`: list

returns: the result of inserting Sep between every element of List.

Inserts Sep between every element of List.

keydelete/3

`K`: the key to match`I`: the position in the tuple to compare (1..tuple_size)`L`: the list from which to delete the element

returns: the result of deleting any element in L who's Ith element matches K

Delete the entry in L whose Ith element matches K.

keyfind/3

`K`: the key to match`I`: the position in the tuple to compare (1..tuple_size)`L`: the list from which to find the element

returns: the tuple in L who's Ith element matches K; the atom false, otherwise

Find the entry in L whose Ith element matches K.

keymember/3

`K`: the key to match`I`: the position in the tuple to compare (1..tuple_size)`L`: the list from which to find the element

returns: true if there is a tuple in L who's Ith element matches K; the atom false, otherwise

Returns true if a Ith element matches K.

keyreplace/4

`K`: the key to match`I`: the position in the tuple to compare (1..tuple_size)`L`: the list from which to find the element`NewTuple`: tuple containing the new key to replace param `K`

returns: result of replacing the first element in L who's Ith element matches K with the contents of NewTuple.

Returns the result of replacing NewTuple for the first element in L with who's Ith element matches K.

map/2

`Fun`: the function to apply`List`: the list over which to map

returns: the result of mapping over L

Map a list of terms, applying Fun(E)

member/2

`E`: the member to search for`L`: the list from which to get the value

returns: true if E is a member of L; false, otherwise.

Determine whether a term is a member of a list.

nth/2

`N`: the index in the list to get`L`: the list from which to get the value

returns: the value in the list at position N.

Get the value in a list at position N.

Returns the value at the specified position in the list. The behavior of this function is undefined if N is outside of the {1..length(L)}.

reverse/1

`L`: the list to reverse

returns: the elements of L in reverse order

Reverse the elements of L.

search/2

`Pred`: the predicate to apply to elements in List`List`: search

returns: the first {value, Val}, if Pred(Val); false, otherwise.

If there is a Value in List such that Pred(Value) returns true, returns {value, Value} for the first such Value, otherwise returns false.

seq/2

`From`: from integer`To`: to Integer

returns: list of integers from [From..To]

Returns a sequence of integers in a specified range.

This function is equivalent to `lists:seq(From, To, 1)`.

seq/3

`seq(From, To, Incr) -> any()`

Returns a sequence of integers in a specified range incremented by a specified value.

sort/1

`List`: a list

returns: Sorted list, ordered by <

Returns a sorted list, using < operator to determine sort order.

sort/2

`Fun`: sort function`List`: a list

returns: Sorted list, ordered by Fun(A, B) : boolean() such that A "less than" B.

Returns a sorted list, using Fun(A, B) to determine sort order.

sublist/2

`List`: list to take the sublist from`Len`: the number of elements to get from List

returns: a list made of the first `Len` elements of `List`

Return a sublist made of the first `Len` elements of `List`. It is not an error for `Len` to be larger than the length of `List`.

usort/1

`List`: a list

returns: Sorted list with duplicates removed, ordered by <

Returns a unique, sorted list, using < operator to determine sort order.

See also: sort/1.

usort/2

`Fun`: sort function`List`: a list

returns: Sorted list with duplicates removed, ordered by Fun.

Returns a unique, sorted list.

See also: sort/2.

Module logger

- Description
- Data Types
- Function Index
- Section 9.4.2

A naive implementation of the Erlang/OTP logger interface.

Description

This module implements a strict subset of the Erlang/OTP logger interface, supporting operations for logging messages to various log handlers. A default handler (`logger_std_h`) supports logging to the console.

This module is designed to be API-compatible with the Erlang/OTP logger API, with exceptions noted below. Users can use macros defined in the Erlang/OTP `logger.hrl` header for logging

messages.

Limitations include but are not limited to:

- No support for logging filters
- No support for logging formatters
- No API support for logger configuration; all configuration must be done at initialization of the `logger_manager`
- No support for throttling or compacting sequences of repeated log messages

Data Types

level()

string_or_report()

Function Index

Function Details

alert/1

`StringOrReport`: string or report

returns: `ok`

Log a string at alert log level.

alert/2

`FormatOrReport`: format string or report`ArgsOrMeta`: format string arguments or metadata

returns: `ok`

Log a format string with args at alert log level.

alert/3

`Format`: format string`Args`: format string arguments`MetaData`: log metadata

returns: `ok`

Log a format string with args at alert log level with the specified metadata.

allow/2

`Level`: the log level`Module`: the module

returns: `true` if logging should be permitted at the specified level for the specified module; `false`, otherwise.

Determine whether logging should be permitted at the specified level for the specified module

compare/2

`Level1`: a level`Level2`: a level

returns: `lt | eq | gt`

Return comparison between levels

`lt` if `Level1 < Level2` `eq` if `Level1 == Level2` `gt` if `Level1 > Level2`

critical/1

`StringOrReport`: string or report

returns: `ok`

Log a string at critical log level.

critical/2

`FormatOrReport`: format string or report`ArgsOrMeta`: format string arguments or metadata

returns: `ok`

Log a format string with args at critical log level.

critical/3

`Format`: format string`Args`: format string arguments`MetaData`: log metadata

returns: `ok`

Log a format string with args at critical log level with the specified metadata.

debug/1

`StringOrReport`: string or report

returns: `ok`

Log a string at debug log level.

debug/2

`FormatOrReport`: format string or report`ArgsOrMeta`: format string arguments or metadata

returns: `ok`

Log a format string with args at debug log level.

debug/3

`Format`: format string`Args`: format string arguments`MetaData`: log metadata

returns: `ok`

Log a format string with args at debug log level with the specified metadata.

emergency/1

`StringOrReport`: string or report

returns: `ok`

Log a string at emergency log level.

emergency/2

`FormatOrReport`: format string or report`ArgsOrMeta`: format string arguments or metadata

returns: `ok`

Log a format string with args at emergency log level.

emergency/3

`Format`: format string`Args`: format string arguments`MetaData`: log metadata

returns: `ok`

Log a format string with args at emergency log level with the specified metadata.

error/1

`StringOrReport`: string or report

returns: `ok`

Log a string at error log level.

error/2

`FormatOrReport`: format string or report`ArgsOrMeta`: format string arguments or metadata

returns: `ok`

Log a format string with args at error log level.

error/3

`Format`: format string`Args`: format string arguments`MetaData`: log metadata

returns: `ok`

Log a format string with args at error log level with the specified metadata.

info/1

`StringOrReport`: string or report

returns: `ok`

Log a string at info log level.

info/2

`FormatOrReport`: format string or report`ArgsOrMeta`: format string arguments or metadata

returns: `ok`

Log a format string with args at info log level.

info/3

`Format`: format string`Args`: format string arguments`MetaData`: log metadata

returns: `ok`

Log a format string with args at info log level with the specified metadata.

log/2

`Level`: log level`StringOrReport`: string or report map

returns: `ok`

Log a string at the specified log level.

log/3

`Level`: log level`FormatOrReport`: format string or report`ArgsOrMeta`: format string arguments or metadata

returns: `ok`

Log a format string with args at the specified log level.

log/4

`Level`: log level`Format`: format string`Args`: format string arguments`Meta`: log metadata

returns: `ok`

Log a format string with args at the specified log level with the specified metadata.

notice/1

`StringOrReport`: string or report

returns: `ok`

Log a string at notice log level.

notice/2

`FormatOrReport`: format string or report`ArgsOrMeta`: format string arguments or metadata

returns: `ok`

Log a format string with args at notice log level.

notice/3

`Format`: format string`Args`: format string arguments`MetaData`: log metadata

returns: `ok`

Log a format string with args at notice log level with the specified metadata.

warning/1

`StringOrReport`: string or report

returns: `ok`

Log a string at warning log level.

warning/2

`FormatOrReport`: format string or report`ArgsOrMeta`: format string arguments or metadata

returns: `ok`

Log a format string with args at warning log level.

warning/3

`Format`: format string`Args`: format string arguments`MetaData`: log metadata

returns: `ok`

Log a format string with args at warning log level with the specified metadata.

Module maps

- Description
- Data Types
- Function Index
- Section 9.4.2

A *naive* implementation of the Erlang/OTP `maps` interface.

Description

The `maps` module provides several convenience operations for interfacing with the Erlang map type, which associates (unique) keys with values.

Note that the ordering of entries in a map is implementation-defined. While many operations in this module present entries in lexical order, users should in general make no assumptions about the ordering of entries in a map.

This module implements a subset of the Erlang/OTP `maps` interface. Some OTP functions are not implemented, and the approach favors correctness and readability over speed and performance.

Data Types

iterator()

key()

map_or_iterator()

value()

Function Index

Function Details

filter/2

`Pred`: a function used to filter entries from the map`MapOrIterator`: the map or map iterator to filter

returns: a map containing all elements in `MapOrIterator` that satisfy `Pred`

throws `{badmap, Map} | badarg`

Return a map who's entries are filtered by the supplied predicate.

This function returns a new map containing all elements from the input `MapOrIterator` that satisfy the input `Pred`.

The supplied predicate is a function from key-value inputs to a boolean value.

This function throws a `{badmap, Map}` exception if `Map` is not a map or map iterator, and a `badarg` exception if the input predicate is not a function.

find/2

`Key`: the key to find`Map`: the map in which to search

returns: `{ok, Value}` if `Key` is in `Map`; `error`, otherwise.

throws `{badmap, Map}`

Returns `{ok, Value}` if `Key` is in `Map`; `error`, otherwise.

This function throws a `{badmap, Map}` exception if `Map` is not a map.

fold/3

`Fun`: function over which to fold values`Init`: the initial value of the fold accumulator`MapOrIterator`: the map or map iterator over which to fold

returns: the result of folding over all elements of the supplied map.

throws `{badmap, Map} | badarg`

Fold over the entries in a map.

This function takes a function used to fold over all entries in a map and an initial accumulator value to use as the value supplied to the first entry in the map.

This function throws a `badmap` exception if `Map` is not a map or map iterator, and a `badarg` exception if the input function is not a function.

from_list/1

`List`: a list of `[{Key, Value}]` pairs

returns: the map containing the entries from the list of supplied key-value pairs.

throws `badarg`

This function constructs a map from the supplied list of key-value pairs.

If the input list contains duplicate keys, the returned map will contain the right-most entry.

This function will raise a `badarg` exception if the input is not a proper list or contains an element that is not a key-value pair.

get/2

`Key`: the key to get`Map`: the map from which to get the value

returns: the value in `Map` associated with `Key`, if it exists.

throws `{badkey, Key} | {badmap, Map}`

Get the value in `Map` associated with `Key`, if it exists.

This function throws a `{badkey, Key}` exception if 'Key' does not occur in `Map` or a `{badmap, Map}` if `Map` is not a map.

get/3

`Key`: the key`Map`: the map`Default`: default value

returns: the value in `Map` associated with `Key`, or `Default`, if the key is not associated with a value in `Map`.

throws `{badmap, Map}`

Get the value in `Map` associated with `Key`, or `Default`, if the key is not associated with a value in `Map`.

This function throws a `{badmap, Map}` exception if `Map` is not a map.

is_key/2

`Key`: the key`Map`: the map

returns: `true` if `Key` is associated with a value in `Map`; `false`, otherwise.

throws `{badmap, Map}`

Return `true` if `Key` is associated with a value in `Map`; `false`, otherwise.

This function throws a `{badmap, Map}` exception if `Map` is not a map.

iterator/1

`Map`: the map

returns: an iterator structure that can be used to iterate over associations in a map.

throws `{badmap, Map}`

Return an iterator structure that can be used to iterate over associations in a map.

In general, users should make no assumptions about the order in which entries appear in an iterator. The order of entries in a map is implementation-defined.

This function throws a `{badmap, Map}` exception if `Map` is not a map.

See also: `next/1`.

keys/1

`Map`: the map

returns: the list of keys that occur in this map.

throws `{badmap, Map}`

Returns the list of keys that occur in this map.

No guarantees are provided about the order of keys returned from this function.

This function throws a `{badmap, Map}` exception if `Map` is not a map.

map/2

`Fun`: the function to apply to every entry in the map`Map`: the map to which to apply the map function

returns: the result of applying `Fun` to every entry in `Map`

throws `{badmap, Map} | badarg`

Returns the result of applying a function to every element of a map.

This function throws a `badmap` exception if `Map` is not a map or map iterator, and a `badarg` exception if the input function is not a function.

merge/2

`Map1`: a map`Map2`: a mpa

returns: the result of merging entries from `Map1` and `Map2`.

throws `{badmap, Map}`

Merge two maps to yield a new map.

If `Map1` and `Map2` contain the same key, then the value from `Map2` will be used.

This function throws a `badmap` exception if neither `Map1` nor `Map2` is a map.

new/0

returns: a new map

Return a new (empty) map.

next/1

`Iterator`: a map iterator

returns: the key and value, along with the next iterator in the map, or the atom `none` if there are no more items over which to iterate.

throws `badarg`

Returns the next key and value in the map, along with a new iterator that can be used to iterate over the remainder of the map.

This function throws a `badarg` exception if the supplied iterator is not of the expected type. Only use iterators that are returned from functions in this module.

put/3

`Key`: the key`Value`: the value`Map`: the map

returns: A copy of `Map` containing the `{Key, Value}` association.

throws `{badmap, Map}`

Return the map containing the `{Key, Value}` association.

If `Key` occurs in `Map` then it will be over-written. Otherwise, the returned map will contain the new association.

This function throws a `{badmap, Map}` exception if `Map` is not a map.

remove/2

`Key`: the key to remove`MapOrIterator`: the map or map iterator from which to remove the key

returns: a new map without `Key` as an entry.

throws `{badmap, Map}`

Remove an entry from a map using a key.

If `Key` does not occur in `Map`, then the returned Map has the same entries as the input map or map iterator.

Note. This function extends the functionality of the OTP `remove/2` function, since the OTP interface only takes a map as input.

This function throws a `badmap` exception if `Map` is not a map or map iterator.

size/1

`Map`: the map

returns: the size of the map

throws `{badmap, Map}`

Returns the size of (i.e., the number of entries in) the map

This function throws a `{badmap, Map}` exception if `Map` is not a map.

to_list/1

`Map`: the map

returns: a list of `[{Key, Value}]` tuples

Return the list of entries, expressed as `{Key, Value}` pairs, in the supplied map.

No guarantees are provided about the order of entries returned from this function.

This function throws a `{badmap, Map}` exception if `Map` is not a map.

update/3

`Key`: the key to update`Value`: the value to update`Map`: the map to update

returns: a new map, with `Key` updated with `Value`

throws `{badmap, Map}`

Returns a new map with an updated key-value association.

This function throws a `badmap` exception if `Map` is not a map and `{badkey, Key}` if key doesn`t exist

values/1

`Map`: the map

returns: the list of values that occur in this map.

throws `{badmap, Map}`

Returns the list of values that occur in this map.

No guarantees are provided about the order of values returned from this function.

This function throws a `{badmap, Map}` exception if `Map` is not a map.

Module math

- Function Index
- Section 9.4.2

Function Index

Function Details

acos/1

acosh/1

asin/1

asinh/1

atan/1

atan2/2

atanh/1

ceil/1

cos/1

cosh/1

exp/1

floor/1

fmod/2

log/1

log10/1

log2/1

pi/0

pow/2

sin/1

sinh/1

sqrt/1

tan/1

tanh/1

Module proplists

- Description
- Data Types
- Function Index
- Section 9.4.2

An implementation of the Erlang/OTP proplists interface.

Description

This module implements a strict subset of the Erlang/OTP proplists interface.

Data Types

property()

Function Index

Function Details

get_value/2

Equivalent to `get_value(Key, List, undefined)`.

Get a value from a property list.

get_value/3

`Key`: the key with which to find the value`List`: the property list from which to get the value`Default`: the default value to return, if Key is not in the property list.

returns: the value in the property list under the key, or Default, if Key is not in List.

Get a value from a property list.

Returns the value under the specified key, or the specified Default, if the Key is not in the supplied List. If the Key corresponds to an entry in the property list that is just a single atom, this function returns the atom true.

Module string

- Description
- Function Index
- Section 9.4.2

An implementation of the Erlang/OTP string interface.

Description

This module implements a strict subset of the Erlang/OTP string interface.

Function Index

Function Details

split/2

`String`: a string to split`Pattern`: the search pattern to split at

returns: chardata

Equivalent to `split(String, Pattern, leading)`.

split/3

`String`: a string to split`Pattern`: the search pattern to split at`Where`: position to split (leading, trailing, or all)

returns: chardata

Splits String where SearchPattern is encountered and return the remaining parts.

Where, default leading, indicates whether the leading, the trailing or all encounters of SearchPattern will split String.

Example:

```
    0> string:split("ab..bc..cd", "..").
["ab","bc..cd"]
1> string:split(<<"ab..bc..cd">>, "..", trailing).
[<<"ab..bc">>,<<"cd">>]
2> string:split(<<"ab..bc....cd">>, "..", all).
[<<"ab">>,<<"bc">>,<<>>,<<"cd">>]
```

to_lower/1

`Input`: a string or character to convert

returns: a Character or string

Convert string or character to uppercase.

The specified string or character is case-converted. Notice that the supported character set is ISO/IEC 8859-1 (also called Latin 1); all values outside this set are unchanged

to_upper/1

`Input`: a string or character to convert

returns: a Character or string

Convert string or character to uppercase.

The specified string or character is case-converted. Notice that the supported character set is ISO/IEC 8859-1 (also called Latin 1); all values outside this set are unchanged

trim/1

`String`: a string or character to trim whitespace

returns: a Character or string

Equivalent to `trim(String, both)`.

trim/2

`String`: a string or character to trim`Direction`: an atom indicating the direction from which to remove whitespace

returns: a Character or string

Returns a string, where leading or trailing, or both, whitespace has been removed.

If omitted, Direction is both.

Example:

```
    1> string:trim("\t  Hello  \n").
"Hello"
2> string:trim(<<"\t  Hello  \n">>, leading).
<<"Hello  \n">>
3> string:trim(<<".Hello.\n">>, trailing, "\n.").
<<".Hello">>
```

Module supervisor

- Function Index
- Section 9.4.2

Function Index

Function Details

handle_call/3

```
handle_call(Msg, _from, State) -> any()
```

handle_cast/2

```
handle_cast(Msg, State) -> any()
```

handle_info/2

```
handle_info(Msg, State) -> any()
```

init/1

```
init(X1) -> any()
```

start_link/2

```
start_link(Module, Args) -> any()
```

start_link/3

```
start_link(SupName, Module, Args) -> any()
```

Module timer

- Description
- Function Index
- Section 9.4.2

An implementation of the Erlang/OTP timer interface.

Description

This module implements a strict subset of the Erlang/OTP timer interface.

Function Index

Function Details

sleep/1

`Timeout`: number of milliseconds to sleep or `infinity`

returns: `ok`

Pauses the execution of the current process for a given number of milliseconds, or forever, using `infinity` as the parameter.

Module unicode

- Description
- Data Types
- Function Index
- Section 9.4.2

An implementation of the Erlang/OTP unicode interface.

Description

This module implements a strict subset of the Erlang/OTP unicode interface.

Data Types

chardata()

charlist()

encoding()

latin1_chardata()
unicode_binary()

Function Index

Function Details

characters_to_binary/1

`Data`: data to convert to UTF8

returns: an utf8 binary or a tuple if conversion failed.

Equivalent to `characters_to_binary(Data, utf8, utf8)`.

Convert character data to an UTF8 binary

characters_to_binary/2

`Data`: data to convert to UTF8`InEncoding`: encoding of data

returns: an utf8 binary or a tuple if conversion failed.

Equivalent to `characters_to_binary(Data, InEncoding, utf8)`.

Convert character data in a given encoding to an UTF8 binary

characters_to_binary/3

`Data`: data to convert to UTF8`InEncoding`: output encoding

returns: an encoded binary or a tuple if conversion failed.

Convert character data in a given encoding to a binary in a given encoding.

If conversion fails, the function returns a tuple with three elements:

- First element is `error` or `incomplete`. `incomplete` means the conversion failed because of an incomplete unicode transform at the very end of data.
- Second element is what has been converted so far.
- Third element is the remaining data to be converted, for debugging purposes. This remaining data can differ with what Erlang/OTP returns.

Also, Erlang/OTP's implementation may error with `badarg` for parameters for which this function merely returns an error tuple.

characters_to_list/1

`Data`: data to convert to Unicode

returns: a list of characters or a tuple if conversion failed.

Convert UTF-8 data to a list of Unicode characters.

If conversion fails, the function returns a tuple with three elements:

- First element is `error` or `incomplete`. `incomplete` means the conversion failed because of an incomplete unicode transform at the very end of data.
- Second element is what has been converted so far.
- Third element is the remaining data to be converted, for debugging purposes. This remaining data can differ with what Erlang/OTP returns.

characters_to_list/2

`Data`: data to convert`Encoding`: encoding of data to convert

returns: a list of characters or a tuple if conversion failed.

Convert UTF-8 or Latin1 data to a list of Unicode characters. Following Erlang/OTP, if input encoding is latin1, this function returns an error tuple if a character > 255 is passed (in a list). Otherwise, it will

accept any character within Unicode range (0-0x10FFFF).

See also: characters_to_list/1.

11.1.2 eavmlib

The eavmlib library

Modules

Module atomvm

- Description
- Data Types
- Function Index
- Section 9.4.2

AtomVM-specific APIs.

Description

This module contains functions that are specific to the AtomVM platform.

Data Types

avm_path()

platform_name()

posix_error()

posix_fd()

abstract datatype: `posix_fd()`

posix_open_flag()

Function Index

Function Details

add_avm_pack_binary/2

`AVMData`: AVM data.`Options`: Options, as a property list.

returns: `ok`

Add code from an AVM binary to your application.

This function will add the data in the `AVMData` parameter to your application. The data is assumed to be valid AVM data (e.g, as generated by packbeam tooling).

Failure to properly load AVM data is result in a runtime `error`

add_avm_pack_file/2

`AVMPath`: Path to AVM data.`Options`: Options, as a property list.

returns: `ok`

Add code from an AVM binary to your application.

This function will add the data located in the `AVMPath` parameter to your application. The data is assumed to be valid AVM data (e.g, as generated by packbeam tooling).

On `generic_unix` platforms, the `AVMPath` may be a valid file system path to an AVM file.

On `esp32` platforms, the `AVMPath` should be the name of an ESP32 flash partition, prefixed with the string `/dev/partition/by-name/`. Thus, for example, if you specify `/dev/partition/by-name/main2.app` as the `AVMPath`, the ESP32 flash should contain a data partition with the name `main2.app`

Failure to properly load AVM path is result in a runtime `error`

close_avm_pack/2

`Options`: Options, as a property list.

returns: `ok | error`

Close previously opened AVM binary from your application.

This function will close the data referenced by the `Name` parameter from your application. The `Name` parameter must reference previously opened AVM data.

Failure to close AVM data is result in a runtime `error`

get_start_beam/1

`AVM`: Name of avm (atom)

returns: the name of the start module (with suffix)

Get the start beam for a given avm

platform/0

returns: The platform name.

Return the platform moniker. You may use this function to uniquely identify the platform type on which your application is running.

posix_close/1

`File`: Descriptor to a file to close

returns: `ok` or an error tuple

Close a file that was opened with `posix_open/2,3`

posix_open/2

`Path`: Path to the file to open`Flags`: List of flags passed to `open(3)`.

returns: A tuple with a file descriptor or an error tuple.

Open a file (on platforms that have `open(3)`). The file is automatically closed when the file descriptor is garbage collected.

Files are automatically opened with `O_NONBLOCK`. Other flags can be passed.

posix_open/3

`Path`: Path to the file to open`Flags`: List of flags passed to `open(3)`.`Mode`: Mode passed to `open(3)` for created file.

returns: A tuple with a file descriptor or an error tuple.

Open a file (on platforms that have `open(3)`). This variant can be used to specify the mode for new file.

posix_read/2

`File`: Descriptor to an open file`Count`: Maximum number of bytes to read

returns: a tuple with read bytes, `eof` or an error tuple

Read at most `Count` bytes from a file. Files are open non-blocking. `atomvm:posix_select_read/3' can open non-blocking. `atomvm:posix_select_read/3' can be used to determine if the file can be read. `eof` is returned if no more data can be read because the file cursor reached the end.

posix_write/2

`File`: Descriptor to an open file`Data`: Data to write

returns: a tuple with the number of written bytes or an error tuple

Write data to a file. Files are open non-blocking. `atomvm:posix_select_write/3' can be used to determine if the file can be written.

rand_bytes/1

`Len`: non-negative integer

returns: Binary containing random sequence of bytes of length Len.

Returns a binary containing random sequence of bytes of length Len. Supplying a negative value will result in a badarg error. This function will use a cryptographically strong RNG if available. Otherwise, the random value is generated using a PRNG.

random/0

returns: random 32-bit integer.

Returns a random 32-bit integer value. This function will use a cryptographically strong RNG if available. Otherwise, the random value is generated using a PRNG.

read_priv/2

`App`: application name.`Path`: path to the resource.

returns: Binary containing the resource content.

This function allows to fetch priv/ resources content.

Module avm_pubsub

- Function Index
- Section 9.4.2

Function Index

Function Details

handle_call/3

```
handle_call(X1, From, Table) -> any()
```

handle_info/2

```
handle_info(Info, Table) -> any()
```

init/1

```
init(X1) -> any()
```

pub/3

```
pub(PubSub, Topic, Term) -> any()
```

start/0

```
start() -> any()
```

start/1

```
start(LocalName) -> any()
```

sub/2

```
sub(PubSub, Topic) -> any()
```

sub/3

```
sub(PubSub, Topic, Pid) -> any()
```

terminate/2

```
terminate(Reason, State) -> any()
```

unsub/2

```
unsub(PubSub, Topic) -> any()
```

unsub/3

```
unsub(PubSub, Topic, Pid) -> any()
```

Module console

- Description
- Function Index
- Section 9.4.2

This modules supports output of string data to the console.

Function Index

Function Details

flush/0

returns: ok if the data was written, or {error, Reason}, if there was an error.

Flush any previously written data to the console.

print/1

`Text`: the data to write to the console

returns: ok if the data was written, or {error, Reason}, if there was an error.

Write a string to the console.

See also: erlang:display/1.

puts/1

`Text`: the string data to write to the console

returns: ok if the data was written, or {error, Reason}, if there was an error.

Write a string to the console.

Note. *This operation will only write string data.The output is not suffixed with a newline character or sequence.To print an erlang term, use erlang:display/1.*

See also: erlang:display/1.

Module emscripten

- Description
- Data Types
- Function Index
- Section 9.4.2

emscripten API.

Description

The functions in this module broadly reflect emscripten's API and obviously are only implemented for the emscripten platform.

See Emscripten's API documentation for more information about these APIs.

The counterpart of functions defined in this module are two main Javascript functions that can be used to send messages to Erlang.

```
Module.cast('some_proc', 'message')
await Module.call('some_proc', 'message')
```

These respectively send the following messages to Erlang process registered as `some_proc`:

```
{emscripten, {cast, <<"message">>}}
{emscripten, {call, Promise, <<"message">>}}
```

Promise should be passed to `promise_resolve/1,2` or `promise_reject/1,2` as documented below.

Data Types

focus_event()

html5_target()

keyboard_event()

listener_handle()

abstract datatype: `listener_handle()`

mouse_event()

promise()

abstract datatype: `promise()`

register_error_reason()

register_option()

register_options()

register_result()

run_script_opt()

touch_event()

touch_point()

ui_event()

wheel_event()

Function Index

Function Details

promise_reject/1

Equivalent to `promise_reject(_Promise, 0)`.

promise_reject/2

Reject a promise with a given result. This is similar to `promise_resolve` except the promise is rejected.

promise_resolve/1

Equivalent to `promise_resolve(_Promise, 0)`.

promise_resolve/2

Successfully resolve a promise with a given result. A promise is currently only obtained through synchronous calls using `Module.call()` javascript function. If Javascript calls:

```
await Module.call('some_proc', 'message')
```

and if an Erlang process is registered as `some_proc`, then the process will receive a message:

```
{emscripten, {call, Promise, <<"message">>}}`''
and the Javascript caller will wait until `promise_resolve` or `promise_reject`
is called. If the process doesn't exist, the promise will be rejected with
'no_proc'. Likewise if the Promise is garbage collected by the Erlang VM.

<a name="register_blur_callback-1"></a>

### register_blur_callback/1 ###
```

```
<pre><code>
register_blur_callback(Target::<a href="#type.md5_target">html5_target()</a>)
-&gt; <a href="#type-register_result">register_result()</a>
</code></pre>
<br />

Equivalent to [`register_blur_callback(_Target, [])`](#register_blur_callback-2).

<a name="register_blur_callback-2"></a>

### register_blur_callback/2 ###

<pre><code>
register_blur_callback(Target::<a href="#type.md5_target">html5_target()</a>,
Options::<a href="#type-register_options">register_options()</a>) -&gt; <a
href="#type-register_result">register_result()</a>
</code></pre>
<br />

Register for blur events.
Events are sent as:
```

{emscripten, {blur, focus_event()}}

```
__See also:__ [register_keypress_callback/2](#register_keypress_callback-2).

<a name="register_blur_callback-3"></a>

### register_blur_callback/3 ###

<pre><code>
register_blur_callback(Target::<a href="#type.md5_target">html5_target()</a>,
Options::<a href="#type-register_options">register_options()</a>, UserData::any())
-&gt; <a href="#type-register_result">register_result()</a>
</code></pre>
<br />

Register for blur events.
Events are sent as:
```

{emscripten, {blur, focus_event()}, UserData}

```
__See also:__ [register_keypress_callback/2](#register_keypress_callback-2).

<a name="register_click_callback-1"></a>

### register_click_callback/1 ###

<pre><code>
register_click_callback(Target::<a href="#type.md5_target">html5_target()</a>)
-&gt; <a href="#type-register_result">register_result()</a>
</code></pre>
<br />

Equivalent to [`register_click_callback(_Target,
[])`](#register_click_callback-2).

<a name="register_click_callback-2"></a>

### register_click_callback/2 ###
```

```
<pre><code>
register_click_callback(Target::<a href="#type.md5_target">html5_target()</a>,
Options::<a href="#type-register_options">register_options()</a>) -&gt; <a
href="#type-register_result">register_result()</a>
</code></pre>
<br />

Register for click events.
Events are sent as:
```

{emscripten, {click, mouse_event()}}

```
__See also:__ [register_keypress_callback/2](#register_keypress_callback-2).

<a name="register_click_callback-3"></a>

### register_click_callback/3 ###

<pre><code>
register_click_callback(Target::<a href="#type.md5_target">html5_target()</a>,
Options::<a href="#type-register_options">register_options()</a>, UserData::any())
-&gt; <a href="#type-register_result">register_result()</a>
</code></pre>
<br />

Register for click events.
Events are sent as:
```

{emscripten, {click, mouse_event()}, UserData}

```
__See also:__ [register_keypress_callback/2](#register_keypress_callback-2).

<a name="register_dblclick_callback-1"></a>

### register_dblclick_callback/1 ###

<pre><code>
register_dblclick_callback(Target::<a href="#type.md5_target">html5_target()</a>)
-&gt; <a href="#type-register_result">register_result()</a>
</code></pre>
<br />

Equivalent to [`register_dblclick_callback(_Target,
[])`](#register_dblclick_callback-2).

<a name="register_dblclick_callback-2"></a>

### register_dblclick_callback/2 ###

<pre><code>
register_dblclick_callback(Target::<a href="#type.md5_target">html5_target()</a>,
Options::<a href="#type-register_options">register_options()</a>) -&gt; <a
href="#type-register_result">register_result()</a>
</code></pre>
<br />

Register for dblclick events.

__See also:__ [register_click_callback/2](#register_click_callback-2).
```

```
<a name="register_dblclick_callback-3"></a>

### register_dblclick_callback/3 ###

<pre><code>
register_dblclick_callback(Target::<a href="#type.md5_target">html5_target()</a>,
Options::<a href="#type-register_options">register_options()</a>, UserData::any())
-&gt; <a href="#type-register_result">register_result()</a>
</code></pre>
<br />

Register for dblclick events.

__See also:__ [register_click_callback/2](#register_click_callback-2).

<a name="register_focus_callback-1"></a>

### register_focus_callback/1 ###

<pre><code>
register_focus_callback(Target::<a href="#type.md5_target">html5_target()</a>)
-&gt; <a href="#type-register_result">register_result()</a>
</code></pre>
<br />

Equivalent to [`register_focus_callback(_Target,
[])`](#register_focus_callback-2).

<a name="register_focus_callback-2"></a>

### register_focus_callback/2 ###

<pre><code>
register_focus_callback(Target::<a href="#type.md5_target">html5_target()</a>,
Options::<a href="#type-register_options">register_options()</a>) -&gt; <a
href="#type-register_result">register_result()</a>
</code></pre>
<br />

Register for focus events.

__See also:__ [register_blur_callback/2](#register_blur_callback-2).

<a name="register_focus_callback-3"></a>

### register_focus_callback/3 ###

<pre><code>
register_focus_callback(Target::<a href="#type.md5_target">html5_target()</a>,
Options::<a href="#type-register_options">register_options()</a>, UserData::any())
-&gt; <a href="#type-register_result">register_result()</a>
</code></pre>
<br />

Register for focus events.

__See also:__ [register_blur_callback/2](#register_blur_callback-2).

<a name="register_focusin_callback-1"></a>

### register_focusin_callback/1 ###

<pre><code>
register_focusin_callback(Target::<a href="#type.md5_target">html5_target()</a>)
```

```
-&gt; <a href="#type-register_result">register_result()</a>
</code></pre>
<br />

Equivalent to [`register_focusin_callback(_Target,
[])`](#register_focusin_callback-2).

<a name="register_focusin_callback-2"></a>

### register_focusin_callback/2 ###

<pre><code>
register_focusin_callback(Target::<a href="#type.md5_target">html5_target()</a>,
Options::<a href="#type-register_options">register_options()</a>) -&gt; <a
href="#type-register_result">register_result()</a>
</code></pre>
<br />

Register for focusin events.

__See also:__ [register_blur_callback/2](#register_blur_callback-2).

<a name="register_focusin_callback-3"></a>

### register_focusin_callback/3 ###

<pre><code>
register_focusin_callback(Target::<a href="#type.md5_target">html5_target()</a>,
Options::<a href="#type-register_options">register_options()</a>, UserData::any())
-&gt; <a href="#type-register_result">register_result()</a>
</code></pre>
<br />

Register for focusin events.

__See also:__ [register_blur_callback/2](#register_blur_callback-2).

<a name="register_focusout_callback-1"></a>

### register_focusout_callback/1 ###

<pre><code>
register_focusout_callback(Target::<a href="#type.md5_target">html5_target()</a>)
-&gt; <a href="#type-register_result">register_result()</a>
</code></pre>
<br />

Equivalent to [`register_focusout_callback(_Target,
[])`](#register_focusout_callback-2).

<a name="register_focusout_callback-2"></a>

### register_focusout_callback/2 ###

<pre><code>
register_focusout_callback(Target::<a href="#type.md5_target">html5_target()</a>,
Options::<a href="#type-register_options">register_options()</a>) -&gt; <a
href="#type-register_result">register_result()</a>
</code></pre>
<br />

Register for focusout events.

__See also:__ [register_blur_callback/2](#register_blur_callback-2).
```

```
<a name="register_focusout_callback-3"></a>

### register_focusout_callback/3 ###

<pre><code>
register_focusout_callback(Target::<a href="#type.md5_target">html5_target()</a>,
Options::<a href="#type-register_options">register_options()</a>, UserData::any())
-&gt; <a href="#type-register_result">register_result()</a>
</code></pre>
<br />

Register for focusout events.

__See also:__ [register_blur_callback/2](#register_blur_callback-2).

<a name="register_keydown_callback-1"></a>

### register_keydown_callback/1 ###

<pre><code>
register_keydown_callback(Target::<a href="#type.md5_target">html5_target()</a>)
-&gt; <a href="#type-register_result">register_result()</a>
</code></pre>
<br />

Equivalent to [`register_keydown_callback(_Target,
[])`](#register_keydown_callback-2).

<a name="register_keydown_callback-2"></a>

### register_keydown_callback/2 ###

<pre><code>
register_keydown_callback(Target::<a href="#type.md5_target">html5_target()</a>,
Options::<a href="#type-register_options">register_options()</a>) -&gt; <a
href="#type-register_result">register_result()</a>
</code></pre>
<br />

Register for keydown events.

__See also:__ [register_keypress_callback/2](#register_keypress_callback-2).

<a name="register_keydown_callback-3"></a>

### register_keydown_callback/3 ###

<pre><code>
register_keydown_callback(Target::<a href="#type.md5_target">html5_target()</a>,
Options::<a href="#type-register_options">register_options()</a>, UserData::any())
-&gt; <a href="#type-register_result">register_result()</a>
</code></pre>
<br />

Register for keydown events.

__See also:__ [register_keypress_callback/2](#register_keypress_callback-2).

<a name="register_keypress_callback-1"></a>

### register_keypress_callback/1 ###

<pre><code>
register_keypress_callback(Target::<a href="#type.md5_target">html5_target()</a>)
```

```
-&gt; <a href="#type-register_result">register_result()</a>
</code></pre>
<br />

Equivalent to [`register_keypress_callback(_Target,
[])`](#register_keypress_callback-2).

<a name="register_keypress_callback-2"></a>

### register_keypress_callback/2 ###

<pre><code>
register_keypress_callback(Target::<a href="#type.md5_target">html5_target()</a>,
Options::<a href="#type-register_options">register_options()</a>) -&gt; <a
href="#type-register_result">register_result()</a>
</code></pre>
<br />

Register for keypress events.
This function registers with no user data and events are sent as:
```

{emscripten, {keypress, keyboard_event()}}

```
__See also:__ [register_keypress_callback/3](#register_keypress_callback-3).

<a name="register_keypress_callback-3"></a>

### register_keypress_callback/3 ###

<pre><code>
register_keypress_callback(Target::<a href="#type.md5_target">html5_target()</a>,
Options::<a href="#type-register_options">register_options()</a>, UserData::any())
-&gt; <a href="#type-register_result">register_result()</a>
</code></pre>
<br />

Register for keypress events.
This function registers keypress events on a given target.
Target can be specified as special atoms `window` for Javascript's window,
`document` for window.document. `screen` is also supported, but it doesn't
seem to work, see
[`https://github.com/emscripten-core/emscripten/issues/19865`](https://github.com/emscripten-cor

Second parameter specifies options which can be a `boolean()` to match
`useCapture` in Emscripten's API. Alternatively, the option can be a proplist()
with `use_capture` and `prevent_default` keys. `prevent_default` determines
what the handler should return to Javascript, `true` meaning that the default
should be prevented.

Third parameter is user data that is passed back. Indeed, when an event
occurs, the following message is sent to the process that registered the
event:
```

{emscripten, {keypress, keyboard_event()}, UserData}

```
The function eventually returns a `listener_handle()` or an error. The
handler is an opaque resource that actually contains a copy of `UserData`.
Please note that if the calling process dies, the callback and any callback
for the same event on the same target are unregistered.
```

```
<a name="register_keyup_callback-1"></a>

### register_keyup_callback/1 ###

<pre><code>
register_keyup_callback(Target::<a href="#type.md5_target">html5_target()</a>)
-&gt; <a href="#type-register_result">register_result()</a>
</code></pre>
<br />

Equivalent to [`register_keyup_callback(_Target,
[])`](#register_keyup_callback-2).

<a name="register_keyup_callback-2"></a>

### register_keyup_callback/2 ###

<pre><code>
register_keyup_callback(Target::<a href="#type.md5_target">html5_target()</a>,
Options::<a href="#type-register_options">register_options()</a>) -&gt; <a
href="#type-register_result">register_result()</a>
</code></pre>
<br />

Register for keyup events.

__See also:__ [register_keypress_callback/2](#register_keypress_callback-2).

<a name="register_keyup_callback-3"></a>

### register_keyup_callback/3 ###

<pre><code>
register_keyup_callback(Target::<a href="#type.md5_target">html5_target()</a>,
Options::<a href="#type-register_options">register_options()</a>, UserData::any())
-&gt; <a href="#type-register_result">register_result()</a>
</code></pre>
<br />

Register for keyup events.

__See also:__ [register_keypress_callback/2](#register_keypress_callback-2).

<a name="register_mousedown_callback-1"></a>

### register_mousedown_callback/1 ###

<pre><code>
register_mousedown_callback(Target::<a href="#type.md5_target">html5_target()</a>)
-&gt; <a href="#type-register_result">register_result()</a>
</code></pre>
<br />

Equivalent to [`register_mousedown_callback(_Target,
[])`](#register_mousedown_callback-2).

<a name="register_mousedown_callback-2"></a>

### register_mousedown_callback/2 ###

<pre><code>
register_mousedown_callback(Target::<a href="#type.md5_target">html5_target()</a>,
Options::<a href="#type-register_options">register_options()</a>) -&gt; <a
href="#type-register_result">register_result()</a>
```

```
</code></pre>
<br />

Register for mousedown events.

__See also:__ [register_click_callback/2](#register_click_callback-2).

<a name="register_mousedown_callback-3"></a>

### register_mousedown_callback/3 ###

<pre><code>
register_mousedown_callback(Target::<a href="#type.md5_target">html5_target()</a>,
Options::<a href="#type-register_options">register_options()</a>, UserData::any())
-&gt; <a href="#type-register_result">register_result()</a>
</code></pre>
<br />

Register for mousedown events.

__See also:__ [register_click_callback/2](#register_click_callback-2).

<a name="register_mouseenter_callback-1"></a>

### register_mouseenter_callback/1 ###

<pre><code>
register_mouseenter_callback(Target::<a
href="#type.md5_target">html5_target()</a>) -&gt; <a
href="#type-register_result">register_result()</a>
</code></pre>
<br />

Equivalent to [`register_mouseenter_callback(_Target,
[])`](#register_mouseenter_callback-2).

<a name="register_mouseenter_callback-2"></a>

### register_mouseenter_callback/2 ###

<pre><code>
register_mouseenter_callback(Target::<a
href="#type.md5_target">html5_target()</a>, Options::<a
href="#type-register_options">register_options()</a>) -&gt; <a
href="#type-register_result">register_result()</a>
</code></pre>
<br />

Register for mouseenter events.

__See also:__ [register_click_callback/2](#register_click_callback-2).

<a name="register_mouseenter_callback-3"></a>

### register_mouseenter_callback/3 ###

<pre><code>
register_mouseenter_callback(Target::<a
href="#type.md5_target">html5_target()</a>, Options::<a
href="#type-register_options">register_options()</a>, UserData::any()) -&gt; <a
href="#type-register_result">register_result()</a>
</code></pre>
<br />
```

```
Register for mouseenter events.

__See also:__ [register_click_callback/2](#register_click_callback-2).

<a name="register_mouseleave_callback-1"></a>

### register_mouseleave_callback/1 ###

<pre><code>
register_mouseleave_callback(Target::<a
href="#type.md5_target">html5_target()</a>) -&gt; <a
href="#type-register_result">register_result()</a>
</code></pre>
<br />

Equivalent to [`register_mouseleave_callback(_Target,
[])`](#register_mouseleave_callback-2).

<a name="register_mouseleave_callback-2"></a>

### register_mouseleave_callback/2 ###

<pre><code>
register_mouseleave_callback(Target::<a
href="#type.md5_target">html5_target()</a>, Options::<a
href="#type-register_options">register_options()</a>) -&gt; <a
href="#type-register_result">register_result()</a>
</code></pre>
<br />

Register for mouseleave events.

__See also:__ [register_click_callback/2](#register_click_callback-2).

<a name="register_mouseleave_callback-3"></a>

### register_mouseleave_callback/3 ###

<pre><code>
register_mouseleave_callback(Target::<a
href="#type.md5_target">html5_target()</a>, Options::<a
href="#type-register_options">register_options()</a>, UserData::any()) -&gt; <a
href="#type-register_result">register_result()</a>
</code></pre>
<br />

Register for mouseleave events.

__See also:__ [register_click_callback/2](#register_click_callback-2).

<a name="register_mousemove_callback-1"></a>

### register_mousemove_callback/1 ###

<pre><code>
register_mousemove_callback(Target::<a href="#type.md5_target">html5_target()</a>)
-&gt; <a href="#type-register_result">register_result()</a>
</code></pre>
<br />

Equivalent to [`register_mousemove_callback(_Target,
[])`](#register_mousemove_callback-2).

<a name="register_mousemove_callback-2"></a>
```

```
### register_mousemove_callback/2 ###

<pre><code>
register_mousemove_callback(Target::<a href="#type.md5_target">html5_target()</a>,
Options::<a href="#type-register_options">register_options()</a>) -&gt; <a
href="#type-register_result">register_result()</a>
</code></pre>
<br />

Register for mousemove events.

__See also:__ [register_click_callback/2](#register_click_callback-2).

<a name="register_mousemove_callback-3"></a>

### register_mousemove_callback/3 ###

<pre><code>
register_mousemove_callback(Target::<a href="#type.md5_target">html5_target()</a>,
Options::<a href="#type-register_options">register_options()</a>, UserData::any())
-&gt; <a href="#type-register_result">register_result()</a>
</code></pre>
<br />

Register for mousemove events.

__See also:__ [register_click_callback/2](#register_click_callback-2).

<a name="register_mouseout_callback-1"></a>

### register_mouseout_callback/1 ###

<pre><code>
register_mouseout_callback(Target::<a href="#type.md5_target">html5_target()</a>)
-&gt; <a href="#type-register_result">register_result()</a>
</code></pre>
<br />

Equivalent to [`register_mouseout_callback(_Target,
[])`](#register_mouseout_callback-2).

<a name="register_mouseout_callback-2"></a>

### register_mouseout_callback/2 ###

<pre><code>
register_mouseout_callback(Target::<a href="#type.md5_target">html5_target()</a>,
Options::<a href="#type-register_options">register_options()</a>) -&gt; <a
href="#type-register_result">register_result()</a>
</code></pre>
<br />

Register for mouseout events.

__See also:__ [register_click_callback/2](#register_click_callback-2).

<a name="register_mouseout_callback-3"></a>

### register_mouseout_callback/3 ###

<pre><code>
register_mouseout_callback(Target::<a href="#type.md5_target">html5_target()</a>,
Options::<a href="#type-register_options">register_options()</a>, UserData::any())
-&gt; <a href="#type-register_result">register_result()</a>
```

```
</code></pre>
<br />

Register for mouseout events.

__See also:__ [register_click_callback/2](#register_click_callback-2).

<a name="register_mouseover_callback-1"></a>

### register_mouseover_callback/1 ###

<pre><code>
register_mouseover_callback(Target::<a href="#type.md5_target">html5_target()</a>)
-&gt; <a href="#type-register_result">register_result()</a>
</code></pre>
<br />

Equivalent to [`register_mouseover_callback(_Target,
[])`](#register_mouseover_callback-2).

<a name="register_mouseover_callback-2"></a>

### register_mouseover_callback/2 ###

<pre><code>
register_mouseover_callback(Target::<a href="#type.md5_target">html5_target()</a>,
Options::<a href="#type-register_options">register_options()</a>) -&gt; <a
href="#type-register_result">register_result()</a>
</code></pre>
<br />

Register for mouseover events.

__See also:__ [register_click_callback/2](#register_click_callback-2).

<a name="register_mouseover_callback-3"></a>

### register_mouseover_callback/3 ###

<pre><code>
register_mouseover_callback(Target::<a href="#type.md5_target">html5_target()</a>,
Options::<a href="#type-register_options">register_options()</a>, UserData::any())
-&gt; <a href="#type-register_result">register_result()</a>
</code></pre>
<br />

Register for mouseover events.

__See also:__ [register_click_callback/2](#register_click_callback-2).

<a name="register_mouseup_callback-1"></a>

### register_mouseup_callback/1 ###

<pre><code>
register_mouseup_callback(Target::<a href="#type.md5_target">html5_target()</a>)
-&gt; <a href="#type-register_result">register_result()</a>
</code></pre>
<br />

Equivalent to [`register_mouseup_callback(_Target,
[])`](#register_mouseup_callback-2).

<a name="register_mouseup_callback-2"></a>
```

```
### register_mouseup_callback/2 ###

<pre><code>
register_mouseup_callback(Target::<a href="#type.md5_target">html5_target()</a>,
Options::<a href="#type-register_options">register_options()</a>) -&gt; <a
href="#type-register_result">register_result()</a>
</code></pre>
<br />

Register for mouseup events.

__See also:__ [register_click_callback/2](#register_click_callback-2).

<a name="register_mouseup_callback-3"></a>

### register_mouseup_callback/3 ###

<pre><code>
register_mouseup_callback(Target::<a href="#type.md5_target">html5_target()</a>,
Options::<a href="#type-register_options">register_options()</a>, UserData::any())
-&gt; <a href="#type-register_result">register_result()</a>
</code></pre>
<br />

Register for mouseup events.

__See also:__ [register_click_callback/2](#register_click_callback-2).

<a name="register_resize_callback-1"></a>

### register_resize_callback/1 ###

<pre><code>
register_resize_callback(Target::<a href="#type.md5_target">html5_target()</a>)
-&gt; <a href="#type-register_result">register_result()</a>
</code></pre>
<br />

Equivalent to [`register_resize_callback(_Target,
[])`](#register_resize_callback-2).

<a name="register_resize_callback-2"></a>

### register_resize_callback/2 ###

<pre><code>
register_resize_callback(Target::<a href="#type.md5_target">html5_target()</a>,
Options::<a href="#type-register_options">register_options()</a>) -&gt; <a
href="#type-register_result">register_result()</a>
</code></pre>
<br />

Register for resize events.
Events are sent as:
```

{emscripten, {resize, ui_event()}}

```
__See also:__ [register_keypress_callback/2](#register_keypress_callback-2).

<a name="register_resize_callback-3"></a>

### register_resize_callback/3 ###
```

```
<pre><code>
register_resize_callback(Target::<a href="#type.md5_target">html5_target()</a>,
Options::<a href="#type-register_options">register_options()</a>, UserData::any())
-&gt; <a href="#type-register_result">register_result()</a>
</code></pre>
<br />

Register for resize events.
Events are sent as:
```

{emscripten, {resize, ui_event()}, UserData}

```
__See also:__ [register_keypress_callback/2](#register_keypress_callback-2).

<a name="register_scroll_callback-1"></a>

### register_scroll_callback/1 ###

<pre><code>
register_scroll_callback(Target::<a href="#type.md5_target">html5_target()</a>)
-&gt; <a href="#type-register_result">register_result()</a>
</code></pre>
<br />

Equivalent to [`register_scroll_callback(_Target,
[])`](#register_scroll_callback-2).

<a name="register_scroll_callback-2"></a>

### register_scroll_callback/2 ###

<pre><code>
register_scroll_callback(Target::<a href="#type.md5_target">html5_target()</a>,
Options::<a href="#type-register_options">register_options()</a>) -&gt; <a
href="#type-register_result">register_result()</a>
</code></pre>
<br />

Register for scroll events.

__See also:__ [register_resize_callback/2](#register_resize_callback-2).

<a name="register_scroll_callback-3"></a>

### register_scroll_callback/3 ###

<pre><code>
register_scroll_callback(Target::<a href="#type.md5_target">html5_target()</a>,
Options::<a href="#type-register_options">register_options()</a>, UserData::any())
-&gt; <a href="#type-register_result">register_result()</a>
</code></pre>
<br />

Register for scroll events.

__See also:__ [register_resize_callback/2](#register_resize_callback-2).

<a name="register_touchcancel_callback-1"></a>

### register_touchcancel_callback/1 ###

<pre><code>
```

```
register_touchcancel_callback(Target::<a
href="#type.md5_target">html5_target()</a>) -&gt; <a
href="#type-register_result">register_result()</a>
</code></pre>
<br />

Equivalent to [`register_touchcancel_callback(_Target,
[])`](#register_touchcancel_callback-2).

<a name="register_touchcancel_callback-2"></a>

### register_touchcancel_callback/2 ###

<pre><code>
register_touchcancel_callback(Target::<a
href="#type.md5_target">html5_target()</a>, Options::<a
href="#type-register_options">register_options()</a>) -&gt; <a
href="#type-register_result">register_result()</a>
</code></pre>
<br />

Register for touchcancel events.

__See also:__ [register_touchstart_callback/2](#register_touchstart_callback-2).

<a name="register_touchcancel_callback-3"></a>

### register_touchcancel_callback/3 ###

<pre><code>
register_touchcancel_callback(Target::<a
href="#type.md5_target">html5_target()</a>, Options::<a
href="#type-register_options">register_options()</a>, UserData::any()) -&gt; <a
href="#type-register_result">register_result()</a>
</code></pre>
<br />

Register for touchcancel events.

__See also:__ [register_touchstart_callback/2](#register_touchstart_callback-2).

<a name="register_touchend_callback-1"></a>

### register_touchend_callback/1 ###

<pre><code>
register_touchend_callback(Target::<a href="#type.md5_target">html5_target()</a>)
-&gt; <a href="#type-register_result">register_result()</a>
</code></pre>
<br />

Equivalent to [`register_touchend_callback(_Target,
[])`](#register_touchend_callback-2).

<a name="register_touchend_callback-2"></a>

### register_touchend_callback/2 ###

<pre><code>
register_touchend_callback(Target::<a href="#type.md5_target">html5_target()</a>,
Options::<a href="#type-register_options">register_options()</a>) -&gt; <a
href="#type-register_result">register_result()</a>
</code></pre>
<br />
```

```
Register for touchend events.

__See also:__ [register_touchstart_callback/2](#register_touchstart_callback-2).

<a name="register_touchend_callback-3"></a>

### register_touchend_callback/3 ###

<pre><code>
register_touchend_callback(Target::<a href="#type.md5_target">html5_target()</a>,
Options::<a href="#type-register_options">register_options()</a>, UserData::any())
-&gt; <a href="#type-register_result">register_result()</a>
</code></pre>
<br />

Register for touchend events.

__See also:__ [register_touchstart_callback/2](#register_touchstart_callback-2).

<a name="register_touchmove_callback-1"></a>

### register_touchmove_callback/1 ###

<pre><code>
register_touchmove_callback(Target::<a href="#type.md5_target">html5_target()</a>)
-&gt; <a href="#type-register_result">register_result()</a>
</code></pre>
<br />

Equivalent to [`register_touchmove_callback(_Target,
[])`](#register_touchmove_callback-2).

<a name="register_touchmove_callback-2"></a>

### register_touchmove_callback/2 ###

<pre><code>
register_touchmove_callback(Target::<a href="#type.md5_target">html5_target()</a>,
Options::<a href="#type-register_options">register_options()</a>) -&gt; <a
href="#type-register_result">register_result()</a>
</code></pre>
<br />

Register for touchmove events.

__See also:__ [register_touchstart_callback/2](#register_touchstart_callback-2).

<a name="register_touchmove_callback-3"></a>

### register_touchmove_callback/3 ###

<pre><code>
register_touchmove_callback(Target::<a href="#type.md5_target">html5_target()</a>,
Options::<a href="#type-register_options">register_options()</a>, UserData::any())
-&gt; <a href="#type-register_result">register_result()</a>
</code></pre>
<br />

Register for touchmove events.

__See also:__ [register_touchstart_callback/2](#register_touchstart_callback-2).

<a name="register_touchstart_callback-1"></a>
```

```
### register_touchstart_callback/1 ###

<pre><code>
register_touchstart_callback(Target::<a
href="#type.md5_target">html5_target()</a>) -&gt; <a
href="#type-register_result">register_result()</a>
</code></pre>
<br />

Equivalent to [`register_touchstart_callback(_Target,
[])`](#register_touchstart_callback-2).

<a name="register_touchstart_callback-2"></a>

### register_touchstart_callback/2 ###

<pre><code>
register_touchstart_callback(Target::<a
href="#type.md5_target">html5_target()</a>, Options::<a
href="#type-register_options">register_options()</a>) -&gt; <a
href="#type-register_result">register_result()</a>
</code></pre>
<br />

Register for touchstart events.
Events are sent as:
```

{emscripten, {touchstart, touch_event()}}

```
__See also:__ [register_keypress_callback/2](#register_keypress_callback-2).

<a name="register_touchstart_callback-3"></a>

### register_touchstart_callback/3 ###

<pre><code>
register_touchstart_callback(Target::<a
href="#type.md5_target">html5_target()</a>, Options::<a
href="#type-register_options">register_options()</a>, UserData::any()) -&gt; <a
href="#type-register_result">register_result()</a>
</code></pre>
<br />

Register for touchstart events.
Events are sent as:
```

{emscripten, {touchstart, touch_event()}, UserData}

```
__See also:__ [register_keypress_callback/2](#register_keypress_callback-2).

<a name="register_wheel_callback-1"></a>

### register_wheel_callback/1 ###

<pre><code>
register_wheel_callback(Target::<a href="#type.md5_target">html5_target()</a>)
-&gt; <a href="#type-register_result">register_result()</a>
</code></pre>
<br />

Equivalent to [`register_wheel_callback(_Target,
```

```
[])`](#register_wheel_callback-2).

<a name="register_wheel_callback-2"></a>

### register_wheel_callback/2 ###

<pre><code>
register_wheel_callback(Target::<a href="#type.md5_target">html5_target()</a>,
Options::<a href="#type-register_options">register_options()</a>) -&gt; <a
href="#type-register_result">register_result()</a>
</code></pre>
<br />

Register for wheel events.
Events are sent as:
```

{emscripten, {wheel, wheel_event()}}

```
__See also:__ [register_keypress_callback/2](#register_keypress_callback-2).

<a name="register_wheel_callback-3"></a>

### register_wheel_callback/3 ###

<pre><code>
register_wheel_callback(Target::<a href="#type.md5_target">html5_target()</a>,
Options::<a href="#type-register_options">register_options()</a>, UserData::any())
-&gt; <a href="#type-register_result">register_result()</a>
</code></pre>
<br />

Register for wheel events.
Events are sent as:
```

{emscripten, {wheel, mouse_event()}, UserData}

```
__See also:__ [register_keypress_callback/2](#register_keypress_callback-2).

<a name="run_script-1"></a>

### run_script/1 ###

<pre><code>
run_script(Script::iodata()) -&gt; ok
</code></pre>
<br />

Equivalent to [`run_script(_Script, [])`](#run_script-2).

<a name="run_script-2"></a>

### run_script/2 ###

<pre><code>
run_script(Script::iodata(), Options::[<a
href="#type-run_script_opt">run_script_opt()</a>]) -&gt; ok
</code></pre>
<br />

returns: ok
```

```
Run a script.
By default, the script is run in the current worker thread, which arguably
may not be very useful. Please note that exception handling is disabled, so
the script should not throw and should compile, otherwise this will crash
the VM.

<a name="unregister_blur_callback-1"></a>

### unregister_blur_callback/1 ###

<pre><code>
unregister_blur_callback(TargetOrHandle::<a
href="#type.md5_target">html5_target()</a> | <a
href="#type-listener_handle">listener_handle()</a>) -&gt; ok | {error, <a
href="#type-register_error_reason">register_error_reason()</a>}
</code></pre>
<br />

Unregister a blur event handler.

__See also:__ [unregister_keypress_callback/1](#unregister_keypress_callback-1).

<a name="unregister_click_callback-1"></a>

### unregister_click_callback/1 ###

<pre><code>
unregister_click_callback(TargetOrHandle::<a
href="#type.md5_target">html5_target()</a> | <a
href="#type-listener_handle">listener_handle()</a>) -&gt; ok | {error, <a
href="#type-register_error_reason">register_error_reason()</a>}
</code></pre>
<br />

Unregister a click event handler.

__See also:__ [unregister_keypress_callback/1](#unregister_keypress_callback-1).

<a name="unregister_dblclick_callback-1"></a>

### unregister_dblclick_callback/1 ###

<pre><code>
unregister_dblclick_callback(TargetOrHandle::<a
href="#type.md5_target">html5_target()</a> | <a
href="#type-listener_handle">listener_handle()</a>) -&gt; ok | {error, <a
href="#type-register_error_reason">register_error_reason()</a>}
</code></pre>
<br />

Unregister a dblclick event handler.

__See also:__ [unregister_click_callback/1](#unregister_click_callback-1).

<a name="unregister_focus_callback-1"></a>

### unregister_focus_callback/1 ###

<pre><code>
unregister_focus_callback(TargetOrHandle::<a
href="#type.md5_target">html5_target()</a> | <a
href="#type-listener_handle">listener_handle()</a>) -&gt; ok | {error, <a
href="#type-register_error_reason">register_error_reason()</a>}
</code></pre>
```

```
<br />

Unregister a focus event handler.

__See also:__ [unregister_blur_callback/1](#unregister_blur_callback-1).

<a name="unregister_focusin_callback-1"></a>

### unregister_focusin_callback/1 ###

<pre><code>
unregister_focusin_callback(TargetOrHandle::<a
href="#type.md5_target">html5_target()</a> | <a
href="#type-listener_handle">listener_handle()</a>) -&gt; ok | {error, <a
href="#type-register_error_reason">register_error_reason()</a>}
</code></pre>
<br />

Unregister a focusin event handler.

__See also:__ [unregister_blur_callback/1](#unregister_blur_callback-1).

<a name="unregister_focusout_callback-1"></a>

### unregister_focusout_callback/1 ###

<pre><code>
unregister_focusout_callback(TargetOrHandle::<a
href="#type.md5_target">html5_target()</a> | <a
href="#type-listener_handle">listener_handle()</a>) -&gt; ok | {error, <a
href="#type-register_error_reason">register_error_reason()</a>}
</code></pre>
<br />

Unregister a focusout event handler.

__See also:__ [unregister_blur_callback/1](#unregister_blur_callback-1).

<a name="unregister_keydown_callback-1"></a>

### unregister_keydown_callback/1 ###

<pre><code>
unregister_keydown_callback(TargetOrHandle::<a
href="#type.md5_target">html5_target()</a> | <a
href="#type-listener_handle">listener_handle()</a>) -&gt; ok | {error, <a
href="#type-register_error_reason">register_error_reason()</a>}
</code></pre>
<br />

Unregister a keydown event handler.

__See also:__ [unregister_keypress_callback/1](#unregister_keypress_callback-1).

<a name="unregister_keypress_callback-1"></a>

### unregister_keypress_callback/1 ###

<pre><code>
unregister_keypress_callback(TargetOrHandle::<a
href="#type.md5_target">html5_target()</a> | <a
href="#type-listener_handle">listener_handle()</a>) -&gt; ok | {error, <a
href="#type-register_error_reason">register_error_reason()</a>}
</code></pre>
```

```
<br />

returns: ok or an error

Unregister a keypress listener.

To match Emscripten's API, this function can take a target. This function
unregisters every keypress listeners on the specified target.

Alternatively, this function can take a listener_handle() returned by
`register_keypress_callback/1,2,3`. This will still unregister every
keypress listeners on the same target.

Passing a `listener_handle()` is recommended to avoid surprises and for
memory efficiency. If a handle is passed, it can then be garbage collected.
If a handle is not passed, but the process dies, every keypress listener
on the same target will be unregistered, including listeners that were
later registered. This is a known limitation of the implementation that
favored avoiding memory leaks and crashes.

<a name="unregister_keyup_callback-1"></a>

### unregister_keyup_callback/1 ###

<pre><code>
unregister_keyup_callback(TargetOrHandle::<a
href="#type.md5_target">html5_target()</a> | <a
href="#type-listener_handle">listener_handle()</a>) -&gt; ok | {error, <a
href="#type-register_error_reason">register_error_reason()</a>}
</code></pre>
<br />

Unregister a keyup event handler.

__See also:__ [unregister_keypress_callback/1](#unregister_keypress_callback-1).

<a name="unregister_mousedown_callback-1"></a>

### unregister_mousedown_callback/1 ###

<pre><code>
unregister_mousedown_callback(TargetOrHandle::<a
href="#type.md5_target">html5_target()</a> | <a
href="#type-listener_handle">listener_handle()</a>) -&gt; ok | {error, <a
href="#type-register_error_reason">register_error_reason()</a>}
</code></pre>
<br />

Unregister a mousedown event handler.

__See also:__ [unregister_click_callback/1](#unregister_click_callback-1).

<a name="unregister_mouseenter_callback-1"></a>

### unregister_mouseenter_callback/1 ###

<pre><code>
unregister_mouseenter_callback(TargetOrHandle::<a
href="#type.md5_target">html5_target()</a> | <a
href="#type-listener_handle">listener_handle()</a>) -&gt; ok | {error, <a
href="#type-register_error_reason">register_error_reason()</a>}
</code></pre>
<br />
```

```
Unregister a mouseenter event handler.

__See also:__ [unregister_click_callback/1](#unregister_click_callback-1).

<a name="unregister_mouseleave_callback-1"></a>

### unregister_mouseleave_callback/1 ###

<pre><code>
unregister_mouseleave_callback(TargetOrHandle::<a
href="#type.md5_target">html5_target()</a> | <a
href="#type-listener_handle">listener_handle()</a>) -&gt; ok | {error, <a
href="#type-register_error_reason">register_error_reason()</a>}
</code></pre>
<br />

Unregister a mouseleave event handler.

__See also:__ [unregister_click_callback/1](#unregister_click_callback-1).

<a name="unregister_mousemove_callback-1"></a>

### unregister_mousemove_callback/1 ###

<pre><code>
unregister_mousemove_callback(TargetOrHandle::<a
href="#type.md5_target">html5_target()</a> | <a
href="#type-listener_handle">listener_handle()</a>) -&gt; ok | {error, <a
href="#type-register_error_reason">register_error_reason()</a>}
</code></pre>
<br />

Unregister a mousemove event handler.

__See also:__ [unregister_click_callback/1](#unregister_click_callback-1).

<a name="unregister_mouseout_callback-1"></a>

### unregister_mouseout_callback/1 ###

<pre><code>
unregister_mouseout_callback(TargetOrHandle::<a
href="#type.md5_target">html5_target()</a> | <a
href="#type-listener_handle">listener_handle()</a>) -&gt; ok | {error, <a
href="#type-register_error_reason">register_error_reason()</a>}
</code></pre>
<br />

Unregister a mouseout event handler.

__See also:__ [unregister_click_callback/1](#unregister_click_callback-1).

<a name="unregister_mouseover_callback-1"></a>

### unregister_mouseover_callback/1 ###

<pre><code>
unregister_mouseover_callback(TargetOrHandle::<a
href="#type.md5_target">html5_target()</a> | <a
href="#type-listener_handle">listener_handle()</a>) -&gt; ok | {error, <a
href="#type-register_error_reason">register_error_reason()</a>}
</code></pre>
<br />
```

```
Unregister a mouseover event handler.

__See also:__ [unregister_click_callback/1](#unregister_click_callback-1).

<a name="unregister_mouseup_callback-1"></a>

### unregister_mouseup_callback/1 ###

<pre><code>
unregister_mouseup_callback(TargetOrHandle::<a
href="#type.md5_target">html5_target()</a> | <a
href="#type-listener_handle">listener_handle()</a>) -&gt; ok | {error, <a
href="#type-register_error_reason">register_error_reason()</a>}
</code></pre>
<br />

Unregister a mouseup event handler.

__See also:__ [unregister_click_callback/1](#unregister_click_callback-1).

<a name="unregister_resize_callback-1"></a>

### unregister_resize_callback/1 ###

<pre><code>
unregister_resize_callback(TargetOrHandle::<a
href="#type.md5_target">html5_target()</a> | <a
href="#type-listener_handle">listener_handle()</a>) -&gt; ok | {error, <a
href="#type-register_error_reason">register_error_reason()</a>}
</code></pre>
<br />

Unregister a resize event handler.

__See also:__ [unregister_keypress_callback/1](#unregister_keypress_callback-1).

<a name="unregister_scroll_callback-1"></a>

### unregister_scroll_callback/1 ###

<pre><code>
unregister_scroll_callback(TargetOrHandle::<a
href="#type.md5_target">html5_target()</a> | <a
href="#type-listener_handle">listener_handle()</a>) -&gt; ok | {error, <a
href="#type-register_error_reason">register_error_reason()</a>}
</code></pre>
<br />

Unregister a scroll event handler.

__See also:__ [unregister_resize_callback/1](#unregister_resize_callback-1).

<a name="unregister_touchcancel_callback-1"></a>

### unregister_touchcancel_callback/1 ###

<pre><code>
unregister_touchcancel_callback(TargetOrHandle::<a
href="#type.md5_target">html5_target()</a> | <a
href="#type-listener_handle">listener_handle()</a>) -&gt; ok | {error, <a
href="#type-register_error_reason">register_error_reason()</a>}
</code></pre>
<br />
```

```
Unregister a touchcancel event handler.

__See also:__
[unregister_touchstart_callback/1](#unregister_touchstart_callback-1).

<a name="unregister_touchend_callback-1"></a>

### unregister_touchend_callback/1 ###

<pre><code>
unregister_touchend_callback(TargetOrHandle::<a
href="#type.md5_target">html5_target()</a> | <a
href="#type-listener_handle">listener_handle()</a>) -&gt; ok | {error, <a
href="#type-register_error_reason">register_error_reason()</a>}
</code></pre>
<br />

Unregister a touchend event handler.

__See also:__
[unregister_touchstart_callback/1](#unregister_touchstart_callback-1).

<a name="unregister_touchmove_callback-1"></a>

### unregister_touchmove_callback/1 ###

<pre><code>
unregister_touchmove_callback(TargetOrHandle::<a
href="#type.md5_target">html5_target()</a> | <a
href="#type-listener_handle">listener_handle()</a>) -&gt; ok | {error, <a
href="#type-register_error_reason">register_error_reason()</a>}
</code></pre>
<br />

Unregister a touchmove event handler.

__See also:__
[unregister_touchstart_callback/1](#unregister_touchstart_callback-1).

<a name="unregister_touchstart_callback-1"></a>

### unregister_touchstart_callback/1 ###

<pre><code>
unregister_touchstart_callback(TargetOrHandle::<a
href="#type.md5_target">html5_target()</a> | <a
href="#type-listener_handle">listener_handle()</a>) -&gt; ok | {error, <a
href="#type-register_error_reason">register_error_reason()</a>}
</code></pre>
<br />

Unregister a touchstart event handler.

__See also:__ [unregister_keypress_callback/1](#unregister_keypress_callback-1).

<a name="unregister_wheel_callback-1"></a>

### unregister_wheel_callback/1 ###

<pre><code>
unregister_wheel_callback(TargetOrHandle::<a
href="#type.md5_target">html5_target()</a> | <a
href="#type-listener_handle">listener_handle()</a>) -&gt; ok | {error, <a
href="#type-register_error_reason">register_error_reason()</a>}
```

```
</code></pre>
<br />

Unregister a wheel event handler.

__See also:__ [unregister_keypress_callback/1](#unregister_keypress_callback-1).
```

Module esp

- Description
- Data Types
- Function Index
- Section 9.4.2

ESP32-specific APIs.

Description

This module contains functions that are specific to the ESP32 platform.

Data Types

esp_partition()

esp_partition_address()

esp_partition_props()

esp_partition_size()

esp_partition_subtype()

esp_partition_type()

esp_reset_reason()

esp_wakeup_cause()

interface()

mac()

Function Index

Function Details

deep_sleep/0

Put the esp32 into deep sleep. This function never returns. Program is restarted and wake up reason can be inspected to determine how the esp32 was woken up.

deep_sleep/1

`SleepMS`: time to deep sleep in milliseconds

Put the esp32 into deep sleep. This function never returns. Program is restarted and wake up reason can be inspected to determine if the esp32 was woken by the timeout or by another cause.

freq_hz/0

returns: Clock frequency (in hz)

Return the clock frequency on the chip

get_mac/1

`Interface`: the ESP32 network interface

returns: The network MAC address of the specified interface

Return the network MAC address of the specified interface.

The mac address is returned as a 6-byte binary, per the IEEE 802 family of specifications.

nvs_erase_all/0

This function is deprecated: Please do not use this function.

Equivalent to nvs_erase_all(?ATOMVM_NVS_NS).

nvs_erase_all/1

`Namespace`: NVS namespace

returns: ok

Erase all values in the specified namespace.

nvs_erase_key/1

`Key`: NVS key

returns: ok

This function is deprecated: Please do not use this function.

Equivalent to nvs_erase_key(?ATOMVM_NVS_NS, Key).

nvs_erase_key/2

`Namespace`: NVS namespace`Key`: NVS key

returns: ok

Erase the value associated with a key. If a value does not exist for the specified key, no action is performed.

nvs_fetch_binary/2

`Namespace`: NVS namespace`Key`: NVS key

returns: tagged tuple with binary value associated with this key in NV storage, {error, not_found} if there is no value associated with this key, or in general {error, Reason} for any other error.

Get the binary value associated with a key, or undefined, if there is no value associated with this key.

nvs_get_binary/1

This function is deprecated: Please do not use this function.

Equivalent to nvs_get_binary(?ATOMVM_NVS_NS, Key).

nvs_get_binary/2

`Namespace`: NVS namespace`Key`: NVS key

returns: binary value associated with this key in NV storage, or undefined if there is no value associated with this key.

Get the binary value associated with a key, or undefined, if there is no value associated with this key.

nvs_get_binary/3

`Namespace`: NVS namespace`Key`: NVS key`Default`: default binary value, if Key is not set in Namespace

returns: binary value associated with this key in NV storage, or Default if there is no value associated with this key.

Get the binary value associated with a key, or Default, if there is no value associated with this key.

nvs_put_binary/3

`Namespace`: NVS namespace`Key`: NVS key`Value`: binary value

returns: ok

Set an binary value associated with a key. If a value exists for the specified key, it is over-written.

nvs_reformat/0

returns: ok

Reformat the entire NVS partition. WARNING. This will result in deleting all NVS data and should be used with extreme caution!

nvs_set_binary/2

This function is deprecated: Please use nvs_put_binary instead.

Equivalent to nvs_set_binary(?ATOMVM_NVS_NS, Key, Value).

nvs_set_binary/3

`Namespace`: NVS namespace`Key`: NVS key`Value`: binary value

returns: ok

This function is deprecated: Please use nvs_put_binary instead.

Set an binary value associated with a key. If a value exists for the specified key, it is over-written.

partition_list/0

returns: List of partitions

Gets the list of partitions as tuples, such as {name, type, subtype, offset, size, props}. Type and subtype are integers as described in esp-idf documentation.

reset_reason/0

returns: the reason for the restart

Returns the reason for the restart

restart/0

Restarts the ESP device

rtc_slow_get_binary/0

returns: the currently stored binary in RTC slow memory.

Get the binary currently stored in RTC slow memory. Must not be called unless the binary was stored with rtc_slow_set_binary/1. A limited checksum is ran and this function may throw badarg if the checksum is not valid.

rtc_slow_set_binary/1

`Bin`: binary to be stored in RTC slow memory

returns: ok

Store a binary to RTC slow memory. This memory is not erased on software reset and deep sleeps.

sleep_enable_ext0_wakeup/2

returns: Configure ext0 wakeup

sleep_enable_ext1_wakeup/2

returns: Configure ext1 wakeup

sleep_enable_ulp_wakeup/0

returns: Enable ulp wakeup

sleep_get_wakeup_cause/0

returns: the cause for the wake up

Returns the cause for the wakeup

Module gpio

- Description
- Data Types
- Function Index
- Section 9.4.2

GPIO driver module.

Description

This module provides functions for interacting with micro-controller GPIO (General Purpose Input and Output) pins.

Note: `-type pin()` used in this driver refers to a pin number on Espressif chips, or a tuple {GPIO_-GROUP, PIN} for stm32 chips.

Data Types

direction()

gpio()

high_level()

level()

low_level()

pin()

pull()

trigger()

Function Index

Function Details

attach_interrupt/2

`Pin`: number of the pin to set the interrupt on`Trigger`: is the state that will trigger an interrupt

returns: ok | error

Convenience function for `gpio:set_int/3`

This is a convenience function for `gpio:set_int/3` that allows an interrupt to be set using only the pin number and trigger as arguments.

This function should only be used when only one gpio trigger is used in an application. If multiple pins are being configured with interrupt triggers gpio:set_int/3 should be used otherwise there is a race condition when start() is called internally by this function.

close/1

`GPIO`: pid that was returned from gpio:start/0

returns: ok atom

Stop the GPIO interrupt port

This function disables any interrupts that are set, stops the listening port, and frees all of its resources.

deep_sleep_hold_dis/0

returns: ok

Disable all gpio pad functions during Deep-sleep.

deep_sleep_hold_en/0

returns: ok

Enable all hold functions to continue in deep sleep.

The gpio pad hold function works in both input and output modes, but must be output-capable gpios.

When the chip is in Deep-sleep mode, all digital gpio will hold the state before sleep, and when the chip is woken up, the status of digital gpio will not be held. Note that the pad hold feature only works when the chip is in Deep-sleep mode, when not in sleep mode, the digital gpio state can be changed even you have called this function.

Power down or call `gpio_hold_dis` will disable this function, otherwise, the digital gpio hold feature works as long as the chip enters Deep-sleep.

deinit/1

`Pin`: number to deinitialize

returns: ok

Reset a pin back to the NULL function. Currently only implemented for RP2040 (Pico).

detach_interrupt/1

`Pin`: number of the pin to remove the interrupt

returns: ok | error

Convenience function for `gpio:remove_int/2`

This is a convenience function for `gpio:remove_int/2` that allows an interrupt to be removed using only the pin number as an argument.

Unlike `gpio:attach_interrupt/2` this function can be safely used regardless of the number of interrupt pins used in the application.

digital_read/1

`Pin`: number of the pin to read

returns: high | low

Read the digital state of a GPIO pin

Read if an input pin state is high or low. Warning: if a is not previously configured as an input using `gpio:set_pin_mode/2` it will always read as low.

digital_write/2

`Pin`: number of the pin to write

returns: ok | error

Set GPIO digital output level

Set a pin to `high` (1) or `low` (0).

hold_dis/1

`Pin`: number of the pin to be released

returns: ok | error

Release a pin from a hold state.

When the chip is woken up from Deep-sleep, the gpio will be set to the default mode, so, the gpio will output the default level if this function is called. If you don't want the level changes, the gpio should be configured to a known state before this function is called. e.g. If you hold gpio18 high during Deep-sleep, after the chip is woken up and `gpio:hold_dis` is called, gpio18 will output low level(because gpio18 is input mode by default). If you don't want this behavior, you should configure gpio18 as output mode and set it to hight level before calling `gpio:hold_dis`.

hold_en/1

`Pin`: number of the pin to be held

returns: ok | error

Hold the state of a pin

The gpio pad hold function works in both input and output modes, but must be output-capable gpios.

If pad hold enabled: In output mode: the output level of the pad will be force locked and can not be changed. In input mode: the input value read will not change, regardless the changes of input signal.

The state of digital gpio cannot be held during Deep-sleep, and it will resume the hold function when the chip wakes up from Deep-sleep. If the digital gpio also needs to be held during Deep-sleep `gpio:deep_sleep_hold_en` should also be called.

init/1

`Pin`: number to initialize

returns: ok

Initialize a pin to be used as GPIO. Currently only implemented (and required) for RP2040 (Pico).

open/0

returns: Pid

Start the GPIO driver port

The GPIO port driver will be stared and registered as `gpio`. If the port has already been started through the `gpio:open/0` or `gpio:start/0` the command will fail. The use of `gpio:open/0` or `gpio:start/0` is required before using any functions that require a GPIO pid as a parameter.

read/2

`GPIO`: pid that was returned from gpio:start/0`Pin`: number of the pin to read

returns: high | low

Read the digital state of a GPIO pin

Read if an input pin state is `high` or `low`. Warning: if a is not previously configured as an input using `gpio:set_direction/3` it will always read as low.

remove_int/2

`GPIO`: pid that was returned from `gpio:start/0``Pin`: number of the pin to remove the interrupt

returns: ok | error

Remove a GPIO interrupt

Removes an interrupt from the specified pin.

set_direction/3

`GPIO`: pid that was returned from `gpio:start/0``Pin`: number of the pin to configure`Direction`: is `input`, `output`, or `output_od`

returns: ok | error

Set the operational mode of a pin

Pins can be used for input, output, or output with open drain.

set_int/3

`GPIO`: pid that was returned from `gpio:start/0``Pin`: number of the pin to set the interrupt on`Trigger`: is the state that will trigger an interrupt

returns: ok | error

Set a GPIO interrupt

Available triggers are `none` (which is the same as disabling an interrupt), `rising`, `falling`, `both` (rising or falling), `low`, and `high`. When the interrupt is triggered it will send a tuple: `{gpio_interrupt, Pin}` to the process that set the interrupt. Pin will be the number of the pin that triggered the interrupt.

set_level/3

`GPIO`: pid that was returned from `gpio:start/0``Pin`: number of the pin to write

returns: ok | error

Set GPIO digital output level

Set a pin to `high` (1) or `low` (0).

set_pin_mode/2

`Pin`: number to set operational mode`Direction`: is `input`, `output`, or `output_od`

returns: ok | error

Set the operational mode of a pin

Pins can be used for input, output, or output with open drain.

set_pin_pull/2

`Pin`: number to set internal resistor direction`Pull`: is the internal resistor state

returns: ok | error

Set the internal resistor of a pin

Pins can be internally pulled `up`, `down`, `up_down` (pulled in both directions), or left `floating`.

start/0

returns: Pid

Start the GPIO driver port

Returns the pid of the active GPIO port driver, otherwise the GPIO port driver will be stared and registered as `gpio`. The use of `gpio:open/0` or `gpio:start/0` is required before using any functions that require a GPIO pid as a parameter.

stop/0

returns: ok atom

Stop the GPIO interrupt port

This function disables any interrupts that are set, stops the listening port, and frees all of its resources.

Module http_server

- Function Index
- Section 9.4.2

Function Index

Function Details

parse_query_string/1

```
parse_query_string(L) -> any()
```

reply/3

```
reply(StatusCode, ReplyBody, Conn) -> any()
```

reply/4

```
reply(StatusCode, ReplyBody, ReplyHeaders, Conn) -> any()
```

start_server/2

```
start_server(Port, Router) -> any()
```

Module i2c

- Description
- Data Types
- Function Index
- Section 9.4.2

AtomVM I2c interface.

Description

This module provides and interface into the AtomVM I2C driver.

Use this module to communicate with devices connected to your ESP32 device via the 2-wire I2C interface.

Using this interface, you can read or write data to an I2C device at a given I2C address. In addition, you may read from or write to specific registers on the I2C device.

Data Types

address()

freq_hz()

i2c()

param()

params()

pin()

register()

Function Index

Function Details

begin_transmission/2

`I2C`: I2C instance created via `open/1``Address`: I2C Address of the device (typically fixed for the device type)

returns: `ok` or `{error, Reason}`

Begin a transmission of I2C commands

This command is typically followed by one or more calls to `write_byte/2` and then a call to `end_transmission/1`

close/1

`I2C`: I2C instance created via `open/1`

returns: `ok` atom

Closes the connection to the I2C driver

This function will close the connection to the I2C driver and free any resources in use by it.

end_transmission/1

`I2C`: I2C instance created via `open/1`

returns: `ok` or `{error, Reason}`

End a transmission of I2C commands

This command is typically preceded by a call to `begin_transmission/2` and one or more calls to `write_byte/2`.

open/1

`Param`: Initialization parameters

returns: process id of the driver.

Open a connection to the I2C driver

This function will open a connection to the I2C driver.

read_bytes/3

`I2C`: I2C instance created via `open/1``Address`: I2C Address of the device (typically fixed for the device type)`Count`: The number of bytes to read

returns: `{ok, Data}` which includes the read binary data or `{error, Reason}`

Read a block of bytes from the I2C device.

This command is not wrapped in a `begin_transmission/2` and `end_transmission/1` call.

read_bytes/4

`I2C`: I2C instance created via `open/1``Address`: I2C Address of the device (typically fixed for the device type)`Register`: The register address in the device from which to read data`Count`: The number of bytes to read

returns: `{ok, Data}` which includes the read binary data or `{error, Reason}`

Read a block of bytes from the I2C device starting at a specified register address

This command is not wrapped in a `begin_transmission/2` and `end_transmission/1` call.

write_byte/2

`I2C`: I2C instance created via `open/1``Byte`: value to write

returns: `ok` or `{error, Reason}`

Write a byte to the device.

This command must be wrapped in a `begin_transmission/2` and `end_transmission/1` call.

write_bytes/2

`I2C`: I2C instance created via `open/1``Bytes`: value to write

returns: `ok` or `{error, Reason}`

Write a sequence of bytes to the device.

This command must be wrapped in a `begin_transmission/2` and `end_transmission/1` call.

write_bytes/3

`I2C`: I2C instance created via `open/1``Address`: I2C Address of the device (typically fixed for the device type)`BinOrInt`: The binary or byte value to write

returns: `ok` or `{error, Reason}`

Write a block of bytes to the I2C device.

This command is not wrapped in a `begin_transmission/2` and `end_transmission/1` call.

write_bytes/4

`I2C`: I2C instance created via `open/1``Address`: I2C Address of the device (typically fixed for the device type)`Register`: The register address in the device to which to write data`BinOrInt`: The binary or byte value to write

returns: `ok` or `{error, Reason}`

Write a block of bytes to the I2C device starting at a specified register address.

This command is not wrapped in a `begin_transmission/2` and `end_transmission/1` call.

Module json_encoder

- Description

- Function Index
- Section 9.4.2

JSON specific APIs.

Description

This module contains functions for working with json data.

Function Index

Function Details

encode/1

`Data`: data to encode to json

returns: JSON encoded data

Convert data to json encoded binary

Module ledc

- Description
- Data Types
- Function Index
- Section 9.4.2

LED Controller low-level APIs.

Description

The functions in this module broadly reflect the ESP IDF-SDK LED Controller API.

See the IDF-SDK LEDC documentation for more information about these APIs.

Data Types

channel()

channel_cfg()

channel_config()

duty()

duty_cfg()

duty_resolution()

duty_resolution_cfg()

fade_mode()

freq_hz()

freq_hz_cfg()

gpio_num()

gpio_num_cfg()

hpoint()

hpoint_cfg()

ledc_error_code()

speed_mode()

speed_mode_cfg()

timer_config()

timer_num()

timer_num_cfg()

timer_sel()

timer_sel_cfg()

Function Index

Function Details

channel_config/1

`Config`: channel configuration

returns: ok | {error, ledc_error_code()}

LEDC channel configuration.

Configure LEDC timer with the given source timer/frequency(Hz)/duty_resolution.

fade_func_install/1

`Flags`: Flags used to allocate the interrupt. One (or multiple, using an ORred mask) ESP_INTR_FLAG_* values. See esp_intr_alloc.h for more info.

returns: ok | {error, ledc_error_code()}

Install LEDC fade function.

This function will occupy interrupt of LEDC module.

fade_func_uninstall/0

returns: ok

Uninstall LEDC fade function.

fade_start/3

`SpeedMode`: Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.`Channel`: LEDC channel index (0-7).`FadeMode`: Whether to block until fading done.

returns: ok | {error, ledc_error_code()}

Start LEDC fading.

Note. Call ledc:fade_func_install() once before calling this function. Call ledc:fade_start() after this to start fading.

get_duty/2

`SpeedMode`: Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.`Channel`: LEDC channel index (0-7).

returns: ok | {error, ledc_error_code()}

LEDC get duty.

get_freq/2

`SpeedMode`: Select the LEDC channel group with specified speed mode.`TimerNum`: LEDC timer index (0-3).

returns: ok | {error, ledc_error_code()}

LEDC get channel frequency (Hz)

set_duty/3

`SpeedMode`: Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.`Channel`: LEDC channel index (0-7).`Duty`: Set the LEDC duty, the range of setting is [0, (2^duty_resolution)-1].

returns: ok | {error, ledc_error_code()}

LEDC set duty.

set_fade_with_step/5

`SpeedMode`: Select the LEDC channel group with specified speed mode. Note that not all targets

support high speed mode.`Channel`: LEDC channel index (0-7).`TargetDuty`: Target duty of fading. (0..(2^duty_resolution)-1)`Scale`: Controls the increase or decrease step scale.`CycleNum`: increase or decrease the duty every cycle_num cycles

returns: ok | {error, ledc_error_code()}

Set LEDC fade function

Note. Call ledc:fade_func_install() once before calling this function. Call ledc:fade_start() after this to start fading.

set_fade_with_time/4

`SpeedMode`: Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.`Channel`: LEDC channel index (0-7).`TargetDuty`: Target duty of fading. (0..(2^duty_resolution)-1)`MaxFadeTimeMs`: The maximum time of the fading (ms).

returns: ok | {error, ledc_error_code()}

Set LEDC fade function, with a limited time.

Note. Call ledc:fade_func_install() once before calling this function. Call ledc:fade_start() after this to start fading.

set_freq/3

`SpeedMode`: Select the LEDC channel group with specified speed mode.`TimerNum`: LEDC timer index (0-3).`FreqHz`: Set the LEDC frequency.

returns: ok | {error, ledc_error_code()}

LEDC set channel frequency (Hz)

stop/3

`SpeedMode`: Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.`Channel`: LEDC channel index (0-7).`IdleLevel`: Set output idle level after LEDC stops.

returns: ok | {error, ledc_error_code()}

LEDC stop. Disable LEDC output, and set idle level.

timer_config/1

`Config`: timer configuration

returns: ok | {error, Reason}

LEDC timer configuration.

Configure LEDC timer with the given source timer/frequency(Hz)/duty_resolution.

update_duty/2

`SpeedMode`: Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.`Channel`: LEDC channel index (0-7).

returns: ok | {error, ledc_error_code()}

LEDC update channel parameters.

Module network

- Data Types
- Function Index
- Section 9.4.2

Data Types

ap_config()

ap_config_property()

ap_max_connections_config()
ap_ssid_hidden_config()
ap_sta_connected_config()
ap_sta_disconnected_config()
ap_sta_ip_assigned_config()
ap_started_config()
dhcp_hostname_config()
ip_info()
ipv4_address()
ipv4_info()
mac()
network_config()
octet()
psk_config()
sntp_config()
sntp_config_property()
sntp_host_config()
sntp_synchronized_config()
ssid_config()
sta_config()
sta_config_property()
sta_connected_config()
sta_disconnected_config()
sta_got_ip_config()

Function Index

Function Details

start/1

`Config`: The network configuration

returns: ok, if the network interface was started, or {error, Reason} if a failure occurred (e.g., due to malformed network configuration).

Start a network interface.

This function will start a network interface, which will attempt to connect to an AP endpoint in the background. Specify callback functions to receive definitive information that the connection succeeded. See the AtomVM Network FSM Programming Manual for more information.

start_link/1

stop/0

returns: ok, if the network interface was stopped, or {error, Reason} if a failure occurred.

Stop a network interface.

wait_for_ap/0

Equivalent to wait_for_ap(15000).

wait_for_ap/1

`TimeoutOrApConfig`: The AP network configuration or timeout in ms.

Equivalent to wait_for_ap([], Timeout) or wait_for_ap(StaConfig, 15000).

wait_for_ap/2

`ApConfig`: The AP network configuration`Timeout`: amount of time in milliseconds to wait for

a connection

returns: ok, when the network has started the AP, or {error, Reason} if a failure occurred (e.g., due to malformed network configuration).

Start a network interface in access point mode and wait the AP to be up and running

This function will start a network interface in AP mode, and will wait until the network is up and ready to be connected. This is a convenience function, for applications that do not need to be notified of connectivity changes in the network.

wait_for_sta/0

Equivalent to wait_for_sta(15000).

wait_for_sta/1

`TimeoutOrStaConfig`: The STA network configuration or timeout in ms.

Equivalent to wait_for_sta([], Timeout) or wait_for_sta(StaConfig, 15000).

wait_for_sta/2

`StaConfig`: The STA network configuration`Timeout`: amount of time in milliseconds to wait for a connection

returns: {ok, IpInfo}, if the network interface was started, or {error, Reason} if a failure occurred (e.g., due to malformed network configuration).

Start a network interface in station mode and wait for a connection to be established

This function will start a network interface in station mode, and will wait for a connection to be established. This is a convenience function, for applications that do not need to be notified of connectivity changes in the network.

Module network_fsm

- Description
- Function Index
- Section 9.4.2

network_fsm.

Description

This module is depreciated. Use the network module instead.

Function Index

Function Details

start/1

```
start(Config) -> any()
```

stop/0

```
stop() -> any()
```

wait_for_ap/0

```
wait_for_ap() -> any()
```

wait_for_ap/1

```
wait_for_ap(ApConfig) -> any()
```

wait_for_ap/2

```
wait_for_ap(ApConfig, Timeout) -> any()
```

wait_for_sta/0

`wait_for_sta() -> any()`

wait_for_sta/1

`wait_for_sta(StaConfig) -> any()`

wait_for_sta/2

`wait_for_sta(StaConfig, Timeout) -> any()`

Module pico

- Description
- Function Index
- Section 9.4.2

PICO-specific APIs.

Description

This module contains functions that are specific to the PICO platform.

Function Index

Function Details

rtc_set_datetime/1

Set the datetime on the RTC. The datetime can be obtained through bif erlang:localtime()

Module port

- Description
- Function Index
- Section 9.4.2

AtomVM port driver APIs.

Description

This module contains functions that are intended to be used by drivers that rely on a `port` interface rather than `niffs`.

The port driver should be initialized with: `open_port({spawn, "Name"}, Param)` Where Name is an atom(), and is the name of the driver. The return from `open_port/2` will be the Pid that will be required for future `port:call/2` or `port:call/3` use.

Examples:

```
open_port({spawn, "i2c"}, Param)
```

or

```
open_port({spawn, "spi"}, Params)
```

Function Index

Function Details

call/2

`Message`: the message to send

returns: term() | {error, Reason}.

Send a message to a given port driver pid.

This function is used to send a message to an open port divers pid and will return a term or `{error, Reason}`.

call/3

`Message`: the message to send`Timeout`: the timeout value in milliseconds

returns: term() | {error, Reason}.

Send a message to a given port driver pid with a timeout.

This function is used to send a message to an open port divers pid and will return a term or `{error, Reason}`, or`{error, timeout}` if the TimeoutMs is reached first.

Module spi

- Description
- Data Types
- Function Index
- Section 9.4.2

This module provides an interface into the Serial Peripheral Interface (SPI) supported on many devices.

Description

This module currently support the SPI "leader" (historically known as the "master") interface, allowing the leader to connect to one or more "follower" (historically known as "slave") devices.

Users interact with this interface by creating an instance of the driver via the `open/1` function, with returns an opaque reference to the driver instance. The `open/1` function takes a complex map structure, which configures the driver to connect to follower devices. See the `open/1` documentation for details about the structure of this configuration map.

Subsequent read and write operations use the SPI instance returned from the `open/1` function. Users may read from a specific follower device at a specific address, write to the device at an address, or simultaneously read from and write to the device in a single transaction.

Data Types

address()

bus_config()

device_config()

device_name()

params()

spi()

spi_peripheral()

transaction()

Function Index

Function Details

close/1

`SPI`: SPI instance created via `open/1`

Close the SPI driver.

Close the SPI driver and free any resources in use by the driver.

The SPI instance will no longer be valid and usable after this function has been called.

open/1

`Params`: Initialization parameters

returns: process id of the driver.

throws `badarg`

Open a connection to the SPI driver

This function will open a connection to the SPI driver.

Supply a set of parameters to initialize the driver.

The parameters list must contain an SPI Bus configuration, together with a properties list containing one or more device configurations. This list must contain atom keys as names, which are used to identify the device in the subsequent read and write operations. You may use any atom value of your choosing.

The SPI Bus configuration is a properties list containing the following entries:

Each device configuration is a properties list containing the following entries:

Example:

```
Params = [
   {bus_config, [
       {miso_io_num, 16},
       {mosi_io_num, 17},
       {sclk_io_num, 5}
   },
   {device_config, [
       {device1, [
           {spi_cs_io_num, 18}
       ]},
       {device2, [
           {spi_cs_io_num, 19}
       ]}
   ]}
]
```

Note that `device1` and `device2` are atom names used to identify the device for read and write operations.

This function raises an Erlang exception with a `badarg` reason, if initialization of the SPI Bus or any device fails.

The `write/3` and `write_read/3` functions in this module are designed to provide the maximum mount of flexibility when interfacing with the SPI device. The both make use of a map structure to encapsulate an SPI transaction.

An SPI transaction may contain a command, and address, and/or a blob of data, each of which is optional and each of which depends on how users interact with the device. Consult the data sheet for your SPI device to understand which fields should be used with your device.

The fields of a transaction map are as follows:

read_at/4

`SPI`: SPI instance created via `open/1``DeviceName`: device name from configuration`Address`: SPI Address from which to read`Len`: in bytes to read

returns: `{ok, Value}` or `error`

Read a value from and address on the device.

write/3

`SPI`: SPI instance created via `open/1``DeviceName`: SPI device name (use key in `device_config`)`Transaction`: transaction map.

returns: `ok` or `{error, Reason}`, if an error occurred.

Write data to the SPI device, using the instructions encoded in the supplied transaction.

The supplied `Transaction` encodes information about how data is to be written to the selected SPI device. See the description above for the fields that may be specified in this map.

When a binary is supplied in the `write_data` field, the data is written to the SPI device in the natural order of the binary. For example, if the input binary is `<<16#57, 16#BA>>`, then the first byte is `0x57` and the second byte is `0xBA`.

The value of the `write_bits` field, if specified, must be less than or equal to `8 * byte_size(write_data)`. If `write_bits` is less than `8 * byte_size(write_data)`, only the first `write_bits` bits from `write_data` will be written.

This function will return a tuple containing the `error` atom if an error occurred writing to the SPI device at the specified address. The returned reason term is implementation-defined.

write_at/5

`SPI`: SPI instance created via `open/1``DeviceName`: device name from configuration`Address`: SPI Address to which to write`Len`: in bytes to read`Data`: byte(s) to write

returns: `{ok, Value}` or `error`

Write a value to and address on the device.

The value returned from this function is dependent on the device and address. Consult the documentation for the device to understand expected return values from this function.

write_read/3

`SPI`: SPI instance created via `open/1``DeviceName`: SPI device name (use key in `device_config`)`Transaction`: transaction.

returns: `{ok, binary()}` or `{error, Reason}`, if an error occurred.

Write data to the SPI device, using the instructions encoded in the supplied transaction. device, and simultaneously read data back from the device, returning the read data in a binary.

The supplied `Transaction` encodes information about how data is to be written to the selected SPI device. See the description above for the fields that may be specified in this map.

When a binary is supplied in the `write_data` field, the data is written to the SPI device in the natural order of the binary. For example, if the input binary is `<<16#57, 16#BA>>`, then the first byte is `0x57` and the second byte is `0xBA`.

The value of the `write_bits` field, if specified, must be less than or equal to `8 * byte_size(write_data)`. If `write_bits` is less than `8 * byte_size(write_data)`, only the first `write_bits` bits from `write_data` will be written.

The return value contains a sequence of bytes that have been read from the SPI device. The number of bytes returned will be `ceil(read_bits / 8)`. Only the first `read_bits` will be populated.

This function will return a tuple containing the `error` atom if an error occurred writing to the SPI device at the specified address. The returned reason term is implementation-defined.

Module timestamp_util

- Description
- Data Types
- Function Index
- Section 9.4.2

Utility functions for comparing timestamps.

Description

This module contains functions that are useful for comparing timestamps without running the risk of integer overflow.

Note that the functions in this module may be obsoleted in future versions of AtomVM, as support for

arbitrary sized integers is added; however, the functions may still be useful in their own right.

Data Types

megasecs()

microsecs()

secs()

timestamp()

Function Index

Function Details

delta/2

`TS2`: a timestamp`TS1`: a timestamp

returns: TS2 - TS1, as a timestamp

Computes the difference between TS2 and TS1, as a timestamp.

delta_ms/2

`TS2`: a timestamp`TS1`: a timestamp

returns: TS2 - TS1, in milliseconds

Computes the difference between TS2 and TS1, in milliseconds.

Module uart

- Function Index
- Section 9.4.2

Function Index

Function Details

close/1

```
close(Pid) -> any()
```

open/2

```
open(Name, Opts) -> any()
```

read/1

```
read(Pid) -> any()
```

write/2

```
write(Pid, B) -> any()
```

11.1.3 alisp

The alisp library

Modules

Module alisp

- Function Index
- Section 9.4.2

Function Index

Function Details

booleanize/1

```
booleanize(V) -> any()
```

eval/1

```
eval(S) -> any()
```

run/1

```
run(S) -> any()
```

Module alisp_stdlib

- Function Index
- Section 9.4.2

Function Index

Function Details

''/1*

```
*(L) -> any()
```

'+'/1

```
+(L) -> any()
```

'-'/1

```
-(T) -> any()
```

'='/1

```
=(T) -> any()
```

'remove-if'/1

```
remove-if(X1) -> any()
```

'remove-if-not'/1

```
remove-if-not(X1) -> any()
```

append/1

```
append(X1) -> any()
```

binaryp/1

```
binaryp(X1) -> any()
```

car/1

```
car(X1) -> any()
```

cdr/1

```
cdr(X1) -> any()
```

cons/1

```
cons(X1) -> any()
```

floatp/1

```
floatp(X1) -> any()
```

identity/1

```
identity(X1) -> any()
```

integerp/1

```
integerp(X1) -> any()
```

last/1

```
last(X1) -> any()
```

list/1

```
list(List) -> any()
```

listp/1

```
listp(X1) -> any()
```

mapcar/1

```
mapcar(X1) -> any()
```

numberp/1

```
numberp(X1) -> any()
```

pidp/1

```
pidp(X1) -> any()
```

print/1

```
print(X1) -> any()
```

refp/1

```
refp(X1) -> any()
```

tuple/1

```
tuple(List) -> any()
```

tuplep/1

```
tuplep(X1) -> any()
```

Module arepl

- Function Index
- Section 9.4.2

Function Index

Function Details

start/0

```
start() -> any()
```

Module sexp_lexer

- Function Index
- Section 9.4.2

Function Index

Function Details

string/1

```
string(Bin) -> any()
```

Module sexp_parser

- Function Index
- Section 9.4.2

Function Index

Function Details

parse/1

```
parse(T) -> any()
```

Module sexp_serializer

- Function Index
- Section 9.4.2

Function Index

Function Details

serialize/1

```
serialize(Value) -> any()
```

11.1.4 etest

The etest library

Modules

Module etest

- Description
- Function Index
- Section 9.4.2

This modules provides a basic testing framework for AtomVM Erlang libraries.

Function Index

Function Details

assert_equals/2

`X`: a term`Y`: a term

returns: ok if X and Y are equal; fail otherwise.

assert_failure/1

`F`: a function to evaluate

returns: ok if evaluating F results in Error being thrown; fail, otherwise

assert_failure/2

`F`: a function to evaluate

returns: ok if evaluating F results in Error being thrown; fail, otherwise

assert_match/2

`X`: a term

returns: ok if X and Y unify; fail otherwise.

assert_true/1

returns: ok if X is true; fail otherwise.

test/1

`Tests`: a list of test modules

returns: ok if all of the tests pass, or the atom fail, if any of the tests failed.

Test a sequence of test modules.

This function will execute the test/0 function for each module provided in the input list of test modules. If all of the tests return the atom ok, then this function returns ok. If any of the test modules return a value other than ok, then this function returns the atom fail.

11.2 AtomVM 'C' Internal Libraries

11.2.1 libAtomVM

[libAtomVM (Doxygen theme)]

11.2.2 libAtomVM Source Files

[libAtomVM (Doxygen theme)]

Chapter 12

Contributing

AtomVM is open to any contribution.

Pull requests, bug reports and feature requests are welcome.

However before contributing, please read carefully our Code of Conduct and the following contribution guidelines.

Please, also make sure to understand the Apache 2.0 license and the Developer Certificate of Origin.

Last but not least, **do not use GitHub issues for vulnerability reports**, read instead the security policy for instructions.

12.1 Git Recommended Practices

- Commit messages should have a
- summary and a description
- Remove any trailing white spaces
- Always `git pull --rebase`
- Clean up your branch history with `git rebase -i`
- Squash commits before PR, unless there is a good reason not to
- All your intermediate commits should build

12.2 Coding Style

For all source code modules:

- Remove all trailing whitespace
- Newlines (`\n`) at end of file
- Use line ending conventions appropriate for the platform (e.g., `\n` on UNIX-like systems)

12.2.1 Copyright Headers

All source code modules should include copyright headers that are formatted for the relevant module language. Copyright headers should take the following form:

```
/*
* This file is part of AtomVM.
*
```

```
* Copyright 2020 Your name <your@email.address>
*
* Licensed under the Apache License, Version 2.0 (the "License");
* you may not use this file except in compliance with the License.
* You may obtain a copy of the License at
*
*    http://www.apache.org/licenses/LICENSE-2.0
*
* Unless required by applicable law or agreed to in writing, software
* distributed under the License is distributed on an "AS IS" BASIS,
* WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
* See the License for the specific language governing permissions and
* limitations under the License.
*
* SPDX-License-Identifier: Apache-2.0 OR LGPL-2.1-or-later
*/
```

12.2.2 C Code

C source code style is enforced with clang-format-13. To automatically fix a file, run:

```
clang-format-13 --style=file -i file.c
```

Indentation

- K&R identation and braces style
- Mandatory braces
- 4 space indentation (no tabs)

Good:

```
void f(bool reverse)
{
    if (reverse) {
        puts("!dlroW olleH");
    } else {
        puts("Hello world");
    }
}
```

Bad:

```
void f(bool reverse) {
    if (reverse)
        puts ("!dlroW olleH");
    else
        puts ("Hello world");
}
```

Naming Conventions

- Struct names are PascalCase (e.g. `Context`)
- Scalar types are lower case (e.g. `term`)
- All other names (e.g. functions and variables) are snake_case (e.g. `term_is_integer`)
- Always prefix exported function names with the module in which they are defined (e.g. `term_is_nil`, `term_is_integer`, `context_new`, `context_destroy`)

Other Coding Conventions

- Pointers (`*`) should be with the variable name rather than with the type (e.g. `char *name`, not

char* name)

- Avoid long lines, use intermediate variables with meaningful names.
- Function definitions should be separated by 1 empty line

Function declarations should be structured as follows:

```
func(main_input, additional_inputs, main_output, additional_outputs, opts,
[context])
```

where `context` is a context structure (such as `Context` or `GlobalContext`).

Any functions that are not exported should be qualified with the `static` keyword.

Functions that return booleans should be named `is_something` (or possibly `module_is_something`, if the function is exported).

C header modules (`.h`) should be organized as follows:

```
+-------------------
| Copyright Header
|
| #ifdef MODULE__H__
| #define MODULE__H__
|
| #ifdef __cplusplus
| extern "C" {
| #endif
|
| #includes (alphabetical)
|
| #defines
|
| type definitions
|
| function declarations
|
| #ifdef __cplusplus
| }
| #endif
|
| #endif
+-------------------
 module.h
```

C source modules (`.c`) should be organized as follows:

```
+-------------------
| Copyright Header
|
| #includes (alphabetical)
|
| #defines
|
| type definitions
|
| forward declarations (only if necessary)
|
| function definitions
|   dependent static functions first
|   exported functions and entrypoints last
+-------------------
 module.c
```

Documentation

Doxygen Javadoc style code comments will be picked up and added to the documentation. Changes will automatically be added to the libAtomVM Source Files and the libAtomVM Index. But to have `Data Strucures`, `Types`, `MACROS`, and `Functions` appear in the correct C Library APIs section the corresponding entries must be added to the similarly named `*.rst` files in the `AtomVM/doc/src/apidocs/libatomvm/` directory. The exact names of the flies that need to be altered are: `data_structures.rst`, `functions.rst`, `macros.rst`, and `types.rst`. The other files in the directory handle auto`generated content and do not need to be altered.

In the rare case that a function declaration and definition are both in different header files (rather than the definition in a `*.c` file) this can cause rendering errors for `Doxygen`. The work around for these cases can be demonstrated with this example for the function `sys_listener_destroy` it is documented and declared in `sys.h` and defined as follows in `listeners.h`:

```
#ifndef DOXYGEN_SKIP_SECTION /* documented in sys.h */
void sys_listener_destroy(struct ListHead *item)
{
    EventListener *listener = GET_LIST_ENTRY(item, EventListener,
listeners_list_head);
    free(listener);
}
#endif /* DOXYGEN_SKIP_SECTION */
```

> Note: You should include a short `/* comment */` trailing the `#ifndef` entry mentioning the file where the function is actually documented.

12.2.3 Erlang Code

Erlang source code style is enforced using erlfmt.

12.2.4 Elixir Code

Just use Elixir formatter enforced style.

Chapter 13

Changelog

All notable changes to this project will be documented in this file.

The format is based on Keep a Changelog, and this project adheres to Semantic Versioning.

13.1 [0.6.0-alpha.1] - Unreleased

13.1.1 Added

- Added configurable logging macros to stm32 platform
- Added support for ULP wakeup on ESP32
- Added heap growth strategies as a fine-tuning option to `spawn_opt/2,4`
- Added `crypto:crypto_one_time/4,5` on ESP32
- Improved nif and port support on STM32

13.1.2 Fixed

- Fixed `esp:nvs_set_binary` functions.
- Fixed `monotonic_time/1` and `system_time/1` functions for Raspberry Pi Pico
- Fixed race conditions in atoms table.
- Fixed a bug in the STM32 port that caused the final result to never be returned.

13.2 [0.6.0-alpha.0] - 2023-08-13

13.2.1 Added

- Added the ability to specify the HSPI or VSPI ESP32 hardware interfaces when initializing the SPI Bus.
- Added support for the `spi:close/1` function.
- Added `AVM_VERBOSE_ABORT` CMake define, which when set to on, will print the C module and line number when a VM abort occurs. This define is off by default.
- Added `spi:write/3` and `spi:write_read/3` functions to support generalized SPI transactions and arbitrary-length reads and writes from SPI devices.
- Added support for building ESP32 port with all currently supported versions of Espressif ESP-IDF, version 4.1.x through 4.4.x.

- Added support for `controlling_process/2` in `gen_udp` and `gen_tcp` modules.
- Added ability to get the atomvm version via `erlang:system_info`.
- Added `erlang:is_boolean/1` Bif.
- Added support for `esp:partition_erase_range/2`
- Added support for `i2c:close/1`
- Added support for `erlang:unregister/1`
- Added Elixir ESP32 LEDC driver and example
- Added support for `uart:close/1`
- Added Bitwise support for Elixir
- Added support for esp32-s2, esp32-s3, and esp32-c3 chips.
- Added Elixir I2C driver and example
- Added the ability to specify the I2C port
- Added support for the OTP `math` module
- Added support for `erlang:integer_to_list/2` and `erlang:integer_to_binary/2`
- Added functions `esp:sleep_enable_ext0_wakeup/2` and `esp:sleep_enable_ext1_wakeup/2.`
- Added support for FP opcodes 94-102 thus removing the need for `AVM_DISABLE_FP=On` with OTP-22+
- Added support for stacktraces
- Added support for `utf-8`, `utf-16`, and `utf-32` bit syntax modifiers (put and match)
- Added support for Erlang `gpio:close/1` and Elixir `GPIO.close/1` for ESP32
- Added support for the Erlang `gen_event` module
- Added `start_link` support for the `network` module
- Added support for `erlang:monotonic_time/1`
- Added `start_link` support for the `gen_statem` module
- Added support for serializing floats in erlang external term encoding
- Added support for the `SMALL_BIG_EXT` erlang external term encoding
- Added support for `erlang:memory(binary)`
- Added support for callbacks on SNTP updates
- Multithreading support (SMP)
- Added support for code:load_abs/1, code:load_binary/3
- Added support for loading / closing AVMPacks at runtime
- Added support for ESP-IDF v5.x
- Added support for `calendar:system_time_to_universal_time/2`
- Added support for `calendar:datetime_to_gregorian_seconds/1`
- Added support for Raspberry Pi Pico
- Added support for nodejs with Wasm
- Added support for a subset of the OTP logger interface
- Added `esp:partition_list/0` function
- Added `esp:nvs_fetch_binary/2` and `nvs_put_binary/3` functions (`esp:nvs_set_binary` and functions that default to `?ATOMVM_NVS_NS` are deprecated now).
- Added most format possibilities to `io:format/2` and `io_lib:format/2`

- Added `unicode` module with `characters_to_list/1,2` and `characters_to_binary/1,2,3` functions
- Added support for `crypto:hash/2` (ESP32 and generic_unix with openssl)
- Added erlang:spawn_link/1,3
- Added erlang:exit/2
- Added links to process_info/2
- Added lists:usort/1,2
- Added missing documentation and specifications for available nifs

13.2.2 Fixed

- Fixed issue with formatting integers with io:format() on STM32 platform
- Fixed a bug in the order of child initialization in the `supervisor` module
- Fixed a bug in the evaluation of `receive ... after infinity -> ...` expressions
- Fixed a bug in when putting integers in bit syntax with integer field sizes
- Fixed numerous bugs in memory allocations that could crash the VM
- Fixed SNTP support that had been broken in IDF 4.x builds
- Fixed `erlang:send/2` not sending to registered name
- Fixed incorrect exit reason for exceptions of class exit
- Fixed several incorrect type specifications

13.2.3 Breaking Changes

IMPORTANT: These changes are incompatible with previous releases of AtomVM.

- Changed the configuration model of the SPI driver, in order to allow for multiple "follower" devices to be attached to the same SPI Bus.
- Changed the return value from `erlang:system_info(esp32_chip_info)` from a tuple to a map, with additional information.
- Changed the return type of the `network:start` function to return the tuple `{ok, Pid}` on a successful call, instead of the bare atom `ok`. Applications that use `network:start` and check the return value will need to be modified.
- The return type of `i2c:read_bytes` has changed from returning just a binary to returning the tuple `{ok, Binary}` when successful.
- The return type of many `i2c` operations under error conditions has changed from `error` to `{error, Reason}`, for improved diagnostics.
- The eavmlib logger interface has been removed

13.2.4 Removed

- ESP-IDF v3.x support.

13.3 [0.5.1] - Unreleased

13.3.1 Added

- New function for atom comparison, useful when writing 3rd party components.
- New function for translating an atom term to an int value, according to a given translation table. This function can be used for translating an atom term to an enum const before doing a switch.
- New no-op `ATOM_STR(...)` macro for avoiding issues with clang-format.
- [ESP32] `REGISTER_PORT_DRIVER` for registering additional port drivers without editing any source file. This allows adding new components by just copying them to the components directory.
- [ESP32] `REGISTER_NIF_COLLECTION` for registering additional NIFs sets without editing any source file. This allows adding new NIFs by just copying them to the components directory.
- New function for getting a map or proplist value using an atom string without poluting the atom table.

13.3.2 Fixed

- Fix `gen_statem`: Cancel outstanding timers during state transitions in order to prevent spurious timeout messages from being sent to `gen_statem` process.
- Fix missing Elixir libraries: examvlib was not packed into atomvmlib.avm
- Fix `bs_context_to_binary`: match offset wasn't used, leading in certain situations to infinite loops while matching binaries.
- Fix how `start` option was handled from `bs_restore2` instruction: last saved match offset was used instead of match starting offset, causing some bytes being skipped.
- Fix another potential bug when doing pattern matching using code compiled with OTP 21.
- [ESP32] [UART]: Allow using different pins for rx, tx, cts and rts.
- [ESP32] [UART]: Replace custom UART handling with esp-idf UART event queues, hence other UARTs than UART0 are supported, with better performances and stability.
- Fix binaries concat (`bs_append` instruction) that was adding some extra zeroes at the end of built binaries.
- Fixed a bug in `gen_tcp` that prevents an accepting socket from inheriting settings on the listening socket.
- Fixed a bug in packing and unpacking integers into and from binaries when the bit length is not a multiple of 8.
- Fixed `esp:deep_sleep/1` that did not accept values above 31 minutes.
- Fixed a bug that could cause processes to hang indefinitely when calling ports that have terminated.
- Fixed potential VM crash when parsing external terms.
- Fixed the enforcement of `min_free_space` process option.

13.4 [0.5.0] - 2022-03-22

- genindex

- modindex
- search